Jedi 1.0

Manuel utilisateur

Octobre 2026

Sommaire

Lire cette documentation 2

La philosophie de Jedi 4

Utiliser Jedi au quotidien 6

1. Jedi en quelques pages 7

2. Installer et préparer Jedi 9

3. Choisir un titre-support 18

4. Présélectionner avec tools profile 20

5. Chercher un barème avec tools score 24

6. Évaluer une ouverture avec tools candidate 28

7. Ouvrir un trade 32

8. Suivre un trade au quotidien 35

9. Adapter ou terminer un trade 39

10. Gérer la base de cotations 44

Comprendre la mécanique de Jedi 48

11. Allers-retours et flux du trade 49

12. Positions, échelons et paliers 51

13. Lire la structure d'un barème 53

14. Comprendre un barème de type I 56

15. Comprendre un barème de type II 58

16. Assurer la continuité entre paramétrages 60

17. Passer de l'état du trade aux ordres 63

18. Comprendre le capital, le risque et le dimensionnement 65

19. Vérifier avec tools calc et tools play 69

20. Suivre un exemple complet de trade 72

Étudier les résultats historiques 75

21. Préparer une étude historique 76

22. Simuler un barème avec tools backtest 78

23. Exemple d'étude sur Euronext Growth 81

24. Interpréter, comparer et suivre les résultats 84

Annexe 1. Référence des commandes et options 86

Annexe 2. Configuration des fournisseurs de cours API et Web 91

Annexe 3. Script de sélection externe ProScreener JEDI 96

Annexe 4. Glossaire et notations Jedi 100

Annexe 5. Configuration par défaut et usage des paramètres 105

Lire cette documentation

Cette édition accompagne **`next-version`, en préparation de Jedi 1.1**. Elle intègre `split + reinit=reset` et `split + reinit=merge`, ainsi que les conventions de nommage des trades et des barèmes. Ces combinaisons demandent le binaire mis à jour ; elles restent refusées par la version publiée Jedi 1.0.0. Les exemples d'installation du chapitre 02 utilisent encore la distribution publiée 1.0.0 : adapter la version lors d'une publication ultérieure.

Jedi s'adresse à l'investisseur qui souhaite appliquer et suivre une méthode de trading mécanique. Pour une première lecture, commencer par La philosophie de Jedi, juste avant la partie I : principe de la méthode, discipline, décisions de l'investisseur et limites des résultats historiques.

Trois parcours de lecture

La **partie I**, chapitres 01 à 10, accompagne la pratique : installer Jedi, choisir un titre, étudier un barème, évaluer une ouverture et suivre le trade. Pour une première utilisation, lire les chapitres 01 et 02, puis reproduire les exemples dans l'espace de démonstration. Les chapitres 03 à 06 préparent un projet ; les chapitres 07 à 09 enregistrent des opérations. Le chapitre 10 explique les bases de cotations et prépare la base Demo utilisée dès le chapitre 04.

La **partie II**, chapitres 11 à 20, explique les règles : allers-retours, positions, barèmes, continuité, ordres et capital. Les commandes `tools calc` et `tools play` permettent de vérifier la mécanique avec des mouvements choisis. Le chapitre 20 rassemble ces notions dans un parcours complet.

La **partie III**, chapitres 21 à 24, organise les études historiques : préparation des données, backtest, exemple chronologique et interprétation. Elle distingue la recherche d'un barème, sa vérification sur une autre période et le suivi de décisions datées.

Les annexes regroupent la référence des commandes, la configuration des fournisseurs, le script ProScreener JEDI et le glossaire. Elles permettent de retrouver un détail sans interrompre la lecture principale.

Conventions des exemples

Les commandes sont écrites pour Bash sous Linux ou Ubuntu dans WSL. Les noms de commandes et d'options restent en anglais, quelle que soit la langue des messages. Les prix des exemples sont en euros. `10%` et `0.10` expriment un même coefficient lorsque le champ accepte une valeur décimale ou un pourcentage.

Une barre oblique inverse en fin de ligne prolonge une commande sur la ligne suivante. Dans un texte entre guillemets, les retours à la ligne font partie du texte. Les signes `$` et `>` montrés avec certaines saisies représentent les invites du terminal et ne sont pas à recopier.

Les exemples emploient **`jedidemo`**, le lanceur installé au chapitre 02. Il conserve les essais dans leur propre espace. Avec les lanceurs fournis, **`jedi` désigne toujours la production** ; **`jedidev`** sert aux développements. Les fichiers de `examples/` sont copiés dans le dossier de démonstration selon ce chapitre.

Les titres EXA, EXB, EXC et GAP, leurs identifiants et leurs cotations sont **fictifs**. Le calendrier des fichiers retient les jours de semaine, y compris certains jours fériés. Ils servent à reproduire les commandes et les montants, sans représenter un investissement réel.

Les réglages nécessaires sont indiqués avant les exemples : frais, capital de simulation et état de la base principale. Un portefeuille déjà engagé change notamment les réserves de `candidate`. Une commande de création suppose que l'objet ou la base n'existe pas encore.

Retrouver une information

Les numéros identifient les fichiers et les renvois. Dans les PDF régénérés avec cette édition, le sommaire reprend les trois parties, les 24 chapitres et les cinq annexes. Les versions française et anglaise suivent le même plan.

Pour connaître les possibilités de la version installée, lancer `jedi` sans argument, puis demander l'aide d'une sous-commande, par exemple `jedi tools backtest --help`. `jedi version` affiche la version ; `jedi config --paths` affiche les chemins réellement utilisés.

Chaque étude et chaque trade restent lisibles grâce à leurs données, leurs paramètres et leurs opérations. Conserver ces éléments permet d’expliquer un résultat et de reprendre le travail sans devoir deviner les choix antérieurs.

La philosophie de Jedi

Principe général

Jedi est une méthode de gestion mécanique d'un trade fondée sur des règles prédéfinies.

Une fois un trade ouvert et paramétré, le logiciel détermine les opérations à envisager en fonction :

- du barème utilisé ;
- de la position actuelle du trade ;
- des opérations déjà réalisées ;
- des règles de gestion du capital éventuellement activées.

Jedi ne cherche pas à prévoir le prochain mouvement du marché pour décider quotidiennement s'il faut acheter ou vendre.

Son principe est différent : définir à l'avance une mécanique d'intervention, puis l'appliquer de manière cohérente lorsque les niveaux prévus sont atteints.

Exploiter les fluctuations plutôt que les prévoir

Les cours n'évoluent généralement pas de manière continue dans une seule direction. Même au sein d'une tendance durable, des mouvements de hausse et de baisse se succèdent.

Jedi cherche à exploiter une partie de ces fluctuations au moyen d'achats et de ventes successifs.

Un aller-retour peut, par exemple, consister à vendre une quantité de titres après une hausse puis à la racheter à un cours inférieur.

Le différentiel de cours obtenu sur cet aller-retour est alors positif. Son résultat net dépend toutefois des frais, de la fiscalité éventuelle et des conditions réelles d'exécution.

La méthode ne suppose donc pas que l'utilisateur sache prévoir précisément la tendance future du titre.

Elle ne supprime pas pour autant le risque : une évolution défavorable prolongée, un manque de liquidité, des gaps importants ou un capital insuffisant peuvent notamment dégrader ou empêcher le fonctionnement attendu du trade.

Ce qui reste une décision de l'utilisateur

Jedi automatise des calculs et applique des règles, mais il ne décide pas de tout.

L'utilisateur reste notamment responsable :

- du choix du titre qu'il souhaite examiner ;
- de la décision d'envisager ou non un nouveau trade ;
- du choix final parmi les solutions proposées par les outils d'analyse ;
- de l'exécution effective des ordres chez son intermédiaire financier ;
- de la décision de poursuivre ou d'arrêter un trade lorsque cette décision lui appartient.

Les outils de Jedi fournissent des informations et une mécanique de gestion ; ils ne remplacent pas la décision d'investissement.

Une mécanique reproductible

L'un des objectifs essentiels de Jedi est de rendre les décisions opérationnelles reproductibles.

Pour une même situation et un même paramétrage, la mécanique doit conduire au même résultat.

Cette reproductibilité répond aussi à une difficulté humaine majeure du trading : la tendance à décider sous l'effet de la peur, de l'impatience ou de l'euphorie. Les règles sont définies avant que la situation se présente ; Jedi les applique ensuite sans les infléchir en fonction de l'émotion du moment.

Cette propriété permet :

- de limiter l'improvisation au cours de la vie du trade ;
- d'éviter que les décisions opérationnelles soient dictées par les émotions ;
- de conserver une trace précise des décisions et opérations ;
- de tester les règles sur des données historiques ;
- de comparer différentes configurations ;
- d'étudier leur comportement avec des méthodes reproductibles.

Jedi ne supprime pas toute décision humaine : l'utilisateur conserve les responsabilités décrites précédemment. Il sépare en revanche ces décisions de la réaction immédiate aux fluctuations du marché et aide ainsi à maintenir la discipline choisie.

Cette reproductibilité n'implique pas que les résultats financiers futurs soient eux-mêmes prévisibles.

Ce que Jedi ne garantit pas

Jedi ne garantit :

- ni qu'un titre sélectionné progressera ;
- ni qu'un barème sera profitable dans toutes les configurations de marché ;
- ni qu'un trade pourra toujours être financé ;
- ni qu'un résultat historique se reproduira dans le futur ;
- ni l'absence de pertes.

Les outils statistiques et les backtests servent à étudier le comportement des règles dans des conditions définies. Ils ne transforment pas ces observations en certitudes sur les résultats futurs.

La première partie de cette documentation peut maintenant se concentrer sur une question plus concrète : **comment utiliser Jedi dans la pratique ?**

Utiliser Jedi au quotidien

Cette partie suit le parcours d'un utilisateur, de la première idée de titre jusqu'au suivi d'un trade. Elle s'adresse à un investisseur qui veut comprendre les décisions à prendre et appliquer des règles régulières, sans devoir connaître les formules de Jedi.

Un **titre-support** est le titre acheté et vendu. Un **barème** définit les prix et quantités de la mécanique. Un **trade** regroupe cette gestion, ses réglages et ses opérations. Ces notions sont introduites avant leurs détails.

Le choix du titre peut venir d'une recherche personnelle, d'un screener externe ou de `profile`. `score` aide ensuite à chercher un barème sur l'historique choisi ; `candidate` évalue l'ouverture avec le prix et les ressources disponibles. L'utilisateur garde les choix et exécute les ordres chez son courtier.

Le chapitre 01 présente le parcours ; le chapitre 02 prépare l'installation. Les chapitres 03 à 06 conduisent du choix du titre à l'évaluation du projet. Les chapitres 07 à 09 montrent l'ouverture, le suivi et les adaptations, avec un journal des événements exceptionnels. Le chapitre 10 explique les fichiers de cotations et les bases de marché.

Les exemples utilisent les commandes Jedi et des données explicitement identifiées. Les fichiers de démonstration sont fictifs : ils servent à comprendre les sorties. Après cette partie, le lecteur saura où lire les informations utiles, quoi enregistrer et quels points vérifier avant de poursuivre.

La partie II expliquera la mécanique derrière ces commandes. La partie III montrera comment examiner les résultats sur des historiques.

1. Jedi en quelques pages

Les exemples ci-dessous utilisent le lanceur `jedidemo`, installé au chapitre 02. Il conserve leurs données dans un espace de démonstration séparé du portefeuille réel.

Une règle de conduite pour un trade

Jedi est une application en ligne de commande destinée à gérer des trades selon des règles définies à l'avance. Un **trade** regroupe les achats et les ventes d'un même titre, son barème, ses réglages et son historique.

Le **barème** relie les mouvements du cours à des quantités à acheter ou à vendre. À partir de l'état du trade, Jedi calcule les ordres à envisager. L'utilisateur les place chez son courtier, vérifie leur exécution et enregistre les opérations réellement réalisées.

Cette séparation permet de suivre une mécanique compréhensible : choisir les règles, observer les niveaux atteints, exécuter puis enregistrer. Une hausse ou une baisse prolongée, des frais élevés ou un manque de fonds peuvent toutefois rendre le parcours défavorable.

Exploiter les fluctuations

Un aller-retour peut consister à vendre quelques titres après une hausse, puis à les racheter lors d'une baisse. Voici un exemple **fictif**, calculé avec Jedi :

```
jedidemo --border=markdown tools play -c o0,6-14,17 -- \
  scale=vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0,q0=100,c0=10,tick=0.01 1 -1
```

L'environnement créé indique un barème, 100 titres achetés à 10 € et un pas de cotation de 0,01 €. Les arguments `1 -1` demandent un échelon à la hausse, puis un échelon à la baisse. Le double tiret sépare les options des arguments du scénario.

Dans la sortie obtenue, l'achat initial est suivi de :

Opération	Quantité	Prix	Montant	Stock après opération
Vente	6	11,00 €	66,00 €	94
Achat	6	9,90 €	59,40 €	100

Le stock retrouve 100 titres et l'aller-retour apporte 6,60 € avant frais. Le cours final est cependant de 9,90 € : il faut aussi valoriser les titres restants pour connaître le résultat global. Un aller-retour positif et un trade globalement positif sont deux constats différents.

`tools play` sert ici à expliquer la mécanique. Il ne lit aucun historique et ne passe aucun ordre chez le courtier. Les types I et II de barèmes sont développés dans la partie II.

Les trois décisions avant l'ouverture

Étape	Question	Moyen possible
Choisir un titre	Quel titre examiner ?	Choix personnel, screener externe ou `tools profile`
Choisir un barème	Avec quelles règles l'étudier ?	Barème déjà choisi ou recherche avec `tools score`
Évaluer le projet	Peut-on ouvrir et poursuivre avec ces quantités et ces fonds ?	`tools candidate`

Le choix du titre appartient à l'utilisateur. `profile` l'aide à établir une liste selon la tendance, la liquidité, la volatilité, l'oscillation et trois critères complémentaires. Ses statuts sont **retenu**, **vigilance** et **écarté**.

`score` simule différents barèmes sur l'historique du titre choisi. Son classement courant montre les résultats bruts, l'activité et l'investissement maximal. Le lecteur peut comprendre les critères de tri puis retenir une définition de barème.

`candidate` examine cette définition au prix fourni ou enregistré dans Jedi. Il vérifie les quantités utiles et le financement, compte tenu des trades déjà engagés. Un projet opérable peut demander trop de fonds ; un projet disposant de fonds peut produire des ordres trop petits.

Un backtest complémentaire permet d'examiner un barème fixé sur une période historique. Aucun de ces calculs ne transforme une sélection en ordre d'achat automatique.

Le cycle de suivi

Une fois le trade ouvert, le travail courant consiste à actualiser le cours, consulter les ordres, suivre leur exécution chez le courtier, enregistrer les opérations et recalculer les ordres.

Un **ordre** est une instruction envisagée ou placée ; une **opération** est un achat ou une vente exécuté. Enregistrer une opération signifie décrire ce qui a réellement eu lieu, avec sa quantité et son prix.

Les événements exceptionnels méritent une trace : exécution partielle, annulation du solde d'un ordre, suspension, incident de cotation ou changement de réglage. Le journal `note` permet de conserver la date, le titre ou le nom du trade, les faits et la décision prise. Les chapitres 08 et 09 expliquent son usage.

Les données dont on a besoin

La base principale conserve les titres utilisés, les trades, les paramètres, les opérations, les notes et les cours enregistrés. Une base de marché distincte conserve des historiques de cotations.

Cette seconde base est facultative. `score` et `backtest` peuvent lire un fichier individuel préparé ; `profile` peut aussi examiner ce fichier. Pour comparer une population de titres et gérer sa fraîcheur, on utilise une base de marché.

Un historique **OHLCV** contient, par séance, l'ouverture, le plus haut, le plus bas, la clôture et le volume. Un cours courant est une information ponctuelle. Importer un historique avec `market` n'actualise pas le cours de la base principale.

La commande `quote get` interroge les fournisseurs de cours ; elle ne met aucune base à jour. Les commandes d'enregistrement et de suivi sont expliquées dans les chapitres pratiques.

Où poursuivre la lecture

Le chapitre 03 aide à choisir un titre. Les chapitres 04, 05 et 06 expliquent `profile`, `score` et `candidate`. Les chapitres 07 à 09 suivent un trade réel ; le chapitre 10 explique les fichiers et bases de marché.

La partie II détaille la mécanique avec `play` et `calc`. La partie III explique les backtests, les comparaisons de périodes et le suivi des résultats. Les annexes servent de référence pour les commandes, les fournisseurs, le screener externe et le vocabulaire.

2. Installer et préparer Jedi

Choisir son environnement

Jedi s'utilise dans un terminal. Le mode de diffusion retenu pour Jedi 1.0 est une archive contenant un exécutable Linux pour processeur **x86-64**, compilé statiquement. L'utilisateur n'a pas besoin d'installer Crystal ni de compiler les sources pour employer cet exécutable.

Environnement	Utilisation prévue pour Jedi 1.0
Linux x86-64	Exécutable Linux fourni dans l'archive de version.
Windows x86-64	Même exécutable dans une distribution Linux sous WSL 2.
Windows natif, Linux ARM64, macOS	Aucun exécutable distribué dans ce parcours d'installation.

La bibliothèque SQLite est intégrée au binaire statique. **gnuplot** reste un programme externe, nécessaire aux fonctions graphiques. Les commandes de gestion, de sélection et de simulation peuvent être utilisées sans produire de graphique.

Installer l'exécutable sous Linux

Vérifier l'architecture du poste :

```
uname -m
```

L'exécutable décrit ici demande une sortie `x86_64`. Télécharger l'archive binaire et son fichier de contrôle depuis la rubrique **Releases du dépôt du projet Jedi**. L'archive automatique « Source code » contient le code à compiler ; choisir l'archive de l'exécutable.

Les commandes suivantes utilisent la convention `jedi-1.0.0-linux-x86_64.tar.gz`, avec un répertoire extrait de même nom sans `.tar.gz`. Adapter le numéro et les noms si la livraison téléchargée les précise autrement. Depuis le répertoire du téléchargement :

```
jedi_version="1.0.0"
jedi_archive="jedi-${jedi_version}-linux-x86_64.tar.gz"
sha256sum -c "${jedi_archive}.sha256"
```

Le contrôle doit indiquer `OK`. En cas d'échec, reprendre le téléchargement avant de poursuivre.

Installer le binaire pour son propre compte, sans droits d'administration :

```
tar -xzf "$jedi_archive"
mkdir -p "$HOME/.local/libexec" "$HOME/.local/bin"
install -m 0755 "jedi-${jedi_version}-linux-x86_64/jedi" \
  "$HOME/.local/libexec/jedi"
```

Installer ensuite les trois lanceurs, livrés dans examples/launchers/ de l'archive documentaire. Depuis son répertoire `examples/` :

```
install -m 0755 launchers/jedi "$HOME/.local/bin/jedi"
install -m 0755 launchers/jedidemo "$HOME/.local/bin/jedidemo"
install -m 0755 launchers/jedidev "$HOME/.local/bin/jedidev"
export PATH="$HOME/.local/bin:$PATH"
hash -r
command -v jedi
command -v jedidemo
command -v jedidev
```

Les chemins affichés doivent désigner les trois fichiers dans `~/.local/bin/`. Si `jedi` est déjà défini comme un alias ou une fonction, retirer cette définition pour employer le lanceur installé. `type -a jedi` permet de voir les définitions trouvées.

Le binaire compilé est dans **`~/.local/libexec/jedi`**. Les trois petits scripts dans **`~/.local/bin/`** l'appellent directement : `jedi` pour le portefeuille réel, `jedidemo` pour les exemples et `jedidev` pour le développement. Ils s'exécutent comme des commandes ordinaires et refusent une utilisation avec `source`.

Les noms, commentaires et messages des lanceurs sont en anglais dans les deux éditions de la documentation. Deux messages signalent les erreurs propres à ces scripts :

Message	Action
`Jedi binary missing or not executable`	Vérifier l'installation du binaire dans `~/.local/libexec/jedi` et son droit d'exécution.
`Run this launcher as a command; do not source it`	Appeler `jedi`, `jedidemo` ou `jedidev` directement dans le terminal.

Pour conserver le chemin de recherche entre les sessions Bash, ajouter la ligne `export PATH="$HOME/.local/bin:$PATH"` à `~/.bashrc`. Les commandes d'usage courant de Jedi s'exécutent avec votre compte utilisateur.

Pour les fonctions graphiques, vérifier :

```
command -v gnuplot
gnuplot --version
```

Installer le paquet `gnuplot` avec le gestionnaire de paquets de la distribution si nécessaire.

Sous Windows, utiliser le terminal Linux de WSL

Jedi s'exécute dans Ubuntu sous WSL 2. Garder les deux terminaux bien distincts :

Terminal	Commandes de ce chapitre
PowerShell, côté Windows	Installation et ouverture de WSL ; commandes commençant par `wsl`.
Ubuntu, côté Linux	Installation de Jedi, répertoires, configuration et toutes les commandes `jedi`.

Installer Ubuntu et ouvrir le bon terminal

La procédure suivante concerne Windows 11 ou Windows 10 version 2004 et ultérieure. Elle comporte deux étapes : activer les composants Windows nécessaires à WSL, puis installer la distribution Linux Ubuntu.

Dans **PowerShell ouvert en administrateur**, commencer par activer WSL :

```
wsl --install
```

Si Ubuntu s'ouvre directement, créer le compte Linux comme expliqué ci-dessous. Si Windows demande un redémarrage, redémarrer avant de poursuivre.

Après le redémarrage, vérifier l'installation. Rouvrir PowerShell avec le même compte Windows :

```
wsl --list --verbose
```

Si Ubuntu apparaît dans la liste, passer à la création du compte et au contrôle de la version. Si le message indique qu'aucune distribution n'est installée, rouvrir PowerShell en administrateur et **relancer l'installation** :

```
wsl --install -d Ubuntu
```

La première commande peut s'être arrêtée après l'activation de `VirtualMachinePlatform`, qui exige un redémarrage. Le message de réussite concerne alors cette activation ; il ne prouve pas qu'Ubuntu est installé. Attendre maintenant le téléchargement et l'installation de la distribution. Si un nouveau redémarrage est demandé, l'effectuer puis vérifier à nouveau la liste.

Tant qu'Ubuntu n'est pas installé, `wsl -d Ubuntu` ne peut pas l'ouvrir et renvoie `WSL_E_DISTRO_NOT_FOUND`. Cette commande ouvre une distribution existante ; elle ne l'installe pas. La procédure Microsoft en français décrit l'installation d'une distribution.

Créer le compte Linux au premier lancement. Ubuntu peut s'ouvrir automatiquement à la fin de son installation. Créer alors un nom d'utilisateur et un mot de passe Linux. Ce compte est distinct du compte Windows ; le mot de passe saisi ne s'affiche pas. La création du compte Linux détaille cette étape.

Une fois le compte créé, l'invite d'Ubuntu peut ressembler à `user@ordinateur:~$`. Les commandes saisies à cet endroit sont des commandes Linux. Pour revenir à PowerShell et vérifier l'installation, saisir dans Ubuntu :

```
exit
```

Dans PowerShell, contrôler la liste :

```
wsl --list --verbose
```

Une ligne **Ubuntu** doit maintenant apparaître, avec **2** dans la colonne `VERSION`. L'état peut être `Running` ou `Stopped` ; les deux indiquent que la distribution est installée. Pour une distribution Ubuntu nouvellement installée qui serait encore en version 1, la conversion s'effectue depuis PowerShell avec `wsl --set-version Ubuntu 2`.

Ouvrir ensuite Ubuntu depuis PowerShell :

```
wsl -d Ubuntu
```

Si le compte Linux n'avait pas encore été créé, le créer à ce premier lancement. Si le nom affiché dans la liste diffère d'Ubuntu, employer ce nom dans les commandes. La référence WSL en français décrit la consultation et l'ouverture des distributions.

Toutes les étapes suivantes se font **dans Ubuntu**, avec le compte Linux créé. Revenir dans son répertoire personnel :

```
cd ~
echo "$HOME"
```

Un chemin comme `/home/user` désigne ce répertoire Linux. Il diffère de `C:\Users\...` côté Windows.

Préparer la langue dans Ubuntu

Une installation neuve d'Ubuntu peut utiliser `LANG=C.UTF-8`. Cette locale est un réglage système neutre avec UTF-8. Pour choisir le français ou l'anglais et préparer les exemples, installer les locales dans **le terminal Ubuntu** :

```
sudo apt update
sudo apt install locales
sudo dpkg-reconfigure locales
```

Dans la liste proposée, sélectionner avec **Espace** :

Locale à générer	Usage
`fr_FR.UTF-8 UTF-8`	Français, France.
`en_US.UTF-8 UTF-8`	Anglais, États-Unis.

Utiliser **Tabulation** pour atteindre le bouton de validation, puis **Entrée**. À la question sur la locale par défaut, choisir **fr_FR.UTF-8** pour suivre le parcours français, ou **en_US.UTF-8** pour le parcours anglais. Attendre la fin de la génération.

Vérifier les locales disponibles :

```
locale -a
```

Les noms peuvent apparaître sous les formes `fr_FR.utf8` et `en_US.utf8`. La génération rend ces locales disponibles ; elle ne change pas nécessairement la langue du terminal déjà ouvert. Pour utiliser le français dès cette session :

```
export LANG=fr_FR.UTF-8
locale
```

Pour l'anglais, utiliser `export LANG=en_US.UTF-8`. Dans la sortie de `locale`, vérifier la ligne `LANG`. Jedi s'appuie sur cette variable avec son réglage de langue système ; installer les locales et choisir `LANG` sont deux opérations distinctes.

La section **Retrouver le programme et l'éditeur dans les prochaines sessions**, ci-dessous, conserve ce choix dans `~/.bashrc`. Les références Ubuntu sur la génération des locales et leur consultation détaillent ces commandes.

Préparer les répertoires de Jedi dans Ubuntu

Une installation neuve d'Ubuntu peut ne pas encore contenir les répertoires XDG employés par Jedi. Les créer explicitement permet de savoir où seront placés ses fichiers :

```
mkdir -p "$HOME/.config/jedi"
mkdir -p "$HOME/.local/state/jedi"
mkdir -p "$HOME/.local/share/jedi/market"
mkdir -p "$HOME/.local/bin" "$HOME/.local/libexec"
mkdir -p "$HOME/jedi-examples"
```

Répertoire	Fichier ou usage Jedi
`~/.config/jedi/`	Configuration `jedi.conf`.
`~/.local/state/jedi/`	Journal `jedi.log`.
`~/.local/share/jedi/`	Base principale `jedi.db`.
`~/.local/share/jedi/market/`	Bases d'historiques de marché.
`~/.local/bin/`	Lanceurs publics `jedi`, `jedidemo`, `jedidev`.
`~/.local/libexec/`	Binaire `jedi` compilé, appelé par les lanceurs.
`~/jedi-examples/`	Dossier de travail proposé pour les essais et fichiers livrés.

Il n'est pas nécessaire de définir `XDG_CONFIG_HOME`, `XDG_STATE_HOME` ou `XDG_DATA_HOME`. Lorsque ces variables sont absentes, Jedi utilise respectivement `~/.config`, `~/.local/state` et `~/.local/share`, puis leur sous-répertoire `jedi`. Il sait aussi créer les répertoires parents manquants lors de l'initialisation.

Si des variables XDG sont déjà définies, les chemins peuvent différer. Le lanceur public `jedi` emploie ces chemins de production et retire les anciens réglages `JEDI_*` de chemins hérités du terminal. Vérifier les emplacements effectifs avec `jedi config --paths` après installation. `jedidemo` et `jedidev` choisissent leurs propres espaces locaux, expliqués plus bas.

Installer le programme et les outils utiles

Créer un dossier Linux pour recevoir l'archive binaire téléchargée et son fichier de contrôle :

```
mkdir -p "$HOME/telechargements-jedi"
cd "$HOME/telechargements-jedi"
explorer.exe .
```

L'Explorateur Windows s'ouvre sur ce dossier Linux. Y copier les deux fichiers téléchargés côté Windows. Revenir au terminal Ubuntu et suivre **Installer l'exécutable sous Linux**, plus haut, à partir du contrôle `sha256sum`. Garder les bases et les essais dans le système de fichiers Linux, selon les indications Microsoft.

Installer également un éditeur et le programme graphique dans Ubuntu :

```
sudo apt update
sudo apt install gnuplot nano
```

Le mot de passe demandé est celui du compte Linux. Les commandes Jedi suivantes s'exécutent avec ce compte.

Retrouver le programme et l'éditeur dans les prochaines sessions

Dans Ubuntu, ouvrir le fichier d'initialisation du terminal :

```
nano "$HOME/.bashrc"
```

Ajouter à la fin les quatre lignes suivantes. La dernière choisit le français pour ce terminal :

```
export PATH="$HOME/.local/bin:$PATH"
export EDITOR=nano
export VISUAL=nano
export LANG=fr_FR.UTF-8
```

Dans nano, enregistrer avec **Ctrl+O**, confirmer le nom avec **Entrée**, puis quitter avec **Ctrl+X**. Relire les réglages dans le terminal courant et contrôler :

```
source "$HOME/.bashrc"
command -v jedi
```

```
command -v nano
jedi version
jedi config --paths
```

Au premier lancement de Jedi, accepter explicitement la création des fichiers, comme indiqué dans la section suivante. L'éditeur servira notamment à `note create --edit` et `note update --edit`, expliqués au chapitre 08.

Retrouver les fichiers depuis Windows

Pour ouvrir le répertoire de configuration dans l'Explorateur :

```
cd "$HOME/.config/jedi"
explorer.exe .
```

L'accès depuis l'Explorateur est également possible par `\\wsl$`, puis la distribution et son répertoire `home`. Les chemins affichés par `jedi config --paths` restent des chemins Linux à utiliser dans le terminal Ubuntu. Copier les fichiers `examples/` de l'archive documentaire dans `~/jedi-examples/` pour les essais ; le guide des exemples précise les bases à préparer.

Effectuer le premier lancement

```
jedi version
```

Cette commande affiche la version de Jedi et les informations de compilation. Au premier lancement, elle demande de créer successivement la configuration, le journal et la base principale. **Répondre explicitement 0** à chaque demande en français, ou `Y` en anglais.

La base principale commence vide. Elle recevra les titres, barèmes, trades, paramétrages, opérations et notes. Les bases d'historiques de marché sont préparées séparément au chapitre 10.

Afficher les emplacements effectivement utilisés :

```
jedi config --paths
```

Élément	Emplacement Linux habituel
Configuration	`~/.config/jedi/jedi.conf`
Journal	`~/.local/state/jedi/jedi.log`
Base principale	`~/.local/share/jedi/jedi.db`
Bases de marché	`~/.local/share/jedi/market/`

Les variables XDG du système peuvent modifier ces emplacements. La sortie de `config --paths` donne les chemins à employer pour vos sauvegardes.

Choisir la langue et les réglages

Avec son réglage de langue système par défaut, Jedi utilise le français lorsque `LANG` désigne cette langue, et l'anglais pour les autres valeurs de langue reconnues. Pour choisir la langue d'une commande :

```
LANG=fr_FR.UTF-8 jedi version
LANG=en_US.UTF-8 jedi version
```

Le fichier `jedi.conf` peut rester vide : le programme contient ses réglages par défaut. Ajouter seulement les valeurs à modifier. La configuration effective se consulte avec :

```
jedi config
```

Elle peut contenir les clés d'accès de fournisseurs que vous avez configurées. Vérifier son contenu avant de la transmettre.

Voici les réglages **fictifs** utilisés pour le trade des chapitres 07 à 09, à placer dans le `jedi_test.conf` de l'espace d'essai décrit ci-dessous pour reproduire ce parcours :

```
fees:
  rate: 0.0035
  min: 0.0
trading:
  capital: 5000.0
market_data:
```

```
  default_universe: france
```

`fees.rate` vaut ici 0,35 % et `fees.min` est le minimum par opération. `trading.capital` est le capital de référence consacré aux trades. Pour un usage réel, employer vos propres montants et frais. Les paramètres de simulation sont distincts du capital de suivi réel.

Les options d'une commande remplacent les réglages qu'elles désignent : par exemple, `candidate --capital=...` utilise le capital fourni pour cette évaluation ; il ne modifie pas `trading.capital`. Une modification du fichier est relue au lancement suivant.

Préparer un espace d'essai séparé

Avec les lanceurs installés plus haut, **le nom de la commande choisit l'espace** :

Commande	Usage	Emplacement par défaut
`jedi`	Portefeuille réel.	Fichiers `jedi.conf`, `jedi.log`, `jedi.db` et bases de marché aux chemins XDG.
`jedidemo`	Exercices de la documentation.	Configuration, journal, base `jedi_test.db` et marché dans `~/jedi-examples/jedi/env_test/`.
`jedidev`	Développement et essais d'un contributeur.	Fichiers `jedi_dev.*` et marché dans l'espace `env_dev` du projet.

Les deux premières commandes suffisent pour utiliser Jedi et apprendre avec les exemples. Le fonctionnement des commandes reste le même ; leurs fichiers sont séparés.

Il n'y a pas de bascule à effectuer dans le terminal. Vous pouvez appeler `jedi`, puis `jedidemo`, puis `jedi` à nouveau. **`jedi` utilise toujours la production** ; son lanceur force ce choix et retire les anciens réglages `JEDI_*` de chemins dans son propre processus. Les chemins XDG restent ceux du système.

Copier les fichiers d'exemple

Depuis le répertoire `examples/` de l'archive documentaire :

```
mkdir -p "$HOME/jedi-examples"
cp *.txt "$HOME/jedi-examples/"
jedidemo config --paths
```

Au premier lancement, accepter explicitement la création de la configuration, du journal et de la base principale : `O` en français, `Y` en anglais. Vérifier que `environment` indique **`test`** et relever les chemins :

Élément d'essai	Emplacement par défaut
Configuration	`~/jedi-examples/jedi/env_test/conf/jedi_test.conf`
Journal	`~/jedi-examples/jedi/env_test/log/jedi_test.log`
Base principale	`~/jedi-examples/jedi/env_test/data/jedi_test.db`
Bases de marché	`~/jedi-examples/jedi/env_test/market/`

Ajouter les réglages fictifs de la section précédente dans **`jedi_test.conf`** au chemin affiché. Les commandes d'exemple des chapitres suivants utilisent **`jedidemo`**. Pour les commandes sur une source réelle, préparer les historiques dans le marché de cet espace avant de les étudier.

```
jedidemo trade list
```

Le lanceur ouvre `~/jedi-examples/` pour Jedi : les noms comme `cotations_EXA.txt` sont donc utilisables depuis n'importe quel répertoire du terminal. Les chemins relatifs fournis à `jedidemo` se lisent depuis ce dossier, y compris ceux d'un fichier à importer ou à exporter. Employer un chemin absolu pour un fichier situé ailleurs. Les redirections comme `> sauvegarde.txt` sont effectuées par le terminal dans son propre dossier ; un chemin absolu évite toute ambiguïté.

Le `cd` du script concerne seulement le processus du lanceur et de Jedi. **Le terminal appelant conserve son répertoire et ses variables.** Les lanceurs conservent aussi la langue, l'éditeur et le fuseau du terminal. Pour reproduire un calcul explicitement en UTC sur une commande :

```
TZ=UTC jedidemo version
```

Alterner entre portefeuille et démonstration

```
jedi trade list
jedidemo trade list
jedi config --paths
jedidemo config --paths
```

La première liste concerne le portefeuille réel, la deuxième les trades fictifs. Les deux dernières commandes montrent les chemins propres à chaque espace. Les bases restent en place : `jedidemo` retrouve les mêmes exemples au lancement suivant.

Conserver plusieurs espaces d'essai

Chaque répertoire peut avoir sa propre base de test et son propre marché. Par exemple, conserver les évaluations sans trade des chapitres 03 à 06 dans l'espace habituel, puis créer un autre espace pour les ouvertures des chapitres 07 à 09 :

```
mkdir -p "$HOME/jedi-examples-trades"
cp "$HOME/jedi-examples/"*.txt "$HOME/jedi-examples-trades/"
JEDI_DEMO_DIR="$HOME/jedi-examples-trades" jedidemo config --paths
```

Cette première commande Jedi initialise l'autre espace après vos confirmations. Utiliser le même préfixe **`JEDI_DEMO_DIR=...`** pour chacune de ses commandes :

```
JEDI_DEMO_DIR="$HOME/jedi-examples-trades" jedidemo trade list
jedidemo trade list
```

La première ligne utilise le second espace ; la suivante retrouve l'espace habituel. Pour choisir durablement un autre dossier de démonstration, la ligne `export JEDI_DEMO_DIR="..."` peut être ajoutée à `~/.bashrc`. Elle personnalise `jedidemo` ; `jedi` reste la production.

Depuis un répertoire d'essai déjà créé, on peut aussi lancer :

```
JEDI_DEMO_DIR="$PWD" jedidemo config --paths
```

Dans un dossier `essai`, les fichiers sont sous `essai/jedi/env_test/`. Si le dossier s'appelle déjà `jedi`, ils sont sous `jedi/env_test/`, sans répéter ce nom.

Choisir éventuellement un autre marché

Le marché de démonstration est local à l'espace par défaut. Pour utiliser volontairement un répertoire de marché existant :

```
JEDI_DEMO_MARKET_DIR="/chemin/vers/market" \
  jedidemo market info abcbourse
```

Remplacer le chemin et la source par ceux de vos données. Seules les bases de marché sont alors partagées ; la base principale de test reste dans l'espace choisi. Les commandes `market import`, `update`, `replace` et `group` agissent sur le marché sélectionné : choisir un marché séparé pour les exercices qui modifient les données.

Un chemin relatif de `JEDI_DEMO_MARKET_DIR` est lu depuis le dossier de démonstration. La variable générale `JEDI_MARKET_DIR` est effacée par les lanceurs ; leur sélection volontaire utilise `JEDI_DEMO_MARKET_DIR` ou `JEDI_DEV_MARKET_DIR`.

Pour les développeurs : utiliser jedidev

Cette section est facultative pour l'utilisateur du binaire fourni. Un contributeur installe **Crystal 1.21.1** et **Shards**, selon les instructions officielles Crystal, puis travaille depuis la racine des sources.

Le lanceur `jedidev` utilise le répertoire courant comme projet, ou celui fourni par `JEDI_DEV_DIR`. Il sélectionne le mode `dev` et conserve séparément configuration, journal, base principale et bases de marché :

```
jedidev config --paths
```

Si le projet s'appelle `jedi`, la base est `<projet>/env_dev/data/jedi_dev.db` et le marché `<projet>/env_dev/market/`. Pour un autre nom de dossier, ils sont sous `<projet>/jedi/env_dev/`.

Pour installer les dépendances, vérifier les sources et compiler un binaire de travail :

```
shards install
crystal tool format --check src spec
TZ=UTC crystal spec
mkdir -p bin
crystal build src/jedi_cli.cr --error-trace -o bin/jedi-dev
chmod +x bin/jedi-dev
JEDI_DEV_BINARY="./bin/jedi-dev" jedidev config --paths
```

La compilation demande aussi les bibliothèques de développement des dépendances, notamment SQLite. Les noms des paquets dépendent de la distribution. Les tests sont lancés en UTC pour reproduire les dates des vérifications.

`JEDI_DEV_BINARY` choisit le binaire compilé pour cette commande. Sans cette variable, `jedidev` appelle le binaire installé dans `~/.local/libexec/jedi`. Le chemin relatif du binaire se lit depuis le projet sélectionné. Pour un appel depuis un autre dossier, fournir les deux chemins :

```
JEDI_DEV_DIR="/chemin/du/projet" \
  JEDI_DEV_BINARY="./bin/jedi-dev" jedidev version
```

`JEDI_DEV_MARKET_DIR` permet de choisir volontairement un autre marché, comme pour la démonstration. Pour le suivi réel, employer le lanceur `jedi`.

Compiler rapidement pendant le développement

Compiler sans `--release` pour les modifications quotidiennes. Pour les mesures de performance et la validation finale, compiler avec `--release`. Les deux commandes utilisent `src/jedi_cli.cr` ; l'environnement de données est choisi au lancement par `jedi`, `jedidemo` ou `jedidev`.

```
mkdir -p bin
crystal build src/jedi_cli.cr --error-trace -o bin/jedi-dev
crystal build src/jedi_cli.cr --release --error-trace -o bin/jedi
```

Deux petits scripts Bash, `build-dev` et `build-release`, peuvent regrouper ces commandes. Ils produisent des fichiers distincts dans `bin/`. Garder les binaires générés hors des fichiers source versionnés.

Depuis la racine du projet, sélectionner le binaire de développement pour ce terminal :

```
export JEDI_DEV_BINARY=./bin/jedi-dev
jedidev config --paths
jedidev trade list
```

Relancer la compilation rapide après une modification des sources. Avant la validation finale, exécuter aussi `TZ=UTC crystal spec --release`, puis compiler le binaire optimisé. La compilation locale utilise les bibliothèques de l'hôte ; le binaire statique distribué est construit par la procédure de publication.

Sauvegarder et reprendre les données

Entre deux commandes Jedi, lorsque aucune n'est en cours, sauvegarder la configuration et les bases indiquées par `config --paths`, pour chaque espace utile. Une copie des répertoires de configuration et de données conserve notamment la base principale et les bases de marché. Garder également les fichiers d'origine et les listes nécessaires à leur reconstitution. Le journal peut aider à comprendre un incident.

Jedi fournit en complément un export de la **base principale**, y compris les notes et les cours enregistrés. Ici, le fichier de production est écrit au chemin absolu choisi :

```
jedi database dump > "$HOME/jedi-sauvegarde.txt"
```

Cet export ne contient ni `jedi.conf` ni les bases de marché. Pour vérifier une restauration, créer un dossier neuf et initialiser sa base principale :

```
mkdir -p "$HOME/jedi-restauration"
JEDI_DEMO_DIR="$HOME/jedi-restauration" jedidemo config --paths
JEDI_DEMO_DIR="$HOME/jedi-restauration" \
  jedidemo database load "$HOME/jedi-sauvegarde.txt"
JEDI_DEMO_DIR="$HOME/jedi-restauration" jedidemo trade list
JEDI_DEMO_DIR="$HOME/jedi-restauration" jedidemo note list
```

`load` exige une base entièrement vide, notes comprises. Il ne fusionne pas l'export avec un portefeuille existant. Contrôler les trades, opérations, notes et réglages avant de reprendre le suivi. Le lanceur `jedi` retrouve le portefeuille réel ; `jedidemo` sans ce préfixe retrouve la démonstration habituelle. Pour restaurer une copie des fichiers, remettre chaque fichier au chemin prévu, sans commande Jedi en cours.

Sous WSL, une copie complète de la distribution peut aussi être créée depuis PowerShell :

```
wsl --shutdown
wsl --export Ubuntu jedi-wsl-backup.tar
```

Adapter le nom de la distribution. La référence Microsoft décrit l'export et l'import d'une distribution.

Mettre à jour Jedi

Sauvegarder les données, lire les notes de version, puis vérifier et installer le nouvel exécutable avec la procédure précédente. Le remplacement du programme conserve les fichiers de votre environnement. Après installation, contrôler `jedi version`, `jedi config --paths` et quelques données connues.

Vous pouvez maintenant suivre le choix d'un titre, ou reprendre directement l'ouverture fictive d'EXA.

3. Choisir un titre-support

Commencer par une liste de travail

Le titre-support est le titre que l'on souhaite gérer dans un trade. Sa sélection précède le choix du barème et l'évaluation du montant à engager.

On peut partir d'une liste suivie personnellement, d'une plateforme de marché, d'un screener externe ou de `tools profile`. Dans tous les cas, les titres trouvés constituent une liste à examiner. L'utilisateur conserve le choix final.

Pour comprendre un résultat, noter la population étudiée et la date des cours. Une liste Euronext Growth et une liste réunissant plusieurs marchés ne comparent pas les mêmes titres. Un même titre peut changer de rang si la population change.

Quatre questions pratiques

Critère principal	Question à poser	Lecture pratique
Tendance	Le cours monte-t-il, baisse-t-il ou reste-t-il globalement stable ?	Une baisse persistante peut entraîner des achats successifs et immobiliser du capital.
Liquidité	Quels montants s'échangent habituellement ?	Mettre cette information en regard des quantités et montants envisagés.
Volatilité	Quelle ampleur ont les variations quotidiennes ?	Des mouvements trop faibles limitent les occasions ; des mouvements importants demandent davantage d'attention.
Oscillation	Les mouvements changent-ils régulièrement de sens ?	Les allers-retours reposent sur des mouvements dans les deux directions.

Une variation journalière importante et une oscillation régulière sont différentes. Un titre qui baisse fortement chaque jour peut être volatil tout en offrant peu de retours à la hausse.

Trois critères complètent cette lecture : le **recul maximal**, c'est-à-dire la baisse depuis un sommet antérieur ; les **gaps**, écarts entre une clôture et l'ouverture suivante ; le **pas suggéré**, taille de mouvement proposée pour commencer l'examen d'un barème.

Le chapitre 04 explique les mesures, les seuils et les sept critères de classement. Commencer par les quatre principaux aide à lire le rapport ; les points de vigilance complémentaires restent à examiner.

Utiliser une sélection externe

Un screener filtre une population selon des conditions. Le ProScreener JEDI de l'annexe 03 utilise des barres journalières et recherche une tendance compatible, une liquidité minimale et une volatilité suffisante. Il affiche aussi l'oscillation.

Choisir explicitement le marché ou la liste de titres dans la plateforme, puis lire les résultats sur une séance terminée. Repérer quelques titres et vérifier leur historique, leurs volumes et les éventuels événements récents.

La sélection externe et `profile` doivent fournir des listes cohérentes sur les principaux critères. Des différences peuvent venir de la date, du fournisseur, des ajustements des cours, du calcul de tendance ou de l'historique disponible. La concordance parfaite des listes n'est pas un objectif de choix.

Le script JEDI et sa procédure sont repris dans l'annexe 03.

Utiliser une base de marché Jedi

Pour essayer les commandes avant de préparer vos propres données, suivre le début du chapitre 10 : la base Demo des exemples. Ce parcours crée la source `demo` et le dataset `Demo`, puis les met à jour. Le chapitre 04 les utilise pour expliquer les résultats de `profile`.

Une base de marché classe les historiques par **dataset**, ensemble importé, et peut réunir des titres dans des **groupes**, listes logiques définies par l'utilisateur.

```
jedidemo tools profile --database=abcbourse --dataset=Euronext_A --rank-criteria=4
```

Cette commande examine les titres du dataset `Euronext_A` et additionne leurs rangs sur les quatre premiers critères. Remplacer `abcbourse` et le dataset par ceux de la base effectivement préparée.

Un groupe peut réunir des titres de plusieurs datasets :

```
jedidemo tools profile --database=abcbourse --group=PEA --rank-criteria=4
```

Combiner `--dataset` et `--group` retient leur intersection. Le groupe `PEA` est une liste enregistrée dans la base ; son nom ne vérifie pas automatiquement l'éligibilité de chaque titre.

Les exemples préparatoires de Jedi utilisent notamment des historiques français et les datasets `Euronext_A`, `Euronext_B`, `Euronext_C`, `Euronext_Access` et `Euronext_Growth`. Ces noms n'imposent pas le marché sur lequel utiliser le logiciel.

Travailler avec un fichier individuel

Une base de marché n'est pas nécessaire pour étudier un titre déjà choisi. Un **fichier préparé** contient ses cotations dans un format lisible par Jedi, avec son identité et des lignes OHLCV contrôlées. Sa préparation est expliquée aux chapitres 05 et 10.

Le fichier fictif cotations_EXA.txt se trouve dans le répertoire **examples/** livré avec cette documentation. Les données sont fournies sous forme de fichiers utilisables, séparément du texte des chapitres. Le guide des exemples explique comment préparer le terminal et isoler les essais.

Après la copie des fichiers dans `~/jedi-examples/` au chapitre 02, les commandes suivantes utilisent directement le fichier. `jedidemo` ouvre ce dossier ; aucun import dans une base de marché n'est nécessaire :

```
jedidemo tools profile cotations_EXA.txt
jedidemo tools score cotations_EXA.txt --sort=fitness --budget=40
```

Le premier rapport décrit le titre ; le second recherche des barèmes. Le budget de 40 permet une démonstration courte. Le budget courant du programme est de 500.

`profile` calcule les critères récents sur 252 séances, avec un contrôle de qualité sur l'historique long. `score` conserve 750 séances par défaut pour simuler davantage de situations. Le chapitre 05 explique cette différence et comment examiner une autre période.

Passer au projet de trade

Après choix d'un barème, évaluer le candidat avec son ISIN, une définition de barème et un prix actuel vérifié :

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --amount=1000 --capital=5000
```

Cet exemple utilise un titre et un prix **fictifs**. Il évalue un achat initial de 100 titres et les besoins du projet. Il ne lit pas les cotations du fichier `EXA` : le prix de 10 € est fourni explicitement pour illustrer le calcul.

Avant de retenir le titre, lire ses alertes, examiner le barème, vérifier les ordres utiles et le financement. S'il reste une difficulté, choisir un autre projet ou préciser les données plutôt que retenir automatiquement la première ligne d'un classement.

4. Présélectionner avec tools profile

Ce que fournit la commande

`profile` décrit le comportement d'un titre et compare une population pour aider à établir une liste de choix. Il examine la qualité des cotations, applique des règles d'élimination et de vigilance, puis classe les titres conservés.

Le classement est transparent : il additionne des rangs, sans pondération. Le lecteur peut choisir combien de critères participent au total et consulter les valeurs qui expliquent les rangs.

Préparer les exemples

Les exemples calculés de ce chapitre utilisent trois titres fictifs. Leurs fichiers sont dans **examples/**, à la racine de l'archive de documentation ; le guide des exemples explique les réglages et le répertoire de travail.

Pour les commandes `--database=demo --dataset=Demo`, effectuer d'abord la préparation de la base Demo au début du chapitre 10, jusqu'à la mise à jour des trois titres au 2 octobre 2026. Cette étape crée la base ; `profile` ne la crée pas.

La commande utilisant cotations_EXA.txt lit directement ce fichier depuis `examples/` et ne demande aucun import. Les exemples `abcbourse` et `Euronext_A` ci-dessous supposent, eux, que ces données ont été préparées par l'utilisateur.

Choisir la population

```
jedidemo tools profile --database=abcbourse --dataset=Euronext_A --rank-criteria=4
```

`--database` choisit la source d'une base de marché. `--dataset` choisit un ensemble importé. L'univers utilisé dans le nom de la base vient de la configuration, ou de `--universe` s'il est précisé.

Autres choix possibles :

Besoin	Options
Tous les membres d'un groupe	`--group=PEA`
Membres du groupe appartenant à un dataset	`--dataset=Euronext_A --group=PEA`
Un titre identifié dans la base	`--security=RNO` ou son ISIN
Plusieurs titres identifiés	Répéter `--security`

Un fichier individuel peut aussi être examiné :

```
jedidemo tools profile cotations_EXA.txt
```

Avec un fichier, le rapport décrit un titre ; il n'établit pas de classement transversal ni de référence de fraîcheur du dataset.

Une fenêtre récente et un historique de contrôle

La tendance, la liquidité, l'oscillation, le recul maximal, les gaps et le pas suggéré utilisent les **252 dernières séances** du titre. La volatilité utilise un **ATR sur 20 séances**. L'ATR est une moyenne de l'amplitude réelle quotidienne, qui tient compte des écarts avec la clôture précédente ; il est exprimé en pourcentage de la dernière clôture.

252 séances correspondent approximativement à une année de bourse. Cette fenêtre décrit le comportement récent. Pour vérifier la qualité et la continuité des données, Jedi conserve toutefois l'historique plus long.

Les principaux contrôles de qualité demandent au moins **trois années d'historique** et une **densité minimale de 180 cotations par an en moyenne**, sur l'ensemble de cet historique. Pour calculer cette densité, Jedi divise le nombre de cotations disponibles par la durée entre la première et la dernière date, exprimée en années de 365,25 jours.

180 est un seuil de contrôle fixé dans Jedi, destiné à écarter un historique trop clairsemé. Il ne décrit pas le nombre habituel de séances d'une année de bourse et ne remplace pas la fenêtre

récente de 252 séances. La moyenne porte sur toute la période ; elle ne garantit pas que chaque année est complète.

Des cotations OHLC incohérentes, trop de séances sans volume ou trop de clôtures inchangées peuvent également éliminer le titre. Ces exigences expliquent pourquoi un fichier limité à 252 lignes peut être insuffisant.

Les dates réellement étudiées figurent dans le rapport individuel. Les séances disponibles dans chaque historique déterminent la fenêtre : un titre en retard peut avoir une fenêtre décalée.

Lire les statuts

Statut	Sens pratique
Retenu	Aucune cause d'élimination ni de vigilance détectée par ces contrôles.
Vigilance	Le titre reste dans la liste classée, avec des points à examiner.
Écarté	Au moins une règle d'élimination est déclenchée.

Les règles principales sont :

Mesure	Élimination	Vigilance
Tendance annualisée	Inférieure ou égale à −12 % par an	Lire aussi le parcours et les autres mesures.
Médiane quotidienne de clôture × volume	Inférieure à 100 000 € pour les exemples en euros	Plus de 30 % des séances sont sous ce montant.
ATR20	Inférieur à 1,5 %	Supérieur à 6 %.
Oscillations d'au moins 5 %	—	Moins de quatre par an.
Recul maximal	—	Supérieur à 45 %.
Gaps d'au moins 3 %	—	Présents sur plus de 3 % des séances.
Retard de cotation	Plus de deux séances	Une ou deux séances.

La médiane est la valeur centrale des montants journaliers : elle réduit l'influence de quelques journées exceptionnelles. La tendance est une mesure calculée à partir de l'ensemble de la fenêtre, pas simplement la différence entre son premier et son dernier cours.

La liquidité est mesurée dans l'unité monétaire des prix importés. Le seuil numérique de 100 000 n'effectue aucune conversion de devise. Lorsqu'on utilise des données hors euro, leur unité doit être prise en compte.

Les sept critères, dans leur ordre de classement

Ordre	Critère	À regarder
1	Tendance, `Tend%/a`	Une orientation moins défavorable ou plus favorable reçoit un meilleur rang.
2	Liquidité, `Val.méd`	Un montant médian échangé plus élevé reçoit un meilleur rang.
3	Volatilité, `ATR%`	Jedi classe d'abord des plages de volatilité, puis départage les valeurs dans la même plage.

Ordre	Critère	À regarder
4	Oscillation, `Sw5/a` et `Range%`	Nombre d’oscillations et amplitude ; les classes tiennent compte des deux mesures.
5	Recul maximal, `DD%`	Un recul plus faible reçoit un meilleur rang.
6	Gaps, `Gap%`	Une fréquence plus faible reçoit un meilleur rang.
7	Pas suggéré, `Pas%`	Un pas plus faible reçoit un meilleur rang.

Pour la volatilité, la plage 2 à 5 % est classée avant les plages 1,5 à moins de 2 % et plus de 5 à 6 %. Une volatilité encore plus élevée appelle de la vigilance. « Plus volatil » ne signifie donc pas systématiquement « mieux classé ».

L’oscillation privilégie des mouvements d’au moins 5 % assez fréquents, accompagnés d’une amplitude suffisante. `Sw5/a` compte le premier mouvement significatif puis les retournements, ramenés à une année ; il ne compte pas des allers-retours complets ni des opérations Jedi. Le rapport individuel et l’aide expliquent les plages utilisées :

```
jedidemo tools profile --help
```

`Mouv/a` estime l’activité associée au pas suggéré. Il s’agit d’une information complémentaire ; ce n’est pas un huitième critère de classement. Le pas proposé est un point de départ pour examiner un barème.

Commencer avec les quatre principaux critères

L’option `--rank-criteria=4` additionne les rangs de tendance, liquidité, volatilité et oscillation. C’est une lecture simple pour commencer.

Le défaut du programme reste **sept critères** :

```
jedidemo tools profile --database=abcbourse --dataset=Euronext_A
```

`--rank-criteria=N` accepte un nombre de 1 à 7. Il utilise les **N premiers critères dans l’ordre ci-dessus**. Les critères suivants restent affichés et leurs alertes restent actives.

Dans la colonne `Rangs`, la barre verticale sépare les rangs retenus des rangs seulement informatifs. Par exemple, `1/2/3/2 | 4/3/2` donne un total de 8 avec quatre critères. Cet exemple de somme est illustratif.

Deux valeurs identiques reçoivent le même rang. Le total le plus faible vient en premier ; en cas d’égalité, les rangs retenus sont comparés dans leur ordre, puis le ticker. Modifier la population peut modifier les rangs et les totaux.

Lire un exemple calculé

Après la création et la mise à jour de la base Demo décrites au début du chapitre 10, lancer depuis `examples/` :

```
jedidemo tools profile --database=demo --dataset=Demo \
  --rank-criteria=4 --include-rejected
```

Voici une sélection de colonnes de la sortie obtenue ; la table est abrégée pour la lecture :

Ticker	Total	Tendance/an	Valeur médiane	ATR20	Oscillations/an	Statut
EXA	4	3,7 %	774 k€	3,4 %	8,3	retenu
EXB	—	−27,4 %	377 k€	3,1 %	11,4	écarté
EXC	—	−0,5 %	12 k€	1,1 %	0,0	écarté

EXB possède de la liquidité et des oscillations, mais sa tendance suffit à l'écarter. EXC est écarté pour liquidité et volatilité insuffisantes. EXA est le seul titre classé ; il reçoit donc le rang 1 sur chacun des quatre critères.

`--include-rejected` affiche les titres écartés avec un tiret pour le total et les rangs. Leur présence ne change pas le classement des autres titres. Les lignes portant « vigilance » restent, elles, dans le classement.

Pour comprendre les motifs, consulter le titre individuellement :

```
jedidemo tools profile --database=demo --security=EXA
```

Comprendre la fraîcheur

Jedi utilise la date de fraîcheur globale du dataset : la dernière date atteinte par **au moins 95 % de ses titres**. Cette date est recalculée à partir des cotations présentes après import ou mise à jour.

Le retard est compté en séances présentes dans la base du dataset, jusqu'à cette référence. Il vaut zéro si le titre a atteint ou dépassé la référence, une ou deux séances pour la vigilance, plus de deux pour l'élimination.

Cette mesure compare les titres du dataset entre eux. Elle ne mesure pas l'écart avec la date du jour : si tout le dataset a arrêté de recevoir des cours, vérifier la date affichée dans `market info`.

Le contrôle de fraîcheur est indisponible avec un fichier ou une sélection historique `--date` ou `--select`. Il est aussi indisponible si la référence du dataset manque, ou si un titre appartient à plusieurs datasets sans qu'un `--dataset` explicite lève l'ambiguïté. Un tiret dans `Ret.` signifie « non évalué », pas « à jour ».

Choisir quelques titres

Commencer par les quatre critères principaux, puis lire les alertes et les trois critères complémentaires. Examiner un recul important ou des gaps fréquents avant de retenir une ligne bien classée.

Pour privilégier une lecture particulière, employer un tri nommé :

```
jedidemo tools profile --database=demo --dataset=Demo --sort=liquidity
jedidemo tools profile --database=demo --dataset=Demo --sort=oscillation
```

Les noms d'options restent en anglais dans les deux langues. `--limit=20` limite les lignes affichées ; il ne réduit pas la population analysée pour calculer les rangs.

Retenir quelques titres compréhensibles, puis examiner leurs barèmes avec `score` et leurs projets avec `candidate`. Le classement aide à organiser ce choix ; le prix d'entrée, les quantités et le financement sont étudiés séparément.

5. Chercher un barème avec tools score

Une recherche pour un titre déjà choisi

`score` simule plusieurs barèmes sur l'historique d'un titre et classe ceux qu'il a évalués. Le titre vient du choix de l'utilisateur, de `profile` ou d'une sélection externe.

Le parcours courant utilise une recherche **adaptative**, un classement **raw** et un affichage **compact**. « Raw » désigne ici les résultats directement triés selon les trois mesures affichées. La recherche adaptative explore des configurations, puis approfondit les zones les plus intéressantes selon le critère de tri choisi.

Choisir les cotations

Depuis un fichier individuel :

```
jedidemo tools score cotations_EXA.txt --sort=fitness --budget=40
```

Depuis une base de marché existante :

```
jedidemo tools score --database=demo --security=EXA --sort=fitness --budget=40
```

Le nom du fichier est un argument positionnel. En base, `--database` désigne la source et `--security` le ticker ou l'ISIN. `--universe` permet de choisir un autre univers que celui de la configuration.

Ce que signifie « fichier préparé »

Un fichier préparé est un historique acquis auprès d'une source identifiée, mis dans le format lu par Jedi et contrôlé avant utilisation. Pour un titre, la première ligne recommandée contient **ISIN ; ticker ; nom**. Les suivantes contiennent **ISIN ; date ; ouverture ; plus haut ; plus bas ; clôture ; volume**.

Extrait valide du fichier fictif livré `cotations_EXA.txt` :

```
FR4487601001;EXA;Exemple A
FR4487601001;20230630;10.0000;10.1800;9.8200;10.0000;60000
FR4487601001;20230703;10.1075;10.2895;9.9173;10.0990;61000
```

Ces deux cotations illustrent le format ; elles ne suffisent pas à une étude représentative. Le fichier livré contient 851 séances fictives, du 30 juin 2023 au 2 octobre 2026.

Chaque ligne doit concerner le même ISIN. Les prix emploient le point décimal ; les dates ci-dessus sont en année, mois, jour. Le fichier est ordonné chronologiquement pour faciliter les contrôles. Éviter les doublons, vérifier les séances manquantes et identifier les éventuelles corrections de cours.

Les exports d'un fournisseur peuvent demander une conversion avant lecture : conserver les colonnes nécessaires, compléter l'ISIN et adapter les dates. Les commandes `market` ne téléchargent pas ces fichiers. Le chapitre 10 distingue le fichier individuel du fichier de plusieurs titres destiné à l'import.

Pourquoi 750 séances après un profil sur 252 ?

`profile` examine le comportement récent du titre, principalement sur 252 séances, soit environ un an. `score` évalue le parcours de barèmes sur **750 séances par défaut**, soit environ trois ans. Cette période plus longue expose la mécanique à davantage de mouvements et de conditions historiques.

Ces réglages répondent à des questions différentes : « comment se comporte le titre récemment ? » et « comment ces barèmes ont-ils fonctionné sur l'historique choisi ? ». Ils ne constituent pas une démonstration que ces durées seraient toujours les meilleures.

Avec le fichier de démonstration, la commande précédente indique :

```
Historique : 2023-11-20 → 2026-10-02 (750 séances)
Montant demandé à l'ouverture : 1500.00 €
Recherche adaptive : 40 backtests, 11793 candidats disponibles (budget : 40)
Recherche partielle : le classement porte sur les barèmes évalués.
Classement : brut transparent (raw)
```

Si le fichier contient moins de 750 séances, le rapport indique la période réellement disponible. Le contrôle de trois années propre à `profile` n'est pas une règle supplémentaire de `score`.

Budget et recherche

Sans `--budget`, la recherche adaptative dispose d'un budget de **500 backtests de recherche**. Dans ce chapitre, 40 donne une démonstration courte. Ce budget compte les barèmes effectivement simulés, pas les lignes affichées.

La recherche ne teste pas nécessairement tous les candidats disponibles. Sa conclusion porte sur les configurations évaluées ; la première ligne n'est pas une preuve d'optimum global.

Un pourcentage d'avancement est affiché pendant les calculs. Le temps nécessaire dépend du budget, de la longueur de l'historique et des barèmes. Une recherche exhaustive `--search=grid` existe pour les usages détaillés ; elle peut être beaucoup plus longue.

Lire les trois colonnes compactes

Les résultats compacts `raw` sont limités aux **20 premiers barèmes**, ou moins si la recherche en a évalué moins. Les trois lignes suivantes sont un extrait de référence de l'étude fictive de l'édition 1.0, avec un budget de 40. Après correction du maximum intertrades en v1.1, relancer la commande pour obtenir les valeurs et le classement de votre version :

```
 Fitness  Op/mois  Inv.max  Constructeur typé
   0.433    0.644    1.000  I: kc:1.23,1.14,0.88,0.76, ki:0%, kb:1, kd:0.2
   0.416    0.532    1.000  I: kc:2.01,1.75,1.52,1.32,1.15,0.85,0.72,0.61,0.52,0.44, ki:0%, kb:1, kd:0.0
   0.414    1.960    1.050  I: kc:1.22,1.16,1.1,0.95,0.9,0.86, ki:0%, kb:1, kd:0.0
```

Colonne	Lecture
Fitness	Gain brut final divisé par le montant du premier achat. `0.433` signifie environ 43,3 % de ce premier achat sur toute la période.
Op/mois	Nombre de toutes les opérations, ouvertures comprises, ramené à 21 séances à partir de la première opération.
Inv.max	Investissement maximal divisé par le premier achat, hors frais. `1.050` signifie un pic de 1,05 fois ce premier achat.
Constructeur typé	Type I ou II, puis définition permettant de recréer le barème.

La fitness n'est ni un taux annuel ni un rendement net. Le gain brut inclut les titres encore détenus, valorisés à la dernière clôture. Une hausse du titre peut donc contribuer au résultat sans provenir d'allers-retours.

L'investissement maximal est le plus grand des pics d'achats moins ventes de chaque trade successif, hors frais, comme dans le backtest. Il peut dépasser le premier achat lorsque la mécanique demande des achats supplémentaires. Les frais doivent être examinés dans la lecture économique ou dans un backtest.

Choisir le tri

Option	Premier critère	Départage
`--sort=fitness`, défaut	Fitness décroissante	Activité décroissante, puis investissement maximal croissant
`--sort=activity`	Activité décroissante	Fitness décroissante, puis investissement maximal croissant

Option	Premier critère	Départage
`--sort=investment`	Investissement maximal croissant	Activité décroissante, puis fitness décroissante

Les tris comparent les mesures arrondies à trois décimales. Les anciens noms numériques `1`, `2`, `3` restent acceptés ; les exemples utilisent les noms explicites.

En recherche adaptative, le critère oriente aussi le raffinement. Changer de tri peut donc modifier les barèmes évalués, en plus de changer leur ordre. Pour comparer le parcours de barèmes fixes, utiliser leurs définitions dans `backtest`.

Une activité élevée augmente les interventions et peut donner davantage de frais. Une faible utilisation de capital et une forte fitness ne suffisent pas à vérifier l'opérabilité du projet réel.

Examiner la lecture économique

```
jedidemo tools score cotations_EXA.txt --sort=fitness --budget=40 --display=study
```

Cet affichage conserve le principe du classement et apporte le gain net, le résultat rapporté à l'investissement maximal, les frais et les opérations. Dans la première ligne de notre exemple, la fitness brute de 0,433 devient une fitness nette de 0,421 ; les frais affichés sont de 19 €.

Les colonnes `Rnet%` et `Rann%` rapportent le résultat net à l'investissement maximal. `Op/an` compte les opérations de barème hors ouvertures sur une base annuelle ; il ne remplace pas exactement `Op/mois`. Les montants arrondis de cet affichage servent à une lecture d'ensemble.

Les options `--ranking=jedi`, `--ranking=both` et `--display=technical` restent disponibles pour l'examen expert. Leurs détails figurent dans l'annexe 01. Le parcours courant commence par `raw`.

Examiner une autre période

```
jedidemo tools score cotations_EXA.txt --sort=fitness --select=l252 --budget=40
```

Cette commande utilise les 252 dernières séances. Dans notre exemple, la première ligne sur 750 séances a une fitness de 0,433 ; la première ligne sur 252 séances a une fitness de 0,126 et un autre constructeur.

Ces deux gains couvrent des durées différentes et peuvent concerner des barèmes différents. Leur simple comparaison ne démontre pas la supériorité d'une fenêtre. Fixer un barème, puis le simuler sur les périodes voulues, permet d'examiner son comportement.

Pour arrêter la recherche à une date donnée :

```
jedidemo tools score cotations_EXA.txt --date=le:2025-12-31 \
  --select=l750 --sort=fitness --budget=40
```

Le filtre de date est appliqué avant la sélection de lignes. Les codes sont `ge` pour « à partir de », `gt` pour « après », `le` pour « jusqu'à inclus » et `lt` pour « avant ».

Transmettre un barème à candidate

Le préfixe `I:` ou `II:` indique le type dans le rapport ; il ne fait pas partie de la valeur à transmettre. Reprendre le texte qui suit ce préfixe :

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2" \
  --amount=1000 --capital=5000
```

Cette définition est celle de la première ligne de démonstration. Le prix de 10 € et les montants sont fictifs. Le rapport `candidate` vérifie le projet actuel et affiche la définition opérationnelle du barème ; aucun achat n'est réalisé.

Vérifier un prix avec calc

Pour le barème de type I repris ci-dessus, Jedi peut vérifier le prix du premier échelon à la hausse :

```
jedidemo tools calc price \
  scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2" \
  position=0:0:1 c0=10 tick=0.01 dir=up
```

La sortie obtenue commence par :

```
price = round_up(cpe) = 11.4
  cpe = 11.4
```

La position cible est le premier échelon haussier, le prix de référence est de 10 € et le pas de cotation de 0,01 €. Ce calcul vérifie un prix du barème ; il ne simule pas tout le trade. Pour un barème de type II ou un enchaînement d'opérations, utiliser `tools play`.

Retenir une solution compréhensible

Lire d'abord les dates, le nombre de barèmes évalués et les mesures. Examiner ensuite quelques configurations, leur activité, les frais et les besoins de capital. Une fitness proche peut accompagner des rythmes très différents.

La simulation utilise le moteur de `backtest`, une ouverture par montant fixe, l'exécution `conservative` et des ordres actifs à la séance suivante après l'ouverture. Le montant demandé vient de la configuration de simulation ; il est affiché dans le rapport. Les exemples utilisent 1 500 € par ouverture avec la configuration par défaut de Jedi.

Conserver la définition retenue et les paramètres de l'étude. Vérifier son parcours sur une autre période puis évaluer le candidat au prix actuel. Modifier les dates ou les réglages après lecture des gains doit être considéré comme un nouvel essai.

6. Évaluer une ouverture avec tools candidate

Deux questions avant d'ouvrir

`candidate` évalue un projet de trade au prix fourni ou enregistré. Il vérifie :

- son **opérabilité** : les quantités permettent-elles d'obtenir les ordres utiles contrôlés ?
- son **financement** : les fonds libres couvrent-ils l'ouverture et les achats supplémentaires projetés ?

Ces deux résultats restent distincts. La commande ne recherche pas le titre et ne sélectionne pas son barème. Elle ne nécessite aucun historique OHLCV, ne crée pas le trade et ne passe aucun ordre.

Identifier le titre, le prix et le barème

L'argument du candidat est un **ISIN**, pas un ticker. Une cotation explicitement fournie par `--price` remplace le cours de la base principale. Elle permet même d'examiner un titre qui n'y est pas encore enregistré.

Sans `--price`, le titre doit posséder un cours enregistré dans la base principale. Le rapport indique l'origine et, pour ce cours enregistré, la date connue. La dernière clôture d'une base de marché n'est pas utilisée automatiquement.

Un prix saisi n'emprunte pas la date d'une ancienne cotation enregistrée. À l'utilisateur de vérifier la date et la pertinence du prix qu'il fournit.

`--scale` accepte le nom d'un barème existant ou une définition littérale. Après `score`, reprendre le constructeur sans son préfixe d'affichage `I:` ou `II:`.

Choisir le montant d'ouverture

Voici un exemple **fictif**, sur une base principale sans autre trade engagé :

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --amount=1000 --capital=5000
```

`--amount=1000` fixe le montant souhaité pour le premier achat. À 10 €, il correspond à 100 titres. Pour un prix qui ne divise pas exactement le montant, Jedi retient une quantité entière par défaut ; le montant réellement engagé peut être inférieur à la demande.

`--capital=5000` fournit le capital du portefeuille pour vérifier le financement. Ce capital et le montant d'ouverture ont des rôles différents.

Sortie obtenue :

```
Quantité initiale :              100
Montant réel d'ouverture :       1000.00 €

Opérabilité :                    OUI
Financement :                    OUI
Capital libre après réserve :    5000.00 €
Besoin maximal du candidat :     1187.83 €
Marge après candidat :           3812.17 €
Réserve du candidat calculée sur 3 échelons ; financement hors frais.
```

Le projet demande davantage que son achat initial : 187,83 € supplémentaires sont réservés pour le parcours projeté. Le barème est également affiché sous sa définition opérationnelle complète.

Dimensionner avec la gestion du capital

La seconde forme utilise `--money` à la place de `--amount` :

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --money="r:1%,o:1,n:2" --capital=10000 --extended
```

Les valeurs choisies servent ici à expliquer le calcul.

Paramètre	Signification
`r`	Part du capital utilisée pour calculer le risque monétaire.
`n`	Nombre d'échelons baissiers servant à calculer la distance du StopLoss.
`o`	Coefficient de l'objectif monétaire utilisé pour la protection des gains.

Avec 10 000 € et `r:1%`, le risque calculé est de 100 €. Le barème donne, à deux échelons sous 10 €, un cours théorique de 8,10 €, soit une distance de 19 %. Cette distance conduit à un montant théorique de 526,32 € et à 52 titres pour 520 €.

L'affichage étendu donne notamment :

```
Risque monétaire R :             100.00 €
Quantité initiale :              52
Montant réel d'ouverture :       520.00 €
Quantité minimale opérable :     32
Opérabilité :                    OUI
Besoin maximal du candidat :     617.56 €
Marge après candidat :           9382.44 €
```

Le calcul tient aussi compte de la limite de capital sur le trajet de risque et des quantités minimales utiles. Il peut refuser l'ouverture lorsqu'aucune quantité entière ne satisfait l'ensemble des contraintes.

Sans `--capital`, Jedi utilise le capital configuré pour les trades. Si aucun capital positif n'est disponible, le financement peut être affiché « non évalué » ; cela ne signifie pas qu'il a été validé.

Choisir r, o et n en pratique

Commencer par exprimer le risque en euros, puis le rapporter au capital choisi. Avec 10 000 €, 1 % représente 100 € et 2 % représente 200 €. Augmenter `r` augmente le risque accepté et peut augmenter la quantité ; ce n'est pas un moyen neutre de faire disparaître un refus.

Choisir `n` après lecture du barème : deux échelons ne représentent pas le même pourcentage selon la taille des mouvements. Un nombre d'échelons plus élevé donne généralement un StopLoss plus éloigné et, à risque monétaire identique, une quantité initiale plus faible.

La comparaison suivante reprend trois commandes exécutées sur notre exemple, en changeant un seul paramètre :

Capital	r	n	Distance calculée	Quantité	Achat initial	Besoin maximal hors frais
10 000 €	1 %	2	19,00 %	52	520 €	617,56 €
10 000 €	2 %	2	19,00 %	105	1 050 €	1 253,22 €
10 000 €	1 %	3	27,10 %	36	360 €	425,88 €

Voici les trois commandes complètes, dans le même ordre que les lignes de la table :

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --money="r:1%,o:1,n:2" --capital=10000 --extended

jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --money="r:2%,o:1,n:2" --capital=10000 --extended

jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --money="r:1%,o:1,n:3" --capital=10000 --extended
```

`o` multiplie le risque monétaire pour définir l'objectif utilisé par la protection des gains. Avec un risque de 100 €, `o:1` donne un objectif de 100 €, `o:2` de 200 €. Il ne fixe pas directement la quantité

initiale. Le déclenchement dépend du résultat projeté après `n` échelons de baisse et du parcours du trade ; ce n'est pas simplement « vendre dès que le gain courant atteint 100 € ».

Pour commencer les essais pédagogiques, garder `o:1,n:2` et modifier un paramètre à la fois rend les comparaisons lisibles. Ces valeurs d'exemple ne déterminent pas à elles seules un risque approprié à chaque investisseur. Le chapitre 18 et `tools play` montreront les effets sur les ordres de protection.

Un StopLoss peut être exécuté au-delà du prix calculé, notamment lors d'un gap. Le risque calculé ne constitue donc pas une borne garantie de perte. Examiner également le risque cumulé des trades engagés.

Comprendre l'opérabilité

L'opérabilité vérifie que les ordres utiles gardent une quantité suffisante. Le défaut est de deux titres minimum par ordre et de trois échelons haussiers contrôlés, selon `diagnostic_steps` dans la configuration.

Les options `--minimum-quantity` et `--operability-steps` permettent de changer ce contrôle. Réduire ces exigences change la question posée ; cela ne résout pas automatiquement les frais ou la liquidité.

Sur notre barème, le minimum initial calculé est de 32 titres. Avec un capital de 5 000 €, `r:1%,o:1,n:2` ne permet que 26 titres selon le risque. Le projet est refusé pour quantité insuffisante. L'affichage indique une quantité d'ouverture nulle et le motif ; la base dispose pourtant de capital libre.

Comprendre les réserves et le financement

Jedi prend le capital du portefeuille, retire l'investissement net courant, puis réserve les achats supplémentaires projetés des trades possédant encore des titres. Il reste le **capital libre après réserve**.

Pour le candidat, le besoin maximal est l'achat initial plus sa propre réserve. La **marge après candidat** est le capital libre après réserve diminué de ce besoin.

La projection explore les trajectoires à la hausse et à la baisse sur l'horizon retenu. Pour la réserve, cet horizon vaut `diagnostic_steps`, porté à `n` si le candidat utilise une gestion du capital avec un `n` supérieur. Il ne se confond pas avec l'option d'opérabilité.

Les frais sont exclus de cette projection de financement. Les besoins au-delà de l'horizon, les nouvelles décisions et les conditions réelles d'exécution peuvent changer le résultat. Garder une marge pour ces différences.

Avec la même ouverture de 1 000 € et seulement `--capital=1000`, on obtient :

```
Opérabilité :                  OUI
Financement :                  NON
Besoin maximal du candidat :   1187.83 €
Marge après candidat :         -187.83 €
```

Le premier achat est possible en montant, mais les fonds ne couvrent pas la réserve projetée. Un prix favorable ou un bon résultat historique ne change pas ce calcul.

Quand des trades ont déjà dépassé R

Une perte supérieure au risque calculé `R` peut survenir, par exemple après un gap ou une interruption du suivi. La gestion du capital `money` ne peut pas rétablir rétroactivement cette limite. Lors du calcul des ordres, Jedi peut signaler qu'un trade est hors contrôle lorsque sa perte au cours courant dépasse `R`.

`candidate` ne bloque pas automatiquement une nouvelle ouverture pour ce motif. Il ne vérifie pas les pertes courantes des autres trades par rapport à leur `R`. Il continue à compter tous les trades possédant encore des titres dans les réserves, même ceux qui ont dépassé ce risque. La réserve est projetée à partir de leur dernier état enregistré et de leur barème, sur l'horizon prévu ; elle ne suppose pas qu'un StopLoss a liquidé le trade.

Le financement utilise l'investissement net, c'est-à-dire les achats moins les ventes enregistrés dans la base principale. Il ne remplace pas cet engagement par la valeur des titres au cours courant. Une vente à perte enregistrée affecte donc les fonds calculés ; une perte encore latente n'est pas déduite comme un budget de risque supplémentaire. Le capital servant à calculer le `R` du candidat reste celui fourni par `--capital` ou la configuration. Il n'est pas recalculé automatiquement à partir des pertes des autres trades.

Avec plusieurs trades dans cette situation, `candidate` peut ainsi afficher « financement : OUI » si les fonds couvrent l'engagement et les réserves. Ce résultat valide ce calcul de financement ; il ne valide pas le risque cumulé du portefeuille ni la poursuite des trades en difficulté.

Pour examiner les alertes des trades existants, utiliser :

```
jedidemo trade orders
```

L'affichage étendu de `candidate`, obtenu avec `--extended` dans les commandes ci-dessus, détaille l'investissement net, la réserve totale et la réserve de chaque trade. Consigner les incidents et décisions exceptionnelles avec `note`, en identifiant le trade concerné ; enregistrer séparément les opérations réellement exécutées.

Lire un refus avant de modifier le projet

Résultat	À vérifier
Aucun cours disponible	Fournir un prix valide ou enregistrer une cotation du titre.
Quantité initiale nulle	Montant, prix et quantité entière.
Quantité insuffisante	Barème, quantités utiles et paramètres du risque.
Capital limitant	Achat initial et investissement projeté sur le trajet de risque.
Marge négative	Fonds libres et réserves des trades existants et du candidat.

L'option `--extended` donne les détails utiles. Les prix, montants et capitaux doivent être positifs et finis ; `--amount` et `--money` sont incompatibles.

Avant l'ouverture, conserver l'ISIN, le prix vérifié, la définition du barème, les paramètres et le rapport. Le chapitre 07 explique ensuite l'ouverture chez le courtier et l'enregistrement effectif dans Jedi.

7. Ouvrir un trade

Passer du projet aux opérations réelles

Le chapitre 06 a évalué un projet. `candidate` ne crée aucun titre, barème, trade ni achat dans la base principale. Il ne transmet aucun ordre au courtier.

L'ouverture comporte trois moments : préparer le trade dans Jedi, exécuter l'achat chez le courtier, puis enregistrer l'exécution réelle. Une opération simplement envisagée ne doit pas être enregistrée.

Ce chapitre reprend **EXA, un titre entièrement fictif** : achat initial de 100 titres à 10 €, barème avec mouvements de 10 %, capital de référence de 5 000 €. Utiliser une base principale d'essai vide, préparée au chapitre 02. Les fichiers historiques EXA peuvent exister dans une base de marché ; ils ne créent pas ce titre dans la base principale.

Vérifier le projet au prix retenu

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --amount=1000 --capital=5000
```

Dans une base sans autre trade engagé, le résultat donne :

Information	Valeur
Quantité initiale	100
Achat initial	1 000,00 €
Besoin maximal, ouverture et réserve hors frais	1 187,83 €
Marge après candidat	3 812,17 €

L'opérabilité et le financement sont validés pour cet exemple. Vérifier à nouveau le projet si le prix, le montant, les paramètres ou les engagements du portefeuille changent avant l'exécution.

Cet exemple fixe un montant avec `--amount` et laisse `money` désactivé. Si votre projet a été dimensionné avec `--money`, reprendre ses mêmes valeurs `r`, `o` et `n` lors de la création du trade. Le capital utilisé pendant le suivi vient de `trading.capital` : le `--capital` fourni à `candidate` n'est pas enregistré automatiquement pour le trade.

Enregistrer le titre et un cours courant

Vérifier d'abord si le titre existe :

```
jedidemo security list EXA
```

Dans notre base vide, le créer :

```
jedidemo security create --isin=FR4487601001 --name="Exemple A" --ticker=EXA \
  --tick-buy=0.01 --tick-sell=0.01 --tax-rate=0
jedidemo quote set FR4487601001 --price=10
```

Pour un titre réel, vérifier son ISIN, son libellé, son ticker, ses pas de cotation et sa taxe éventuelle. Les pas de 0,01 € et la taxe nulle sont des choix fictifs de cet exemple.

`quote set` enregistre le cours avec la date et l'heure de la saisie. Ce cours sert au suivi ; il ne remplace pas le prix réel de l'achat. Le prix fourni à `candidate` n'avait pas été écrit dans la base.

Choisir des noms lisibles

Pour un trade, utiliser **`<repère_du_titre>_<numéro>`** : `Exa_01`, puis `Exa_02` si un nouveau trade distinct est créé sur ce titre. Un repère plus long, comme `Airliquide_01`, convient aussi. Le numéro identifie le trade, pas ses paramétrages. Un split, un changement de barème ou une réinitialisation conserve ce nom.

Pour un barème, laisser Jedi attribuer **`B_kc_NN` pour le type I ou `B_vr_NN` pour le type II**, en omettant `--name`. Le type est déduit du builder et le numéro est calculé dans cette famille. Dans notre base vide, le premier barème de type II sera `B_vr_01`. Le chapitre 13 explique le nommage et le refus des définitions équivalentes.

Employer des lettres sans accents, des chiffres et des soulignés _, avec une lettre au début et au moins deux caractères. Les noms doivent être uniques dans leur catégorie et sont insensibles à la casse. Jedi normalise leur écriture : `EXA_01` devient `Exa_01`.

Les exemples de cette édition adoptent ces noms. Un trade déjà créé sous un ancien nom reste utilisable ; employer son nom réel dans les commandes ou le corriger avec `trade update --name`, selon le chapitre 09.

Conserver le barème retenu

Le paramétrage d'un trade demande un barème permanent de la base principale. Enregistrer sous un nom la définition utilisée pour le candidat :

```
jedidemo scale create \
  --comment="Exemple avec mouvements de 10 %" \
  --builder="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0"
jedidemo scale show B_vr_01
```

Si le barème existe déjà, le consulter et le réutiliser. Lorsqu'il vient de `score`, copier le constructeur sans son préfixe d'affichage `I:` ou `II:`. Le nom temporaire d'un barème littéral d'évaluation ne suffit pas à le conserver.

Créer le trade et son paramétrage initial

```
jedidemo trade create --name=Exa_01 --security=FR4487601001
jedidemo trade show Exa_01
```

Le nom identifie cette gestion et doit être unique. `--security` accepte l'ISIN ou l'identifiant interne du titre, pas son ticker. Si le projet utilise la gestion du capital, ajouter la définition évaluée, par exemple `--money="r:1%,o:1,n:2"`, à la commande de création.

Associer ensuite le trade au barème :

```
jedidemo settings create Exa_01 --scale=B_vr_01 \
  --date=2026-09-21 --time=09:00:00
```

Avant l'achat, ce premier paramétrage possède une position `0:0:0`, avec `q0` et `c0` à zéro. La première opération renseignera la quantité et le prix de référence. Les options d'adaptation `--split`, `--sync` et `--reinit` concernent un trade possédant déjà un paramétrage.

Respecter la chronologie de toute la base principale. Un nouveau paramétrage doit être strictement postérieur à la dernière opération et au dernier paramétrage, même s'ils concernent un autre trade. Une opération doit être postérieure au dernier paramétrage global et ne peut pas précéder la dernière opération globale. Les dates et heures saisies sont interprétées dans le fuseau du terminal.

À ce stade, Jedi connaît le projet mais aucun achat n'est encore enregistré.

Acheter chez le courtier et enregistrer l'exécution

Passer vous-même l'ordre d'achat, suivre son exécution et reprendre les valeurs confirmées par le courtier. Notre exemple suppose l'exécution de 100 titres à 10 €, avec 3,50 € de frais :

```
jedidemo operation create Exa_01 \
  --date=2026-09-21 --time=09:01:00 \
  --type=Buy --mode=Limit --quantity=100 --price=10 --fees=3.50
```

La première opération est un achat de mode `Limit` ou `Market`, selon l'ordre réellement exécuté. Sa date est distincte de celle du paramétrage. La quantité et le prix sont ceux de l'exécution ; ils peuvent différer de l'évaluation préalable.

`--fees` donne le total des frais de cette opération, taxe éventuelle comprise. Si l'option est omise, Jedi applique les frais configurés et la taxe du titre pour un achat. Lorsque l'avis d'opéré donne les frais définitifs, les saisir explicitement. Après enregistrement, `operation fees ID MONTANT` permet de corriger ce total sur toute opération ; le chapitre 08 explique comment retrouver son identifiant et vérifier la correction.

Un achat non exécuté ne crée aucune opération. S'il est exécuté en plusieurs fragments, attendre de pouvoir enregistrer une exécution consolidée ; le chapitre 08 explique la procédure.

Contrôler l’ouverture et les premiers ordres

```
jedidemo settings list Exa_01
jedidemo operation list Exa_01 -c 0,6-14,17
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Le stock doit être de **100 titres**, `q0` de 100, `c0` de 10 € et la position `0:0:0`. L’investissement net hors frais vaut 1 000 €. Comparer le stock et l’opération au relevé du courtier.

Les deux ordres de fonctionnement obtenus sont :

Type	Mode	Quantité	Cours	Montant hors frais
Buy	Limit	8	9,00 €	72,00 €
Sell	Limit	6	11,00 €	66,00 €

Ces lignes sont des propositions calculées. Il vous reste à placer les ordres chez le courtier et à surveiller leur exécution. Ne pas les enregistrer comme des opérations avant leur réalisation.

Pour examiner un mouvement de hausse puis de baisse sans toucher aux opérations de la base :

```
jedidemo tools play -c o0,6-14,17 -- \
  scale=vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0,q0=100,c0=10,tick=0.01 1 -1
```

Le parcours calculé vend 6 titres à 11 €, puis en rachète 6 à 9,90 € : le stock revient à 100. `play` sert ici à comprendre le barème ; il ne décrit pas des exécutions réelles et ne garantit pas ce parcours.

Conserver la définition du barème et le rapport d’ouverture. Le chapitre 08 reprend ce trade pour montrer le suivi d’une vente réelle et son enregistrement.

8. Suivre un trade au quotidien

Une boucle de suivi

Le suivi consiste à disposer d'un cours pertinent, lire l'état et les ordres, les maintenir chez le courtier, puis enregistrer seulement les exécutions réelles. Après une opération, recalculer les ordres.

Trois informations restent distinctes :

Information	Ce qu'elle décrit
Cours enregistré	Une observation de prix utilisée pour le suivi.
Ordres calculés	Les prochaines opérations proposées à partir de l'état du trade.
Opérations enregistrées	Les achats et ventes réellement exécutés.

Un changement de cours ne modifie pas le stock. Un ordre placé ou annulé chez le courtier ne modifie pas la base Jedi. Une exécution du courtier n'y apparaît qu'après votre enregistrement.

Ce chapitre continue **le trade fictif Exa_01 ouvert au chapitre 07** : 100 titres achetés à 10 €, barème `B_vr_01`. Les commandes d'enregistrement sont des essais à effectuer dans la base séparée du chapitre 02.

Disposer d'un cours pertinent

Pour les essais fictifs, saisir le prix voulu :

```
jedidemo quote set FR4487601001 --price=11
```

`quote set` enregistre le prix avec l'heure de saisie, la source `manual` et sans variation mémorisée. Il ne sait pas de quand date votre observation : vérifier vous-même que le cours correspond à la séance et au moment voulus.

Avec des fournisseurs configurés, les commandes courantes sont :

```
jedidemo quote update FR4487601001
jedidemo quote refresh
jedidemo quote refresh --mode=eod
```

`update` actualise un titre enregistré. `refresh` actualise, par défaut, les titres liés aux trades en cours. Pour employer la collecte sur un titre réel, remplacer l'ISIN fictif par celui de votre titre enregistré : EXA ne possède aucun cours auprès des fournisseurs de marché.

`--mode=eod` demande le mode de fin de journée ; `--mode=live` demande le mode de cours courant. Le contenu, les délais et la disponibilité dépendent du fournisseur. L'option de mode peut aussi être donnée à `quote update`.

Pour une consultation sans écrire le résultat dans la base principale :

```
jedidemo quote get FR4487601001
```

La configuration détaillée des fournisseurs figure dans l'annexe 02. La saisie manuelle permet de suivre le trade même si la collecte est indisponible. Les commandes `market update` du chapitre 10 entretiennent les historiques ; elles ne remplacent pas ces cours de suivi.

Les commandes de suivi peuvent tenter d'actualiser un cours dont la durée de conservation configurée est dépassée. En cas d'échec, un ancien cours peut être réutilisé avec une alerte. Lire cette alerte et contrôler la pertinence du cours avant d'utiliser les ordres.

Lire l'état et les ordres

```
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Comparer le stock affiché au stock du courtier. Lire pour chaque ordre son type `Buy` ou `Sell`, son mode, sa quantité, son prix et son montant. Avec le cours de 11 € et sans nouvelle opération, Exa_01 conserve encore 100 titres et propose les ordres du chapitre précédent.

Mode	Sens pratique
Limit	Ordre à cours limité.
Stop	Ordre de fonctionnement déclenché au franchissement d'un seuil.
StopLoss	Ordre de protection issu de la gestion du capital `money`.

Un Stop de fonctionnement et un StopLoss ont des rôles différents. Même si vous avez choisi un barème qualifié d'optimisé, lire le mode réellement proposé ; certains parcours peuvent produire un Stop de fonctionnement.

La marque **PRÊT** signale que le cours connu a franchi le prix de l'ordre dans le sens correspondant à son exécution. **À SUIVRE** signale une proximité selon le seuil configuré. Une égalité exacte de cours peut encore être marquée « à suivre ». Aucune de ces marques ne prouve une exécution chez le courtier.

Si Jedi signale des quantités insuffisantes, un cours trop ancien ou une gestion du capital inapplicable, examiner le motif avant de continuer. Le chapitre 06 précise le cas d'une perte supérieure à R.

Maintenir les ordres chez le courtier

Après le calcul, comparer les lignes de Jedi aux ordres réellement actifs. Annuler les ordres devenus caducs, puis modifier ou replacer ceux qui doivent l'être. Vérifier le récapitulatif du courtier : sens, mode, quantité, prix et durée de validité.

Une vente peut réduire le stock disponible pour un autre ordre de vente. Le courtier et Jedi ne coordonnent pas automatiquement les ordres entre eux. Après une exécution, vérifier les ordres restants avant de les conserver.

Lorsqu'un StopLoss et une vente Limit doivent coexister, `trade orders --net-stop` peut afficher une quantité de StopLoss diminuée de la quantité de la vente à la hausse. Cette option ne crée aucun lien d'annulation entre les ordres chez le courtier ; leur coordination reste à votre charge.

Enregistrer une vente réelle

Notre exemple suppose la vente confirmée de 6 titres à 11 €, avec 0,23 € de frais. L'enregistrer :

```
jedidemo operation create Exa_01 \
  --date=2026-09-22 --time=10:15:00 \
  --type=Sell --mode=Limit --quantity=6 --price=11 --fees=0.23
jedidemo operation list Exa_01 -c 0,6-14,17
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Le stock passe à **94 titres**. L'investissement net hors frais passe de 1 000 € à **934 €**, puisque la vente a rapporté 66 €. La position devient `0:0:+1`.

Voici les ordres recalculés, avec les arrondis au pas de cotation de notre exemple :

Type	Mode	Quantité	Cours	Montant hors frais
Buy	Limit	6	9,90 €	59,40 €
Sell	Limit	6	12,11 €	72,66 €

Reprendre ces nouvelles valeurs chez le courtier. Un ordre seulement expiré, annulé ou non exécuté ne crée aucune opération Jedi.

Traiter une exécution partielle

Plusieurs fragments d'un même ordre doivent être regroupés dans **une seule opération consolidée**. Chaque enregistrement d'opération peut faire avancer la position mécanique ; enregistrer les fragments comme des étapes successives fausserait le parcours de cet ordre.

Dans la vente de notre exemple, le courtier peut avoir exécuté :

Fragment	Quantité	Prix	Heure	Frais
Premier	2	11,00 €	10 :14 :00	0,10 €
Second	4	11,00 €	10 :15 :00	0,13 €
À enregistrer	6	11,00 €	10 :15 :00	0,23 €

La commande de vente ci-dessus enregistre cette consolidation. Si les prix diffèrent, utiliser leur moyenne pondérée par les quantités, le total des frais et l'heure du dernier fragment.

Tant que le solde peut encore être exécuté, attendre que la situation se stabilise avant l'enregistrement. Pendant cette attente, l'état Jedi est provisoirement en retard : ne pas s'en servir pour lancer une autre série d'ordres.

Si vous décidez de conserver et d'enregistrer seulement la partie exécutée, **annuler immédiatement le reliquat chez le courtier et vérifier l'annulation avant de replacer des ordres recalculés**. Reprendre uniquement la quantité effectivement exécutée. Le trade continue avec son stock réel.

Un prix moyen de plusieurs exécutions peut ne pas respecter le pas de cotation. Jedi contrôle normalement ce pas. Pour enregistrer un prix moyen attesté par le courtier sans l'arrondir artificiellement, le préfixe + désactive ce contrôle pour cette saisie : par exemple `--price=+11.013333`. Il ne majore pas le prix. Cette forme sert à reporter une exécution réelle, pas à contourner le contrôle des prix d'ordre.

Corriger les frais d'une opération enregistrée

L'avis d'opéré peut préciser des frais différents du montant saisi ou calculé automatiquement. La commande `operation fees` remplace le **montant total des frais** d'une opération déjà enregistrée, taxe éventuelle comprise.

Retrouver d'abord son numéro en affichant les opérations avec les frais, colonne `16` :

```
jedidemo operation list Exa_01 -c 0,5-9,16,17
```

Dans notre base fictive préparée sans opération préalable, la vente de 6 titres porte le numéro `2`. Si ses frais avaient été enregistrés à tort à 0,20 €, les corriger au montant confirmé de 0,23 € :

```
jedidemo operation fees 2 0.23
jedidemo operation list Exa_01 -c 0,5-9,16,17
jedidemo trade results Exa_01
```

Utiliser **l'identifiant de l'opération réellement affiché**, qui peut être différent dans votre base. Le montant est exprimé dans la monnaie du trade, avec un point décimal, puis arrondi au centime. `0` annule les frais. Le montant remplace la valeur précédente ; ce n'est pas une différence à ajouter.

La correction peut porter sur n'importe quelle opération, même ancienne ou appartenant à un trade clôturé. Les quantités, prix, stock, position, investissement hors frais et ordres restent identiques. Les listes et exports reprennent les frais corrigés ; `trade results` recalcule les frais cumulés et les résultats nets. Un nouveau `tools play` démarrant depuis cette opération ou une opération ultérieure du même trade reprend les frais historiques corrigés. `backtest` et `score` calculent leurs propres opérations simulées à partir de la configuration.

Si nécessaire, conserver le motif et l'ancien montant dans une note. Pour la suite de notre exemple, les frais de la vente restent **0,23 €**.

Garder une trace avec note

Les opérations donnent les mouvements du stock et de trésorerie. `note` permet de conserver le contexte d'un événement exceptionnel : exécution partielle, annulation, interruption du suivi, incident ou décision particulière.

Pour écrire plusieurs lignes, ouvrir les guillemets du contenu et les fermer seulement après la dernière ligne :

```
$ jedidemo note create 2026-09-22 "Exa_01 - vente partielle" "Vente de 6 titres :
> 2 puis 4 titres à 11 EUR. Exécution terminée à 10:15.
> Frais totaux : 0,23 EUR. Une seule opération consolidée enregistrée."
```

$ et > représentent les invites du terminal : ne pas les saisir. Tant que les guillemets restent ouverts, le terminal continue la même commande ; les retours à la ligne sont conservés dans la note. Cette syntaxe convient aussi aux textes longs qui seraient coupés à l'impression sur une seule ligne.

```
jedidemo note list Exa_01
```

Dans une base sans note préalable, cette note porte le numéro 1. Utiliser le numéro réellement affiché pour la consulter ou la compléter :

```
jedidemo note show 1
jedidemo note update 1 --edit
```

`--contents` reste disponible et remplace tout le contenu, qui peut aussi être saisi sur plusieurs lignes. Conserver le texte précédent lorsque vous le complétez.

Rédiger avec un éditeur

L'option **`--edit`** facilite la création ou la modification d'une note. Jedi utilise le programme indiqué par `VISUAL`, ou par `EDITOR` si `VISUAL` est absente. Ces variables doivent contenir le nom ou le chemin de l'exécutable, par exemple `nano`.

Dans le terminal Ubuntu ou Linux, si ce réglage n'a pas encore été fait au chapitre 02 :

```
export EDITOR=nano
export VISUAL=nano
```

Puis créer une note :

```
jedidemo note create --edit
```

L'éditeur ouvre un modèle contenant trois repères, à conserver dans cet ordre. Voici un exemple rempli ; les noms des repères restent en anglais, même pour une note française :

```
# Date
2026-09-22

# Summary
Exa_01 - vente partielle

# Contents
Vente de 6 titres : 2 puis 4 titres à 11 EUR.
Exécution terminée à 10:15 ; frais totaux 0,23 EUR.
Une seule opération consolidée enregistrée.
```

Dans nano, enregistrer avec **Ctrl+O**, confirmer le nom avec **Entrée**, puis quitter avec **Ctrl+X**. Quitter le modèle sans le modifier annule la création. Pour reprendre une note existante, `jedidemo note update 1 --edit` ouvre son contenu actuel : remplacer 1 par son identifiant. Ne pas combiner `--edit` avec les options de champs `--date`, `--summary` ou `--contents`.

Les notes constituent un journal général ; indiquer le nom du trade dans le résumé ou le texte permet de les retrouver. Il n'existe pas de rattachement automatique au trade. Une note ne remplace ni l'opération exécutée ni son enregistrement.

Après ce suivi, l'exemple Exa_01 possède 94 titres. Le chapitre 09 part de cet état pour présenter les adaptations possibles et la clôture.

9. Adapter ou terminer un trade

Partir d'un état vérifié

Une adaptation change les règles ou les références utilisées pour les prochaines opérations. Elle ne doit pas remplacer les opérations réelles du passé.

Avant d'intervenir, vérifier le stock, les dernières opérations et les ordres actifs chez le courtier. Annuler les ordres que le changement rendra caducs et confirmer leur annulation. Sauvegarder la base selon le chapitre 02.

À l'exception du cas distinct de regroupement de 1 790 titres présenté plus bas, les exemples ci-dessous sont **des choix séparés**, chacun partant du trade fictif Exa_01 à la fin du chapitre 08 : 94 titres, investissement net de 934 €, position `0:0:+1`, barème `B_vr_01`. Ne pas exécuter toutes les variantes successivement sur ce même état. Pour les comparer, conserver une sauvegarde de cet état et la restaurer dans une base d'essai vide avant chaque variante.

Choisir le changement nécessaire

Besoin	Commande ou option
Poursuivre avec un autre barème	`settings create` avec ce barème, sans option spéciale.
Donner une nouvelle origine mécanique au stock conservé	`--reinit=reset,c0`.
Relier la nouvelle origine à une transaction réelle	`--reinit=merge,c0`, puis l'opération.
Prendre en compte un fractionnement ou regroupement	`--split=NOUVEAUX:ANCIENS`.
Reprendre exceptionnellement une position de type I	`--sync=x:y:e`.
Modifier le nom ou la gestion du capital	`trade update`.
Corriger les frais d'une opération, même ancienne	`operation fees ID MONTANT`, selon le chapitre 08.
Terminer le trade	Enregistrer la vente réelle de tout le stock restant.

À partir de cette évolution de Jedi 1.1, **`--split` peut accompagner `--reinit=reset,prix` ou `--reinit=merge,prix` dans un seul paramétrage**. Le split s'applique d'abord au stock ; le prix fourni à `reinit` est exprimé dans la nouvelle unité du titre. `--sync` reste incompatible avec `--split` et avec `--reinit`. Un nouveau paramétrage doit respecter la chronologie globale rappelée au chapitre 07.

Le dernier paramétrage du trade doit avoir été utilisé par une opération avant d'en créer un autre. Cette condition concerne aussi un changement simple, un split ou un reset. Si le paramétrage inutilisé résulte d'une erreur de saisie, la correction de fin d'historique est expliquée plus bas ; ne pas inventer une opération pour satisfaire ce contrôle.

Changer de barème en conservant la continuité

Dans cette variante, enregistrer un barème avec mouvements de 12 %, puis créer le nouveau paramétrage :

```
jedidemo scale create --name=B_vr_02 \
  --comment="Exemple avec mouvements de 12 %" \
  --builder="vr:12%,np:3,nn:3,ki:10%,kb:1,kd:0"
jedidemo settings create Exa_01 --scale=B_vr_02 \
  --date=2026-09-23 --time=09:00:00
jedidemo settings list Exa_01
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Le stock reste 94. Le nouveau paramétrage conserve ici `q0=100` et `c0=10` ; ces références ne sont pas le stock et le cours courants. Il reprend l'échelon `+1`, avec une position `0:0:+1`.

Lors d'un changement simple, Jedi remet les compteurs de saut `x` et `y` à zéro et intègre leurs effets dans les coefficients cumulés. Il conserve donc la continuité des calculs sans recopier tous les compteurs. Les chapitres 12 et 16 expliquent cette mécanique.

Le nouveau barème doit pouvoir reprendre l'échelon conservé ; sinon Jedi refuse ce changement. Lire les nouveaux ordres avant de les placer.

Repartir du stock courant avec reset

Cette variante conserve les 94 titres et choisit un cours de référence fictif de 11 € :

```
jedidemo settings create Exa_01 --scale=B_vr_01 \
  --date=2026-09-23 --time=09:00:00 --reinit=reset,11
jedidemo settings list Exa_01
jedidemo trade orders Exa_01
```

Le nouveau paramétrage possède `q0=94`, `c0=11` et une position `0:0:0`. Aucun achat ni aucune vente n'est créé. Les opérations anciennes et l'investissement déjà engagé restent dans l'historique.

Choisir le nouveau cours selon une règle explicite, par exemple le cours constaté au moment de l'intervention. Un reset peut changer les prochains ordres ; il ne corrige pas un stock erroné et n'efface pas les pertes anciennes. Conserver dans une note le motif, le cours retenu et les ordres annulés.

Relier une réinitialisation à une opération avec merge

Cette autre variante suppose un achat manuel **réel** de 6 titres à 11 €. Créer le paramétrage, puis enregistrer cette exécution :

```
jedidemo settings create Exa_01 --scale=B_vr_01 \
  --date=2026-09-23 --time=09:00:00 --reinit=merge,11
jedidemo operation create Exa_01 \
  --date=2026-09-23 --time=09:01:00 \
  --type=Buy --mode=Market --quantity=6 --price=11 --fees=0.23
jedidemo settings list Exa_01
jedidemo trade status Exa_01
```

Après la création du paramétrage, les nouvelles références restent en attente. L'opération suivante les établit à partir du stock conservé, du cours indiqué et de la transaction. Dans cet exemple, le stock et `q0` passent à 100, `c0` vaut 11 € et la position est `0:0:0`.

Utiliser `merge` lorsque la transaction fait partie de la décision de réinitialisation. Tant que cette opération n'est pas enregistrée, `q0` et `c0` restent à zéro et **Jedi ne propose aucun ordre automatique**. Enregistrer l'achat ou la vente réellement exécuté pour établir la nouvelle base. La note doit décrire le motif et la transaction attendue.

Prendre en compte un split ou un regroupement

Un fractionnement modifie le nombre de titres et leur unité de prix sans achat ni vente. Pour deux titres nouveaux contre un ancien :

```
jedidemo settings create Exa_01 --scale=B_vr_01 \
  --date=2026-09-23 --time=09:00:00 --split=2:1
jedidemo quote set FR4487601001 --price=5.50
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Le stock de suivi passe de 94 à **188 titres**. `q0` passe de 100 à 200 et `c0` de 10 à 5 €. La position conserve l'échelon `+1` ; l'investissement net reste 934 €. Le cours fictif de 5,50 € est saisi dans la nouvelle unité de prix.

L'ordre du ratio Jedi est toujours **nouveaux : anciens** :

Événement	Option Jedi
10 titres nouveaux pour 1 ancien	`--split=10:1`
1 titre nouveau pour 10 anciens	`--split=1:10`

Pour un regroupement de 101 titres anciens à raison de dix contre un, il reste 10 titres nouveaux entiers et un droit fractionnaire de 0,1 titre, ou **rompu**. Consulter les modalités de l'événement pour son traitement. Jedi ne gère ni ce rompu ni son éventuel règlement en espèces. Comparer le stock entier au relevé du courtier et noter tout montant reçu ; cette somme reste extérieure aux résultats calculés du trade. Ne pas la transformer en opération fictive d'achat ou de vente.

Après l'événement, vérifier aussi le pas de cotation et tous les ordres. Le ratio et sa date d'effet doivent correspondre à l'événement réel.

Combiner un regroupement et une réinitialisation

Ce cas est **distinct des variantes portant sur les 94 titres d'Exa_01**. Supposons un trade fictif déjà enregistré, nommé `Exr_01`, qui conserve **1 790 titres** après sa dernière transaction. Le titre est regroupé à raison d'une action nouvelle pour dix anciennes, puis la reprise du suivi justifie une nouvelle référence. Le prix choisi après regroupement est de **12 €** ; il ne doit pas être divisé par dix dans la commande.

Pour l'illustration, le barème enregistré est `B_vr_03`, défini par `vr:10%,ks:0.8,1.2,ki:0%,kb:0`. Employer le barème réellement retenu pour votre trade et des dates respectant toute la base. Le chapitre 19 reproduit ce cas en mémoire, sans devoir créer `Exr_01`.

Avec **reset**, créer un seul paramétrage :

```
jedidemo settings create Exr_01 --scale=B_vr_03 \
  --date=2026-10-12 --time=09:00:00 \
  --split=1:10 --reinit=reset,12
jedidemo settings list Exr_01
jedidemo trade status Exr_01
jedidemo trade orders Exr_01
```

Le stock effectif et `q0` deviennent **179 titres**, `c0` vaut **12 €**, la position est `0:0:0` et les coefficients cumulés `kspc` et `kcpc` valent 1. Le regroupement ne crée aucune transaction et ne change pas les résultats financiers passés. Ordres et stops utilisent les 179 titres dès ce paramétrage ; le nouveau prix de reset sert immédiatement aux premiers calculs de type II.

Avec **merge**, choisir cette autre variante depuis le même état de départ :

```
jedidemo settings create Exr_01 --scale=B_vr_03 \
  --date=2026-10-12 --time=09:00:00 \
  --split=1:10 --reinit=merge,12
```

Le stock effectif est déjà de 179 titres, mais la référence reste en attente : `q0=0`, `c0=0`, aucun ordre automatique. Si l'investisseur fait réellement acheter **21 actions nouvelles à 10 €**, enregistrer l'exécution dans les nouvelles unités :

```
jedidemo operation create Exr_01 \
  --date=2026-10-12 --time=09:01:00 \
  --type=Buy --mode=Market --quantity=21 --price=10 --fees=0.74
jedidemo settings list Exr_01
jedidemo trade orders Exr_01
```

Le stock et `q0` deviennent **200**, et `c0=(179×12+21×10)/200`, soit **11,79 €**. Une vente réelle convient aussi : vendre 20 actions nouvelles à 14 € donnerait 159 titres et `c0=(179×12−20×14)/159`, soit **11,7484 €**. Ces références sont calculées hors frais ; les frais réellement payés sont enregistrés avec l'opération.

Le prix de référence, le prix d'exécution et la quantité négociée sont tous exprimés **après le split**. Jedi ne reconvertit pas ces prix. Dès qu'une transaction appartient au nouveau paramétrage, le split ne s'applique plus à son stock une seconde fois.

Les 1 790 actions ne produisent aucun rompu. Si le stock avait été de 1 793, la référence de reset ou merge aurait utilisé les **179 titres entiers** ; les droits fractionnaires et leur indemnisation éventuelle restent hors des résultats de Jedi, comme expliqué plus haut.

Cette combinaison ne permet pas deux paramétrages inutilisés successifs. Elle regroupe deux changements cohérents dans le même paramétrage, sans opération blanche ni conversion manuelle des ordres.

Resynchroniser exceptionnellement une position

Une interruption du suivi ou un gap important peut conduire à reprendre la gestion à une autre position théorique. Les opérations qui n'ont pas eu lieu ne doivent pas être enregistrées.

`--sync` est réservé aux **barèmes de type I**. Voici une variante fictive utilisant explicitement un tel barème :

```
jedidemo scale create --name=B_kc_02 \
  --comment="Exemple de type I pour sync" \
  --builder="kc:1.4,1.2,0.8,0.6,ki:10%,kb:1,kd:0"
jedidemo settings create Exa_01 --scale=B_kc_02 \
  --date=2026-09-23 --time=09:00:00 --sync=1:0:+1
jedidemo settings list Exa_01
jedidemo trade orders Exa_01
```

La position fournie `1:0:+1` décrit le raccord demandé. Jedi en intègre les effets et ouvre le paramétrage actif en `0:0:+1`. Il ne corrige pas le stock et n'ajoute aucune opération manquée.

Le choix de position demande de comprendre le barème et le cours de reprise ; il est approfondi dans la partie II. Avant cette intervention, résoudre les écarts de stock à partir des exécutions réelles. Noter la période d'interruption ou le gap, l'ancienne position, le cours et la position retenue.

Modifier le nom ou la gestion du capital

Conserver normalement le nom choisi au chapitre 07 pendant toute la vie du trade. Pour corriger un repère mal choisi, par exemple remplacer la forme courte `Exa_01` par `Exemplea_01` :

```
jedidemo trade update Exa_01 --name=Exemplea_01
```

Pour activer ou modifier `money` :

```
jedidemo trade update Exemplea_01 --money="r:1%,o:1,n:2"
jedidemo trade orders Exemplea_01
```

Ces valeurs utilisent le capital configuré, ici 5 000 €. Elles sont des choix d'illustration ; elles modifient les règles de protection et ne changent pas les opérations passées.

Si un stop mémorisé existe et que `money` change, Jedi exige sa réinitialisation explicite :

```
jedidemo trade update Exemplea_01 --money="r:1.5%,o:1,n:2" --reset-stop
jedidemo trade orders Exemplea_01
```

Lire la demande de confirmation. Cette option efface le stop mémorisé dans Jedi ; elle n'annule aucun ordre chez le courtier. Vérifier les ordres de protection avant et après le changement. Augmenter le risque pour rendre une perte acceptable à l'affichage ne répare pas le trade.

Corriger la fin de l'historique

Pour une erreur portant uniquement sur les frais, utiliser `operation fees ID MONTANT` : cette correction est possible sur toute opération, même d'un trade clôturé, sans remonter l'historique. Le chapitre 08 décrit la procédure.

Pour corriger les autres données d'une opération, il n'existe pas de modification libre d'un ancien enregistrement. La correction remonte la fin de l'historique global dans l'ordre inverse, puis recrée les enregistrements corrects.

```
jedidemo operation delete
```

Cette commande propose de supprimer **la dernière opération de toute la base**, pas une opération choisie dans Exa_01. Lire sa confirmation. Si un paramétrage plus récent existe, l'opération ne peut pas être supprimée avant lui.

Si ce dernier paramétrage n'a servi à aucune opération, la séquence devient :

```
jedidemo settings delete
jedidemo operation delete
```

Après contrôle des confirmations, recréer les éléments corrigés dans leur chronologie réelle et vérifier les ordres. Une suppression peut annuler un état de clôture ; elle doit servir à corriger une saisie, pas à réécrire une exécution. Consigner le motif et les valeurs corrigées dans une note.

Terminer le trade par sa vente réelle

Cette dernière variante repart des 94 titres à la fin du chapitre 08, sans appliquer les adaptations précédentes. Annuler les ordres incompatibles avec la sortie, faire vendre le stock chez le courtier et enregistrer l'exécution confirmée :

```
jedidemo operation create Exa_01 \
  --date=2026-09-23 --time=15:00:00 \
  --type=Sell --mode=Market --quantity=94 --price=11.20 --fees=3.68
jedidemo operation list Exa_01 -c 0,6-14,17
jedidemo trade list
jedidemo trade results Exa_01
```

Le stock devient **zéro** : le trade est clôturé. Il n'y a pas de commande administrative `trade close`. Selon l'ordre réellement exécuté, reporter le mode `Market`, `Limit`, `Stop` ou `StopLoss` approprié.

Les valeurs utiles de notre exemple sont :

Information	Valeur
Achats enregistrés hors frais	1 000,00 €
Ventes enregistrées hors frais	1 118,80 €
Gains hors frais	118,80 €
Frais cumulés	7,41 €
Stock final	0

Après déduction des frais, le gain de cet exemple est 111,39 €. Le taux annualisé affiché sur un parcours aussi court est une extrapolation ; il ne remplace pas ce bilan monétaire.

Si la vente finale reste partielle, appliquer la consolidation et l'annulation du reliquat expliquées au chapitre 08. Le trade reste ouvert tant qu'il possède des titres.

Une fois le stock nul, Jedi refuse une nouvelle opération sur ce trade. Pour une nouvelle gestion du même titre, créer un autre trade. Conserver le trade terminé : ses opérations, ses réglages, ses notes de contexte et ses résultats servent au contrôle ultérieur. `trade delete` refuse la suppression d'un trade possédant un historique.

10. Gérer la base de cotations

La base Demo des exemples

Ce parcours prépare les données utilisées dès les chapitres 03 et 04. Les fichiers sont livrés dans examples/, à la racine de l'archive de documentation. Suivre d'abord le chapitre 02 pour installer `jedidemo` et copier les fichiers dans `~/jedi-examples/`. Le lanceur ouvre ce dossier pour les commandes, quel que soit le répertoire du terminal. Les commandes ci-dessous supposent l'univers par défaut `france` et une base `demo_france.db` encore absente.

Exécuter les étapes dans l'ordre, jusqu'à la mise à jour des trois titres au 2 octobre. Les exemples de `profile` du chapitre 04 utiliseront alors ce dataset `Demo`. Les fichiers individuels `cotations_EXA.txt`, `cotations_EXB.txt` et `cotations_EXC.txt` peuvent aussi être étudiés directement, sans créer cette base.

Importer le dataset

Le répertoire `examples/` de ce lot contient des données **entièrement fictives**, avec un calendrier simplifié de jours de semaine. Avec les fichiers copiés dans le dossier de démonstration, les commandes suivantes créent une base `demo_france` qui doit être absente au départ :

```
jedidemo market init demo
jedidemo market import demo Demo Demo_initial.txt labels_demo.txt
jedidemo market info demo --dataset=Demo
jedidemo market list demo --dataset=Demo --last-quote
```

`init` crée la base. `import` crée le dataset ; un dataset déjà présent doit être mis à jour ou remplacé avec les commandes adaptées.

La première importation de démonstration contient trois titres et 2 550 cotations, jusqu'au 1er octobre 2026. Le rapport indique les compteurs et anomalies. Certaines cotations invalides ou sans libellé peuvent être signalées et ignorées : examiner le rapport et le journal.

Pour consulter deux séances d'un titre :

```
jedidemo market quote demo FR4487601001 --select=l2
```

`market quote` affiche l'historique de cet ISIN. Ce n'est pas `quote get`, qui interroge les fournisseurs de cours courant.

Mettre les cotations à jour

Une mise à jour apporte un fichier contenant de nouvelles séances, avec les libellés nécessaires. Elle fusionne les données du dataset existant.

Dans la démonstration, n'apporter d'abord que le cours du 2 octobre pour EXA :

```
jedidemo market update demo Demo Demo_partiel.txt labels_demo.txt
jedidemo market info demo --dataset=Demo
```

EXA est récent, les deux autres titres restent au 1er octobre. La sortie obtenue distingue :

```
Période du dernier import complet : 2023-06-30 → 2026-10-01
Période actuellement disponible   : 2023-06-30 → 2026-10-02
Date de fraîcheur globale (95 %)  : 2026-10-01
```

Mettre ensuite les trois titres à jour :

```
jedidemo market update demo Demo Demo_update.txt labels_demo.txt
jedidemo market info demo --dataset=Demo
```

Le doublon équivalent d'EXA est ignoré, les deux autres séances sont ajoutées. La référence globale passe au **2 octobre 2026**, tandis que la période du dernier import complet reste inchangée.

Ce parcours illustre l'entretien normal du dataset. Il n'est pas nécessaire de réimporter tout l'historique pour faire avancer sa fraîcheur.

Deux bases aux usages distincts

Base	Ce qu'elle conserve	Usage
Base principale Jedi	Titres, trades, paramètres, opérations, notes et cours enregistrés	Gestion des trades
Base de marché	Historiques OHLCV, datasets et groupes	Comparaison, recherche de barèmes et simulations

OHLCV signifie ouverture, plus haut, plus bas, clôture et volume par séance. Ces données décrivent un historique ; un cours courant décrit un instant.

La base de marché est facultative. Un fichier individuel peut alimenter `profile`, `score` ou `backtest`. La base facilite le travail sur de nombreux titres et leurs mises à jour.

Les commandes `market` importent des fichiers fournis par l'utilisateur. Elles ne les téléchargent pas. Importer ces historiques ne met à jour ni le cours de la base principale, ni les opérations des trades.

Source, univers, dataset et groupe

Notion	Exemple	Rôle
Source	`abcbourse`, ou `demo` pour nos exemples	Nom logique de l'origine des fichiers
Univers	`france`	Périmètre qui complète le nom de la base
Dataset	`Euronext_A`, ou `Demo`	Ensemble de titres et de cotations importé
Groupe	`PEA`, ou `EXEMPLES`	Liste logique de titres

Une source et un univers identifient un fichier SQLite, par exemple `abcbourse_france.db` ou `demo_france.db`. Le nom de la source ne déclenche aucune collecte.

Un même titre peut appartenir à plusieurs datasets et groupes. Des cotations peuvent être partagées entre datasets : les nombres affichés pour chaque dataset ne s'additionnent pas nécessairement.

Les datasets `Euronext_A`, `Euronext_B`, `Euronext_C`, `Euronext_Access` et `Euronext_Growth` appartiennent au contexte français utilisé pour préparer Jedi. Les commandes restent utilisables avec d'autres périmètres, en tenant compte des données et de leur unité monétaire.

Vérifier les emplacements

```
jedidemo config --paths
```

Avec `jedidemo`, les bases de marché sont normalement dans `~/jedi-examples/jedi/env_test/market/`. `JEDI_DEMO_MARKET_DIR` permet de sélectionner un autre répertoire. Avec `jedi`, le marché de production est normalement dans `~/.local/share/jedi/market/`, en tenant compte de `XDG_DATA_HOME`. Le chapitre 02 explique ces choix.

L'univers par défaut vient de `market_data.default_universe`, normalement `france`. Une option explicite le remplace :

```
jedidemo market info demo --universe=france
```

Employer le même univers dans toute une suite de commandes. Changer d'univers désigne une autre base physique.

Préparer les fichiers

« Préparer » signifie identifier la provenance, mettre les données dans le format attendu et vérifier leur cohérence. Il peut être nécessaire de convertir un export obtenu hors de Jedi. La licence du logiciel ne définit pas les droits d'usage des cotations fournies par un tiers.

Pour un import de dataset, préparer deux fichiers.

Le fichier de cotations contient **ISIN ; date ; ouverture ; plus haut ; plus bas ; clôture ; volume**, sans ligne d'identification à trois champs :

```
FR4487601001;20230630;10.0000;10.1800;9.8200;10.0000;60000
FR4487601019;20230630;30.0000;30.4800;29.5200;30.0000;30000
```

Le fichier de libellés contient **ISIN ; nom ; ticker** :

```
FR4487601001;Exemple A;EXA
FR4487601019;Exemple B;EXB
FR4487601027;Exemple C;EXC
```

Attention à l'ordre : un fichier individuel destiné aux outils commence, lui, par **ISIN ; ticker ; nom**, puis ses cotations :

```
FR4487601001;EXA;Exemple A
FR4487601001;20230630;10.0000;10.1800;9.8200;10.0000;60000
```

Ne pas confondre ces deux premières lignes. Le fichier individuel doit concerner un seul ISIN ; le fichier de dataset peut en contenir plusieurs. Éviter une ligne d'en-tête de colonnes dans le fichier de cotations. Le fichier de libellés accepte un en-tête `ISIN;Name;Ticker`.

Le lecteur de cotations accepte le point-virgule ou la virgule comme séparateur, et plusieurs formats de dates. Pour préparer un nouveau fichier, les dates `AAAAMMJJ` et le point décimal évitent les ambiguïtés. Pour des dates ambiguës, `--date-order=dmy` ou `ymd` précise l'ordre ; consulter l'aide de la commande.

Avant l'import, contrôler les ISIN, les dates, les colonnes, les prix positifs, le plus haut et le plus bas, les volumes non négatifs et les doublons. Les prix bruts et ajustés doivent être identifiés et employés de façon cohérente. Un import réussi ne prouve pas que les séances manquantes, les suspensions ou les corrections du fournisseur ont été traitées correctement.

Conserver les fichiers d'origine, la période couverte, les transformations, la liste des titres et la date d'acquisition. Prévoir assez d'historique pour les outils : `profile` étudie récemment 252 séances mais demande un historique plus long pour ses contrôles ; `score` en utilise 750 par défaut.

Les trois dates à comprendre

Information	Ce qu'elle décrit
Dernier import complet et sa période	L'opération d'import et la période de cotations apportée par cet import
Période actuellement disponible	Les plus anciennes et plus récentes cotations présentes pour le dataset
Date de fraîcheur globale	La date atteinte par au moins 95 % des titres

La référence globale est calculée à partir de la dernière cotation de chaque titre du dataset. Elle peut avancer avec quelques titres en retard, si au moins 95 % ont atteint la nouvelle date. Pour 600 titres, il en faut au moins 570 ; le cas de 595 titres récents et cinq retardataires satisfait cette condition.

Elle ne dépend pas simplement de l'heure du dernier appel à `market update`. Elle reste également une référence relative : si tous les titres sont anciens, vérifier la date face à la séance que l'on veut étudier.

`profile` utilise cette référence pour les retards individuels : zéro séance, vigilance à une ou deux séances, élimination au-delà de deux. Une sélection historique ou un dataset ambigu peut rendre ce contrôle indisponible ; le chapitre 04 précise ces cas.

Lire les anomalies et les trous

Une cotation déjà présente avec les mêmes valeurs est considérée comme un doublon équivalent. Une valeur contradictoire lors d'un `update` est signalée et ignorée ; la valeur enregistrée reste conservée. Une mise à jour n'est donc pas une procédure de correction des cotations anciennes conflictuelles.

Si la première nouvelle séance est située à plus de sept jours calendaires après la fin actuelle du dataset, l'extension est refusée sauf avec `--allow-gap` :

```
jedidemo market update demo Demo Demo_update.txt labels_demo.txt --allow-gap
```

Cette option accepte le trou ; elle ne le comble pas. Le contrôle porte sur l'extension globale du dataset et ne détecte pas toutes les lacunes de chaque titre. Consulter les séries individuelles lorsque le rapport ou le fournisseur indique un problème.

Remplacer un dataset et gérer les groupes

`market replace` sert à reprendre un dataset avec un nouveau corpus complet, par exemple après une correction de données. Vérifier les fichiers et garder une sauvegarde avant d'utiliser cette commande ; les données partagées avec d'autres datasets sont préservées.

Un groupe est créé ou synchronisé à partir d'une liste d'ISIN ou de tickers, un par ligne :

```
jedidemo market sync-group demo EXEMPLES groupe_exemples.txt \
  --as-of=2026-10-02 --reference-source=demonstration
jedidemo market list demo 'group=EXEMPLES' --last-quote
jedidemo tools profile --database=demo --dataset=Demo --group=EXEMPLES --rank-criteria=4
```

La synchronisation remplace la composition actuelle : un membre absent du fichier est retiré du groupe. La date `--as-of` décrit la référence de la liste, pas une reconstruction automatique de sa composition passée.

Maintenir les fichiers, lire `market info`, vérifier les dernières cotations avec `market list`, puis lancer les outils sur la population voulue. La base principale et les cours du suivi réel continuent d'être gérés séparément.

Comprendre la mécanique de Jedi

La première partie montrait comment préparer et suivre un trade. Cette partie explique comment un barème relie les mouvements de cours aux quantités de titres, aux ordres et au capital engagé.

Le prérequis est la connaissance du parcours pratique : titre choisi, barème, paramétrage, ordre et opération exécutée. Les notions plus précises sont définies au moment de leur emploi. Les formules viennent après une explication en mots courants.

Les chapitres 11 et 12 expliquent les flux, les positions, les échelons et les paliers. Les chapitres 13 à 15 présentent la structure des barèmes et leurs deux types. Le chapitre 16 traite la continuité lors d'un changement de paramétrage. Les chapitres 17 et 18 relient cet état aux ordres, au risque et au dimensionnement. Les chapitres 19 et 20 vérifient les calculs et suivent un exemple complet.

Préparer les exemples de mécanique

La plupart des exemples partent de **100 titres fictifs à 10 €**, avec un barème de type I et un pas de cotation de 0,01 €. Ce barème diffère du type II d'Exa_01 dans la partie I. Au chapitre 15, la comparaison utilise un type II à 20 %. Les chapitres 18 et 20 emploient aussi 125 titres pour montrer le dimensionnement.

Les parcours se calculent avec `tools play` et, pour un type I, `tools calc`. Ils ne nécessitent ni téléchargement ni import des fichiers de cotations. Les mouvements sont choisis pour montrer le calcul, sans prétendre représenter un marché réel.

Utiliser l'espace d'essai du chapitre 02. Les frais sont de **0,35 %, sans minimum**. Pour les protections du chapitre 18, ajouter dans la configuration de cet espace :

```
simulation:
  capital: 20000
```

Ce capital concerne `play`. Le capital de `candidate` est fourni explicitement par `--capital=20000`. Ses évaluations supposent une base sans trade ouvert. Les créations de barèmes pédagogiques et les notes écrivent dans la base d'essai ; les parcours définis directement dans `play` restent en mémoire.

Le guide des exemples rappelle ces conventions. Commencer par les flux du chapitre 11, puis suivre les chapitres dans l'ordre.

À la fin de cette partie, le lecteur pourra expliquer une variation de stock, comprendre un ordre Limit ou Stop, distinguer le StopLoss de protection et lire l'effet de `r`, `o` et `n`. Comprendre ces calculs permet de vérifier le processus ; leur résultat historique sera étudié dans la partie III.

11. Allers-retours et flux du trade

Un cours qui monte ne fait entrer de l'argent que si une vente a lieu. Un cours qui baisse ne fait sortir de l'argent que si un achat a lieu. Pour lire un trade, commencer par le prix d'un titre, le nombre de titres détenus et les achats ou ventes exécutés.

Ce chapitre utilise **Exemple A, un titre fictif**, avec 100 titres achetés à 10 €. C'est un exemple de type I, distinct du barème d'Exa_01 dans la partie I. Les prix sont choisis pour expliquer les calculs ; ils ne décrivent aucun marché réel.

Vendre puis racheter, ou acheter puis revendre

Un aller-retour comporte deux transactions opposées sur la même quantité. Après les deux transactions, le stock retrouve son niveau précédent.

Aller-retour achevé	Argent reçu	Argent dépensé	Écart hors frais
Vendre 20 à 12 €, puis racheter 20 à 10 €	240 €	200 €	+40 €
Acheter 20 à 8 €, puis revendre 20 à 10 €	200 €	160 €	+40 €

Ces écarts sont connus lorsque **les deux transactions ont eu lieu**. Après la vente seule, le rachat reste à faire ; après l'achat seul, la revente reste à faire. Une proposition d'ordre n'est pas un aller-retour achevé.

Voir le parcours dans Jedi

`tools play` déroule des mouvements choisis dans une simulation en mémoire. Les chapitres 13 et 14 expliqueront le barème. Lire d'abord les opérations obtenues :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1 -1 1
```

L'avertissement sur une future vente de très petite quantité concerne la poursuite de la hausse au-delà de cet aller-retour. Il est expliqué dans le chapitre 12.

`1` demande un mouvement haussier ; `-1` un mouvement baissier. `--` permet de transmettre les nombres négatifs. Les antislashs prolongent la commande, y compris à l'intérieur des guillemets.

Opération	Flux hors frais	Stock détenu	Investissement courant
Achat initial : 100 à 10 €	−1 000 €	100	1 000 €
Vente : 20 à 12 €	+240 €	80	760 €
Rachat : 20 à 10 €	−200 €	100	960 €
Achat : 20 à 8 €	−160 €	120	1 120 €
Revente : 20 à 10 €	+200 €	100	920 €

Une vente réduit le stock et rend de l'argent disponible ; un achat augmente le stock et consomme de l'argent. `play` exécute les ordres calculés aux prix de son scénario et n'enregistre aucune opération dans votre portefeuille.

Comprendre l'investissement et son maximum

L'**investissement courant** est le total des achats moins le total des ventes, hors frais. Il vaut 920 € à la fin : 1 360 € d'achats diminués de 440 € de ventes.

L'**investissement maximal historique** est le plus grand investissement courant déjà atteint. Il vaut 1 120 €, après l'achat à 8 €. La revente suivante ne fait pas disparaître ce maximum.

Ces nombres ne sont pas le solde du compte du courtier : ils suivent uniquement les flux du trade. Les futurs achats peuvent demander une réserve supplémentaire, étudiée au chapitre 18.

Ajouter la valeur des titres restants

Les deux allers-retours ont rapporté 80 € hors frais et il reste 100 titres. Le résultat dépend aussi de leur valeur au cours retenu :

```
valeur du stock = quantité détenue × cours retenu
résultat brut = valeur du stock − investissement courant
```

Évaluation du même stock final	Valeur	Investissement	Résultat brut
100 titres à 10 €	1 000 €	920 €	+80 €
100 titres à 9 €	900 €	920 €	−20 €

Les transactions bénéficiaires ne mettent donc pas les titres restants à l'abri d'une baisse. Le cours retenu évalue le stock ; il ne garantit pas le prix auquel tous les titres pourraient être vendus.

Tenir les frais à part

Jedi suit les frais séparément. Avec 0,35 %, sans minimum ni taxe sur ce titre fictif :

Opération	Frais
Achat initial	3,50 €
Vente à 12 €	0,84 €
Rachat à 10 €	0,70 €
Achat à 8 €	0,56 €
Revente à 10 €	0,70 €
Total	**6,30 €**

Au cours final de 10 €, le résultat après frais vaut **73,70 €**. L'investissement courant et son maximum restent exprimés hors frais. `trade results` distingue le résultat brut et les frais. Dans `play`, la sortie financière `f` est brute. Pour lire aussi les frais dans les opérations, ajouter la colonne `16` au sélecteur : `-c o0,6-14,16-17`.

Lorsque le retour ne se produit pas

Après la vente à 12 €, le cours peut continuer à monter sans permettre le rachat à 10 €. Le trade détient moins de titres pour la hausse suivante. Après l'achat à 8 €, il peut continuer à baisser sans permettre la revente : davantage de titres restent exposés.

Juste après cet achat, le stock vaut 120 et l'investissement 1 120 €. À un cours de 6 €, sa valeur serait 720 €, soit un résultat brut de −400 €. Le premier aller-retour bénéficiaire ne compense pas cette baisse.

Une variation de cours seule ne modifie ni le stock ni les flux enregistrés. Le chapitre 12 ajoute la position mécanique utilisée pour calculer la suite.

12. Positions, échelons et paliers

Le stock indique combien de titres sont détenus. La **position mécanique** indique où le trade se trouve dans son barème. Elle ne donne ni une quantité ni un résultat en euros.

Lire les trois nombres

Une position s'écrit `x:y:e` :

Nombre	Lecture
`x`	Nombre de sauts vers un palier supérieur depuis le début du paramétrage actif.
`y`	Nombre de sauts vers un palier inférieur depuis le début de ce paramétrage.
`e`	Échelon courant dans le palier atteint.

Les compteurs `x` et `y` sont positifs ou nuls. L'échelon `e` peut être positif, négatif ou nul. `1:0:+1` signifie un saut haussier, aucun saut baissier et l'échelon `+1`.

Un **échelon** est une étape locale du barème. Un **palier** est un niveau de référence à partir duquel une nouvelle série d'échelons se déroule. Le premier achat place le trade à `0:0:0`.

Dans certaines sorties, un nombre précède la position : `1  0:0:+1`. Ce premier `1` est le rang du paramétrage. La position reste composée de trois nombres.

Atteindre une borne et changer de palier

Le barème de type I du chapitre 11 a pour bornes `emax=+2` et `emin=-2`. Ses échelons intérieurs sont `−1`, `0` et `+1`.

Position actuelle	Mouvement	Position suivante
`0:0:0`	Hausse	`0:0:+1`
`0:0:0`	Baisse	`0:0:-1`
`0:0:+1`	Hausse atteignant la borne haute	`1:0:0`
`0:0:-1`	Baisse atteignant la borne basse	`0:1:0`

Quand une borne est atteinte, Jedi augmente le compteur correspondant et remet `e` à zéro. La position ne reste pas à `+2` ou `−2`. Un échelon zéro après un saut n'est pas un retour automatique au premier prix ou au premier stock.

Observer les deux sauts avec play

Ce barème pédagogique génère au saut supérieur une vente d'un seul titre à 14 €. Jedi affiche un avertissement après l'achat initial, puis « Fonctionnement de Jedi dégradé » après la première vente : l'ordre haussier suivant ne porte que sur un titre. Ces messages signalent une quantité trop faible pour un fonctionnement satisfaisant ; la simulation se poursuit et permet d'observer les positions ci-dessous. Le chapitre 06 explique ce contrôle.

Deux hausses, puis deux baisses :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 2 -2
```

Moment	Position	Prix simulé	Stock
Achat initial	`0:0:0`	10 €	100
Première hausse	`0:0:+1`	12 €	80
Deuxième hausse, saut supérieur	`1:0:0`	14 €	79

Moment	Position	Prix simulé	Stock
Première baisse depuis ce palier	`1:0:-1`	11,20 €	94
Deuxième baisse, saut inférieur	`1:1:0`	8,40 €	118

À la fin, `x=1` et `y=1` : les compteurs ne s'annulent pas. Les prix et stocks viennent du barème ; les seules coordonnées ne suffiraient pas à les calculer.

Position actuelle et position cible

Une **position cible** est celle qu'atteindrait une opération calculée. Depuis `0:0:+1`, la prochaine hausse viserait `1:0:0`, et la prochaine baisse `0:0:0`. Ce sont deux possibilités alternatives.

Placer, annuler ou laisser expirer un ordre chez le courtier ne change pas la position enregistrée. Jedi met à jour le parcours lorsque vous enregistrez l'exécution réelle. Le sens mécanique dépend du type et du mode de l'opération, comme l'explique le chapitre 17.

Quand le paramétrage actif a déjà servi, les ordres partent de sa dernière opération. Avant sa première opération, ils partent de la position inscrite dans le nouveau paramétrage.

Reprendre l'échelon dans un nouveau paramétrage

Un changement ordinaire conserve `e` et remet `x` et `y` à zéro. Une dernière opération à `1:1:-1` est ainsi reprise par un paramétrage commençant à `0:0:-1`.

Le stock, les flux et la position historique restent conservés. Les effets des sauts sont reportés dans des coefficients cumulés, étudiés au chapitre 16. La remise à zéro des compteurs ne supprime pas leur effet sur les calculs.

Les réinitialisations `reset` et `merge` établissent une nouvelle origine `0:0:0`. Une synchronisation de type I reprend une position choisie. Le chapitre 09 donne les commandes pratiques.

Le chapitre 13 présente le barème associant cette position à un cours et à un stock cibles.

13. Lire la structure d’un barème

Un barème associe les mouvements du cours à des variations du stock visé. Il ne décrit pas des transactions déjà réalisées. Jedi calcule une cible, puis la compare au stock effectivement détenu pour déterminer l’ordre suivant.

Deux références : c0 et q0

Au premier achat, `c0` est le prix unitaire et `q0` la quantité. Avec 100 titres achetés à 10 €, les références valent `c0=10` et `q0=100`.

Un paramétrage ultérieur peut les reprendre, les adapter à un split ou les remplacer lors d’une réinitialisation. **`c0` n’est donc pas toujours le cours de la première opération, et `q0` n’est pas toujours le stock initial.** Ce ne sont pas non plus automatiquement le dernier cours et le stock courant.

Le **cours théorique** est le prix associé à une cible mécanique. Le **stock théorique** est la quantité que le barème voudrait détenir à cette cible. L’ordre utilise ensuite un prix conforme au pas de cotation et une quantité entière. Les exécutions réelles peuvent différer de ces valeurs.

Lire les éléments du barème

Un coefficient est un multiplicateur : une référence de 10 € avec un coefficient de cours de 1,20 donne 12 €. Une référence de 100 titres avec un coefficient de stock de 0,80 donne 80 titres, avant les effets de palier.

Élément	Ce qu’il règle
`kc`	Liste des coefficients de cours d’un type I.
`vr`	Taux de variation du prochain cours d’un type II.
`ks`	Liste des coefficients de stock, dans les deux types.
`ki`	Ajustement de la référence théorique de capital lors d’un saut.
`kb`	Convention de cet ajustement ; nos exemples courants utilisent `kb:1`.
`kd`	Répartition des variations de stock lorsque Jedi construit la liste `ks`.
`np`, `nn`	Bornes haute et basse d’un constructeur compact de type II.

`ki` est distinct du risque `r` de gestion du capital. Un ajustement de palier de 10 % n’est ni un risque de portefeuille de 10 %, ni un gain promis.

Nommer et numéroter un barème

À la création, **Jedi calcule le nom à partir du type produit par le builder**. Omettre `--name` dans `scale create` permet ce nommage automatique.

Format du nom	Type opérationnel
`B_kc_NN`	Type I : cours déterminés par `kc`.
`B_vr_NN`	Type II : cours déterminés par `vr`.

Tous les constructeurs de type I partagent `B_kc_NN`, qu’ils reçoivent `ks` ou construisent le stock avec `kd`. Les trois constructeurs de type II partagent `B_vr_NN` : liste `ks`, construction avec `kd`, ou forme Legacy avec `sr` et `mi`. Dans le code actuel, Legacy est bien de **type II**.

`NN` est le plus grand numéro déjà présent dans cette famille, augmenté de 1. La numérotation est indépendante pour les deux types : `B_kc_01` et `B_vr_01` peuvent donc coexister. Elle s’affiche sur au

moins deux chiffres, puis continue avec `100`, `101`… Les numéros explicitement attribués sont pris en compte ; les trous de la séquence ne sont pas comblés tant qu'un numéro supérieur est présent.

Une définition opérationnelle équivalente est refusée, même si son constructeur est différent. Jedi compare le type, les coefficients de cours ou `vr`, les coefficients de stock, `ki`, `kb` et les bornes. Par exemple, `kc/kd` et `kc/ks` peuvent produire exactement les mêmes règles : ils correspondent alors à un seul barème à réutiliser. Le refus indique le nom et l'identifiant du barème existant. Il intervient avant l'attribution du nom automatique et ne consomme aucun numéro.

Le constructeur et le commentaire restent consultables avec `scale show`. Ils conservent la manière de construire le barème et son contexte ; le nom fournit son type et son numéro. Deux barèmes partageant quelques valeurs comme `vr` et `ki` peuvent encore avoir des définitions différentes.

`--name` reste disponible pour fixer une référence, mais le nom doit respecter le type du builder : `B_kc_NN` ou `B_vr_NN`, avec un numéro positif d'au moins deux chiffres. La casse et les zéros supplémentaires sont normalisés. Un numéro déjà attribué reste réservé, même si un ancien nom possède davantage de zéros. Les exemples fixent parfois un numéro pour conserver les références entre chapitres et variantes indépendantes ; l'usage courant consiste à laisser Jedi le choisir.

Les noms déjà enregistrés sont conservés. Un ancien barème peut être réutilisé sous son nom réel, ou renommé explicitement avec `scale update --name` dans le format correspondant. Les doublons présents dans une ancienne base restent également conservés, y compris lors de la restauration d'une sauvegarde. Les nouvelles créations et les modifications de définition ne peuvent pas en introduire d'autres ; `--force` ne contourne pas ce contrôle.

Pour changer une règle déjà utilisée, créer un nouveau barème puis lui associer un nouveau paramétrage préserve la définition de référence des anciennes opérations. Si la définition d'un barème numéroté est modifiée vers l'autre type, conformément aux protections de `scale update`, Jedi calcule son nouveau nom dans la famille correspondante, sauf si un nom conforme est fourni explicitement.

Développer un type I simple

Enregistrer ce barème pédagogique dans une base d'essai :

```
jedidemo scale create --name=B_kc_01 \
  --comment="Barème pédagogique de type I" \
  --builder="kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1"
jedidemo scale show B_kc_01
```

Créer ce nom une seule fois. S'il existe, consulter le barème avant de le réutiliser. Les autres exemples donnent souvent la définition directement à l'outil, sans créer de nom permanent.

Jedi ajoute les coefficients 1 à l'échelon zéro :

Échelon	Coefficient de cours kc	Coefficient de stock ks
+2, borne supérieure	1,40	Saut vers le palier suivant
+1	1,20	0,80
0	1,00	1,00
−1	0,80	1,20
−2, borne inférieure	0,60	Saut vers le palier suivant

Dans le premier palier, +1 vise 80 titres à 12 € et −1 vise 120 titres à 8 €. Aux bornes, Jedi change de palier et calcule de nouvelles références.

Jedi accepte des bornes allant de deux à six mouvements dans chaque sens, soit de un à cinq échelons intérieurs par côté. En type II compact, `np:3,nn:3` signifie trois mouvements jusqu'à chaque borne, avec deux échelons intérieurs de chaque côté.

Distinguer les deux types

Question	Type I	Type II
Prochain prix	Avec `kc` et les références du palier.	En appliquant `vr` au dernier prix d'exécution pertinent.
Stock cible	Avec `ks` et les références de stock.	Avec `ks` et les références de stock.
Hausse puis baisse	Dans un même palier, les coefficients désignent les mêmes niveaux.	Les pourcentages repartent du dernier prix ; le retour peut viser un autre niveau.

Les chapitres 14 et 15 calculent ces différences. Le type ne prédit pas un résultat financier.

Constructeur et définition développée

Le **constructeur** est le texte fourni à Jedi. La **définition canonique** est la règle développée et normalisée qu'il produit.

Forme	Champs principaux
Type I, stock saisi directement	`kc`, `ks`, `ki`, `kb`
Type I, stock construit par Jedi	`kc`, `ki`, `kb`, `kd`
Type II, stock saisi directement	`vr`, `ks`, `ki`, `kb`
Type II, stock construit par Jedi	`vr`, `np`, `nn`, `ki`, `kb`, `kd`
Type II Legacy	`vr`, `np`, `sr`, `nn`, `mi`, `ki`, `kb`

La définition développée expose toujours `ks`, même avec un constructeur utilisant `kd`. Le type I expose `kc`, le type II expose `vr`. `scale show` permet de lire le résultat avant de l'utiliser.

Comprendre le mot optimisé

Jedi vérifie si la structure réduit le stock à la frontière haute et l'augmente à la frontière basse. Il emploie **optimisé** pour ce contrôle structurel.

Cette qualification ne mesure ni la rentabilité, ni les frais, ni les besoins du portefeuille. Avec des arrondis ou certains parcours de type II, il faut encore lire les ordres effectivement calculés. Le chapitre 17 montrera un Stop de fonctionnement.

Un constructeur optimisé est une méthode pour fabriquer `ks`. Une liste saisie directement peut aussi réussir le contrôle ; c'est le cas de `B_kc_01`.

Pour changer une règle déjà utilisée, créer un nouveau barème permet de conserver l'interprétation des anciennes opérations. Le chapitre 16 explique comment lui associer un nouveau paramétrage daté.

14. Comprendre un barème de type I

Le type I associe chaque échelon à un coefficient de cours `kc` et à un coefficient de stock `ks`. Dans un même palier, ces coefficients donnent des niveaux connus. Un saut change les références auxquelles ils s’appliquent.

Nous reprenons 100 titres à 10 €, un pas de 0,01 € et le barème :

```
kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1
```

Calculer les premiers niveaux

Dans le premier palier, multiplier les références par les coefficients :

Position	Cours théorique	Stock théorique
`0:0:0`	10 € × 1 = 10 €	100 × 1 = 100
`0:0:+1`	10 € × 1,20 = 12 €	100 × 0,80 = 80
`0:0:-1`	10 € × 0,80 = 8 €	100 × 1,20 = 120

Depuis 100 titres, la hausse propose de vendre 20 et la baisse d’acheter 20. Revenir à zéro, avant tout saut, retrouve le prix de 10 € et le stock théorique de 100.

Passer au palier suivant

À la borne +2, le nouveau prix de référence vaut 10 € × 1,40 = 14 €. La référence théorique de capital est multipliée par `1+ki`, soit 1,10. Elle passe de 1 000 à 1 100 €.

Le stock théorique à l’origine du nouveau palier vaut 1 100 € ÷ 14 €, soit **78,571429 titres** environ. Il décrit une cible de calcul, pas un stock fractionnaire détenu.

À la borne −2 atteinte directement depuis le premier palier, le prix de référence vaut 6 € et la référence de capital 900 €, avec `kb:1`. Le stock théorique devient 900 € ÷ 6 € = **150 titres**.

Ces valeurs ne sont ni l’investissement courant, ni le solde du compte, ni le gain du trade.

Vérifier les multiplicateurs avec calc

Jedi nomme `kcp` le multiplicateur de cours du palier, `kcb` celui de sa référence de capital et `ksp` celui du stock. Leur relation est `ksp = kcb / kcp`.

Pour le premier saut haussier :

```
jedidemo tools calc cpe \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:0:0 c0=10
jedidemo tools calc spe \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:0:0 q0=100
```

On retrouve `cpe=14` et `spe≈78.571429`, respectivement cours et stock théoriques.

Après un saut dans chaque sens, la référence de cours est multipliée par 1,40 × 0,60 = 0,84. La référence de capital est multipliée par 1,10 × 0,90 = 0,99 :

```
jedidemo tools calc kcb \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:1:0
```

La valeur obtenue est 0,99. Ces deux sauts ne rétablissent pas les références du premier palier.

Suivre les transactions entières avec play

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 2 -2
```

Opération	Position	Quantité	Prix	Stock après opération
Buy Market	`0:0:0`	100	10 €	100
Sell Limit	`0:0:+1`	20	12 €	80
Sell Limit	`1:0:0`	1	14 €	79
Buy Limit	`1:0:-1`	15	11,20 €	94
Buy Limit	`1:1:0`	24	8,40 €	118

La cible de 78,571429 est arrondie à 79, puis comparée aux 80 titres détenus : la vente porte sur un titre. La dernière baisse part du palier supérieur ; elle n'est pas une baisse directe depuis l'ouverture.

Le contrôle structurel optimisé n'assure donc pas des ordres assez grands après arrondi. L'opérabilité et les frais restent à examiner.

Laisser Jedi répartir les variations de stock

Le constructeur compact remplace la liste `ks` par `kd` :

Cet exemple reprend `B_kc_02`, également utilisé pour la synchronisation du chapitre 09. Si ce nom existe déjà dans votre base d'essai, consulter sa définition avec `scale show` et réutiliser le barème correspondant. La commande de création ci-dessous sert à l'enregistrer une seule fois.

```
jedidemo scale create --name=B_kc_02 \
  --comment="Construction pédagogique" \
  --builder="kc:1.4,1.2,0.8,0.6,ki:10%,kb:1,kd:0"
jedidemo scale show B_kc_02
```

Créer ce nom une seule fois dans la base d'essai. Avec `kd:0`, la répartition est régulière entre les intervalles. Jedi produit **`ks:0.89,1.25`** : les stocks cibles intérieurs sont 89 et 125 pour `q0=100`.

C'est un autre barème que `ks:0.8,1.2`. Un `kd` positif donne davantage de poids aux intervalles tardifs ; un `kd` négatif aux premiers. Consulter le barème développé pour examiner les quantités.

Situer le barème historique Qdt

`Qdt` est conservé pour la compatibilité. Sa définition canonique est :

```
kc:1.61,1.38,1.23,1.14,0.88,0.76,0.64,0.52,0.4,
ks:0.4,0.68,0.87,1.9,2.9,4.1,5.5,ki:14%,kb:0
```

Il utilise `kb:0` : le saut baissier emploie l'inverse de `1+ki`. Ses bornes sont `+4` et `−5`. Il échoue au contrôle optimisé et peut demander un achat dans une hausse ou une vente dans une baisse.

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.61,1.38,1.23,1.14,0.88,0.76,0.64,0.52,0.4,\
ks:0.4,0.68,0.87,1.9,2.9,4.1,5.5,ki:14%,kb:0,\
q0=100,c0=10,tick=0.01" 4
```

Juste avant le saut supérieur, il reste 40 titres. Le saut vise 71 titres à 16,10 € : **achat Stop de 31 titres**. Sur un trajet baissier distinct, le saut inférieur demande une vente Stop de 331 titres, de 550 vers 219 à 4 €.

Ces Stops font fonctionner le barème ; ils diffèrent des StopLoss de protection. Le chapitre 17 explique leur mode. Le chapitre 15 présente d'abord le type II.

15. Comprendre un barème de type II

En fonctionnement ordinaire, le type II calcule le prochain prix en appliquant un taux `vr` au **prix de la dernière opération exécutée**, adapté à un split si nécessaire. Il conserve les positions et les coefficients de stock, sans liste de coefficients de cours `kc`.

Après un `reinit=reset`, le prix de référence fourni à reset remplace immédiatement celui de la dernière opération pour les premiers ordres. Si le reset accompagne un split, ce prix est déjà exprimé après le split. Les opérations suivantes reprennent ensuite leur prix réellement exécuté comme base. Avec `merge`, Jedi attend d'abord la transaction manuelle qui établit la nouvelle référence. Les chapitres 09 et 16 présentent ces cas.

Notre exemple fictif utilise `vr:20%` pour rendre les différences lisibles, avec 100 titres à 10 € et un pas de 0,01 € :

```
vr:20%,ks:0.8,1.2,ki:10%,kb:1
```

Calculer le prochain prix

Depuis une exécution à 10 €, le prochain prix théorique vaut 12 € à la hausse et 8 € à la baisse. Après une exécution à 12 €, une baisse vise 12 € × 0,80 = **9,60 €**.

Une hausse de 20 % puis une baisse de 20 % ne revient pas à 10 €. Les pourcentages s'appliquent à des prix différents. Une baisse de 20 % puis une hausse de 20 % donne également 9,60 € depuis 10 €.

Le cours courant du marché situe les ordres ; il ne remplace pas le dernier prix exécuté dans cette formule.

Rejouer les deux allers-retours

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=vr:20%,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1 -1 1
```

Opération	Position	Prix au pas	Stock
Achat initial	`0:0:0`	10 €	100
Vente de 20	`0:0:+1`	12 €	80
Rachat de 20	`0:0:0`	9,60 €	100
Achat de 20	`0:0:-1`	7,68 €	120
Vente de 20	`0:0:0`	9,22 €	100

Le dernier cours théorique est 7,68 € × 1,20 = 9,216 €. Pour cette vente Limit, il est arrondi vers le haut à 9,22 €. Le calcul suivant repartira du prix exécuté de 9,22 €.

Les achats totalisent 1 345,60 €, les ventes 424,40 € et l'investissement courant 921,20 €. Les 100 titres à 9,22 € valent 922 € : résultat brut **0,80 €**. Les deux allers-retours ont pourtant apporté 78,80 € de trésorerie ; la baisse de valeur du stock compte aussi.

Comparer au type I illustre la mécanique. Les mêmes directions produisent d'autres prix ; ce n'est pas un classement historique de rentabilité.

Comprendre le recalage au saut

Les bornes restent +2 et −2. Au saut, Jedi recale la référence de cours du nouveau palier sur le prix du parcours. Pour l'ordre, il utilise le cours théorique ; pour l'opération enregistrée, le prix réellement exécuté.

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=vr:20%,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 2 -2
```

Opération	Position	Quantité	Prix	Stock
Buy Market	`0:0:0`	100	10 €	100
Sell Limit	`0:0:+1`	20	12 €	80
Sell Limit	`1:0:0`	4	14,40 €	76
Buy Limit	`1:0:-1`	16	11,52 €	92
Buy Limit	`1:1:0`	15	9,21 €	107

Au saut supérieur, la référence théorique de capital est 1 100 €. À 14,40 €, le stock théorique vaut environ 76,388889, arrondi à 76 pour l'ordre. Au dernier saut, 9,216 € est arrondi vers le bas à 9,21 € pour l'achat.

Les arrondis de prix influencent les calculs suivants. Un calcul sans arrondi et un parcours exécuté au pas peuvent donc se distinguer après plusieurs mouvements.

Construire le stock avec np, nn et kd

```
jedidemo scale create --name=B_vr_04 \
  --comment="Construction pédagogique" \
  --builder="vr:20%,np:2,nn:2,ki:10%,kb:1,kd:0"
jedidemo scale show B_vr_04
```

Créer ce nom une seule fois. `np:2,nn:2` fixe deux mouvements jusqu'à chaque borne, avec un échelon intérieur par côté. `kd:0` répartit régulièrement les variations. La définition développée contient **`ks:0.88,1.2`**.

Elle diffère donc de notre première liste `ks:0.8,1.2`. Lire `scale show` avant de comparer les ordres. Les autres valeurs de `kd` modifient la répartition comme en type I.

La forme historique utilisant `sr` et `mi` reste disponible pour compatibilité, avec ses propres règles. L'annexe 01 décrit cette syntaxe ; le parcours courant utilise les formes ci-dessus.

Lire les ordres même si le barème est optimisé

Le contrôle optimisé de type II s'appuie sur une suite de mouvements de même sens. Après des oscillations, le prix au saut peut différer de cette suite de référence.

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=vr:20%,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1 1 -1 1 1
```

Juste avant le dernier saut, il reste 80 titres après une vente à 11,06 €. Le saut vise un cours théorique de 13,272 € et un stock cible entier de 83. Jedi simule **un achat Stop de 3 titres à 13,28 €**, bien que le barème ait réussi le contrôle optimisé.

Ce qualificatif ne remplace pas la lecture des modes et quantités. Le chapitre 16 examine la reprise du trade dans un nouveau paramétrage.

16. Assurer la continuité entre paramétrages

Un trade peut changer de barème sans recommencer son histoire. Achats, ventes, frais et titres détenus restent ceux des opérations enregistrées. Le nouveau paramétrage doit retrouver des références cohérentes avec la position atteinte.

Le chapitre 09 donne les commandes de suivi. Nous expliquons ici ce que Jedi conserve et recalcule.

Changer le repère local

Un changement ordinaire conserve l'échelon `e`. Les compteurs de paliers `x` et `y` repartent de zéro. Les sauts déjà parcourus sont reportés dans des coefficients cumulés :

Coefficient	Ce qu'il conserve
`kcpc`	Multiplicateur de cours provenant des paramétrages antérieurs.
`kspc`	Multiplicateur de stock provenant des paramétrages antérieurs.

Leur valeur initiale est 1. Un changement ordinaire conserve `c0` et `q0`, qui ne deviennent pas automatiquement le dernier prix et le stock détenu. Il transporte les multiplicateurs nécessaires à la continuité.

Les anciennes positions ne sont pas réécrites. Le nouveau paramétrage emploie un autre repère local, identifié par son rang dans les listes.

Vérifier un changement sans modifier le barème

Trois hausses de notre type I conduisent de 100 titres à 10 € à 63 titres à 16,80 €, en position `1:0:+1`. Réappliquer le même barème montre la continuité sans changer les règles :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 3 \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 1
```

Moment	Position dans le repère utilisé	Stock	Dernier prix exécuté
Avant changement	Ancien paramétrage, `1:0:+1`	63	16,80 €
Départ du nouveau	Nouveau repère, `0:0:+1`	63	16,80 €
Hausse suivante	Nouveau repère, `1:0:0`	62	19,60 €

Au changement, `kcpc=1.40` et `kspc=1.10/1.40≈0.785714`. Avec `c0=10` et `q0=100`, les références locales à zéro correspondent à 14 € et 78,571429 titres théoriques. L'échelon conservé +1 retrouve 16,80 € et une cible proche des 63 titres détenus.

La hausse suivante vend **1 titre à 19,60 €**. Comparer sans changement :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 3 1
```

Prix et quantité sont identiques. La position finale est `2:0:0` : le premier saut reste dans les compteurs initiaux.

Modifier réellement les règles

Un autre barème emploie le même report. Le parcours passé est conservé ; les nouvelles règles fixent les prochaines cibles et peuvent changer prix, quantité ou mode des ordres.

Pour le type II, la dernière opération intervient aussi dans le recentrage du chapitre 15. Reconstituer les ordres avec le dernier cours seul ferait perdre une partie du contexte.

`tools calc` ne prend pas l'historique des changements ni les coefficients cumulés d'un trade. Pour vérifier un changement, utiliser `play` avec le parcours complet, ou consulter les paramétrages et les ordres du trade réel.

Distinguer report et réinitialisation

Action	Intention
Changement ordinaire	Conserver `q0`, `c0` et l’échelon ; reporter les sauts.
`reinit=reset,prix`	Nouveau départ mécanique avec le stock détenu et le prix choisi.
`reinit=merge,prix`	Nouveau départ incorporant l’opération complémentaire suivante.
`split=nouveau:ancien`	Adapter les références à une division ou un regroupement.
`sync=x:y:e`	Imposer une position justifiée pour un type I.

Une réinitialisation remet à zéro la position de départ et les coefficients cumulés. Elle n’efface pas les pertes et gains enregistrés. `sync` conserve l’échelon imposé dans le nouveau repère ; ce n’est pas une transaction.

Après la vente de 20 à 12 €, réinitialiser à 12 € utilise les **80 titres détenus** :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 reinit=reset,12 -1
```

La baisse suivante achète 16 à 9,60 €, portant le stock à 96. Sans reset, le retour aurait acheté 20 à 10 €.

Pour incorporer un achat complémentaire de 20 à 12 € :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 reinit=merge,12 "20@12.00" -1
```

Les références deviennent 100 titres à 12 €. La baisse suivante achète 20 à 9,60 €, pour un stock de 120. L’achat complémentaire est bien une opération ; il ne résulte pas du seul changement de réglage.

Appliquer le split avant la réinitialisation

`split + reinit=reset` et `split + reinit=merge` sont acceptés dans un même paramétrage à partir de cette évolution de Jedi 1.1. Le stock de la dernière opération est converti d’abord. Les prix fournis à `reinit` et à la transaction suivante sont déjà dans la nouvelle unité : ils ne subissent pas une conversion supplémentaire.

Pour 1 790 anciens titres regroupés à raison de `1:10`, le stock effectif est de 179 titres. Avec un prix de reprise de 12 € :

État	Stock effectif	q0	c0	Position de départ
Après `split=1:10,reinit=reset,12`	179	179	12 €	`0:0:0`
Après `split=1:10,reinit=merge,12`, avant transaction	179	0	0	`0:0:0`
Merge, après achat de 21 nouveaux titres à 10 €	200	200	11,79 €	`0:0:0`

Les coefficients cumulés repartent à 1 dans les deux réinitialisations. Pour merge, le prix de 12 € reste conservé dans l’option `reinit` pendant l’attente ; il sert au calcul pondéré de la transaction manuelle. Les ordres automatiques ne sont disponibles qu’après celle-ci.

Les opérations anciennes gardent leurs unités et leurs positions. Avant la première transaction du nouveau paramétrage, la synthèse, les ordres et les projections utilisent son stock effectif et sa nouvelle position. Après cette transaction, le stock enregistré est déjà converti ; Jedi ne réapplique pas le split.

`split + sync`, `sync + reinit` et les trois options réunies restent refusés. Un paramétrage doit toujours être utilisé par une opération avant d'en créer un autre. Les exemples pratiques figurent au chapitre 09 et les simulations complètes au chapitre 19.

Contrôler le changement

Conserver l'état avant modification et examiner les nouvelles propositions dans une simulation. Après enregistrement, relire paramétrages, opérations et ordres comme au chapitre 09.

Un changement doit avoir une raison explicite : nouveau barème étudié, opération sur titres, correction ou nouvelle décision de suivi. Une note en conserve le contexte. Les ordres du courtier doivent ensuite être adaptés à l'état réellement exécuté ; la continuité des calculs ne réalise pas cette adaptation.

17. Passer de l’état du trade aux ordres

Un paramétrage plus récent que la dernière transaction peut déjà modifier le stock effectif ou la position de départ. Après `split + reset`, Jedi utilise immédiatement le stock converti, la position `0:0:0` et le prix de reprise. Après `split + merge`, il attend la transaction manuelle avant de calculer les ordres automatiques. Voir le chapitre 09.

Après une opération, Jedi calcule les cibles voisines du barème. Il transforme chaque cible de stock en une quantité à négocier et chaque cours théorique en un prix compatible avec le pas de cotation.

Le courtier exécute les ordres ; Jedi ne lui transmet rien. Une opération est enregistrée quand une exécution a réellement eu lieu, comme au chapitre 08.

Calculer la quantité à négocier

Le stock théorique est arrondi au titre entier le plus proche, puis comparé au stock détenu :

```
quantité signée = stock cible arrondi − stock détenu
```

Un résultat positif demande un achat ; un résultat négatif une vente de sa valeur absolue. Depuis 100 titres, une cible de 80 donne `−20` : vendre 20 titres.

Jedi ne vend donc pas toujours une fraction fixe de `q0`. Il rejoint une cible depuis le stock effectivement enregistré, y compris lorsque l’exécution précédente a différé de la théorie.

Lire Limit et Stop

Le sens du mouvement et celui de la transaction déterminent le mode :

Cible atteinte par	Transaction	Mode
Une hausse	Vendre	Sell Limit
Une baisse	Acheter	Buy Limit
Une hausse	Acheter	Buy Stop
Une baisse	Vendre	Sell Stop

Un Stop de fonctionnement peut donc venir du barème. Il ne signifie pas nécessairement liquidation ou protection du capital.

Au premier échelon de notre type I, vendre 20 à 12 € ou acheter 20 à 8 € sont deux propositions Limit.

Observer un Stop de fonctionnement

Une autre liste `ks` peut inverser le sens attendu. Garder les cours et remplacer le stock par `ks:0.4,1.8` :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.4,1.8,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 2
```

La première hausse vend 60 à 12 €, laissant 40 titres. Le saut vise 79 à 14 € : **achat Stop de 39 titres**.

Sur un trajet baissier distinct :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.4,1.8,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" -2
```

La première baisse achète 80 à 8 €, portant le stock à 180. Le saut inférieur vise 150 à 6 € : **vente Stop de 30 titres**. Le trade n’est pas liquidé ; il reste 150 titres.

Le contrôle optimisé aide à repérer certaines inversions. Il ne garantit ni des quantités suffisantes après arrondi ni l’absence de Stop sur tous les parcours de type II.

Arrondir au pas de cotation

La cible d'une hausse est arrondie vers le haut, celle d'une baisse vers le bas, avec le pas correspondant à l'achat ou à la vente.

Une référence de 10,03 € multipliée par 1,20 donne 12,036 €. Avec un pas de 0,05 € :

```
jedidemo tools calc price \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=0:0:+1 c0=10.03 tick=0.05 dir=up
jedidemo tools calc price \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=0:0:+1 c0=10.03 tick=0.05 dir=down
```

On obtient **12,05 €** avec `dir=up` et **12,00 €** avec `dir=down`. La seconde commande montre l'autre arrondi du même nombre ; elle ne transforme pas `+1` en cible baissière. Déclarer le pas du titre et du marché concernés.

Examiner les petites quantités

Un stock initial apparemment important peut produire un petit ordre :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 3
```

Les trois hausses vendent 20, **1**, puis 16 titres. Avec le contrôle par défaut de deux titres minimum, la quantité 1 signale un fonctionnement dégradé. Une quantité nulle bloque la direction et l'ordre n'est pas ajouté aux propositions. Un avertissement peut aussi annoncer une petite quantité à venir.

Le chapitre 18 montre pourquoi le contrôle d'ouverture ne dispense pas de relire les ordres après dimensionnement et pendant le suivi.

Distinguer le StopLoss

Avec la gestion du capital `money`, Jedi peut proposer une vente StopLoss du stock. Son état indique `CL` pour contrôler la perte ou `GP` pour protéger un gain calculé. L'option de quantité nette peut tenir compte de l'ordre de vente haussier.

Ce StopLoss découle d'une projection financière et de l'état du trade. Ce n'est pas un troisième échelon. Il peut supprimer les ordres ordinaires situés à son prix ou plus bas. Le chapitre 18 explique son calcul et ses limites.

Les indicateurs de prix de `trade orders` situent les propositions par rapport au cours connu. Ils ne prouvent pas une exécution et ne garantissent pas son prix. Après une exécution partielle, un gap ou une annulation, contrôler le stock, les ordres encore actifs et les opérations enregistrées.

18. Comprendre le capital, le risque et le dimensionnement

Un projet doit répondre à deux questions : quelle quantité acheter au départ, et combien de capital garder disponible pour les achats suivants ? La gestion du capital donne un cadre de risque au dimensionnement. `tools candidate` vérifie aussi l'opérabilité et le financement, comme au chapitre 06.

Nos montants décrivent des simulations. Le risque théorique ne garantit pas la perte maximale effectivement obtenue : frais, arrondis ou exécution défavorable peuvent l'augmenter.

Après un split, les gains courants et les quantités des stops utilisent le **stock converti**, même si aucune transaction nouvelle n'a encore été enregistrée. Avec reset, le trajet de protection part de la nouvelle position mécanique. Ces adaptations ne remettent pas le résultat financier à zéro : les achats, ventes et frais passés restent conservés. Avec merge, les propositions automatiques attendent la transaction manuelle, selon le chapitre 09.

Donner un sens à K, r, o et n

Paramètre	Question pratique	Effet
`K`	Quel capital est affecté au trading ?	Base monétaire des calculs.
`r`	Quelle fraction accepte-t-on de risquer par trade ?	Risque monétaire `R = K × r`.
`n`	À quelle distance du barème examine-t-on la baisse ?	Prix à `n` échelons et distance de dimensionnement.
`o`	Quel gain projeté faut-il dépasser avant de le protéger ?	Objectif monétaire `O = o × R`.

Avec `K=20 000 €`, `r=2,5 %`, `n=2` et `o=1`, le risque et l'objectif valent chacun 500 €. `o=2` porterait l'objectif à 1 000 € sans changer la quantité initiale.

Avec sa configuration d'origine, Jedi accepte `r` entre 0,5 % et 5 %, `n` entre 1 et 5, et `o` entre 0,5 et 5. Ces bornes sont configurables. Nos valeurs sont des choix d'illustration à examiner, pas des réglages conseillés pour tout portefeuille.

Une définition `money` exige les trois champs `r`, `o` et `n` : aucun n'est complété implicitement. Sans `money`, le barème fonctionne sans les protections décrites ici.

Utiliser le bon capital

Pour un trade réel, le capital vient de `trading.capital`. Pour **`tools play`**, il vient de **`simulation.capital`**. `candidate --capital=20000` impose le capital de cette évaluation sans modifier la configuration.

Dans `play`, le taux de risque est le champ `r` de `money`, obligatoirement fourni ; `simulation.risk_rate` et `simulation.distance` n'y interviennent pas. Ils servent au montant initial de référence de `score` et du backtest sans `--capital`. L'annexe 05 donne les valeurs intégrées et l'usage de chaque réglage.

Avant les exemples de protection, utiliser l'espace d'essai du chapitre 02 et vérifier dans son fichier de configuration :

```
simulation:
  capital: 20000
```

Les frais restent à 0,35 %, sans minimum. Le capital de 5 000 € d'Exa_01 dans la partie I est un autre exemple. Modifier seulement `trading.capital` ne modifie pas le capital de `play`.

Dimensionner une ouverture

Notre type I entre à 10 €. Deux échelons de baisse conduisent à 8 €, puis 6 €. La distance depuis l'entrée est de 40 %.

Le montant issu du risque vaut `R/d`, avec `d` exprimé en fraction : 500 € ÷ 0,40 = **1 250 €**, soit **125 titres** à 10 €.

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:1,n:2 --capital=20000 --extended
```

L'ISIN est celui du titre fictif de démonstration. Le prix est fourni par la commande ; aucune cotation téléchargée n'intervient.

Résultat utile	Valeur
Prix théorique à deux échelons	6,00 €
Distance de risque	40 %
Risque monétaire	500,00 €
Quantité initiale	125
Montant d'ouverture hors frais	1 250,00 €
Quantité minimale opérable annoncée	82
Besoin maximal du candidat hors frais	1 855,60 €
Marge après candidat hors frais	18 144,40 €

Jedi compare les contraintes de risque, de capital et de minimum opérable. Le risque limite ici la quantité à 125. La quantité maximale selon le capital sur le trajet de risque est 1 492 ; ce nombre n'autorise pas à ignorer le plafond de risque.

Le prix théorique de 6 € sert au dimensionnement. Ce n'est pas une instruction permanente de vente à 6 € : le StopLoss évolue avec les opérations.

Garder une réserve pour les achats futurs

L'ouverture immobilise 1 250 €, mais le trajet baissier sans protection demande d'autres achats :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -3
```

État	Stock	Investissement net hors frais
Ouverture à 10 €	125	1 250,00 €
Achat de 25 à 8 €	150	1 450,00 €
Achat de 38 à 6 €	188	1 678,00 €
Achat de 37 à 4,80 €	225	1 855,60 €

`candidate` retient ici trois échelons et une réserve future de **605,60 €** en plus de l'ouverture. Cet horizon vient du contrôle opérationnel et peut dépasser `n`. La projection examine les achats entiers du barème ; elle ne suppose pas que le StopLoss les interrompra.

Les réserves des autres trades ouverts sont déduites du capital libre. Les frais sont exclus de cette projection. Prévoir aussi les frais et examiner les besoins au-delà de l'horizon affiché.

Une ouverture imposée de 1 250 € avec seulement 1 700 € de capital reste opérable, mais donne une marge de **−155,60 €** :

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --amount=1250 --capital=1700 --extended
```

Pouvoir payer l'achat initial ne suffit donc pas à financer le scénario.

Choisir ses paramètres

Commencer par le capital affecté et le montant de perte accepté pour le projet ; en déduire `r`. Examiner ensuite les prix correspondant à `n` avec le barème choisi : deux échelons ne représentent pas toujours 40 %.

Ces comparaisons changent un seul paramètre à la fois. Chaque commande est complète :

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:1%,o:1,n:2 --capital=20000
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:5%,o:1,n:2 --capital=20000
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:1,n:1 --capital=20000
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:1,n:3 --capital=20000
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:2,n:2 --capital=20000
```

Réglage	Risque R	Prix à n échelons	Quantité retenue	Objectif O
`r:1%,o:1,n:2`	200 €	6,00 €	0 : refus	200 €
`r:2.5%,o:1,n:2`	500 €	6,00 €	125	500 €
`r:5%,o:1,n:2`	1 000 €	6,00 €	250	1 000 €
`r:2.5%,o:1,n:1`	500 €	8,00 €	250	500 €
`r:2.5%,o:1,n:3`	500 €	4,80 €	96	500 €
`r:2.5%,o:2,n:2`	500 €	6,00 €	125	1 000 €

À `r:1%`, la contrainte de risque ne permet pas d'atteindre le minimum opérable annoncé. Jedi refuse l'ouverture et retient zéro titre. Augmenter `r` pour obtenir « OUI » augmente aussi le risque monétaire : ce doit rester une décision de risque.

Augmenter `n` éloigne ici le prix de risque et réduit la quantité initiale. Réduire `n` augmente la quantité et les besoins de financement. Choisir `n` en examinant les prix, les mouvements habituels du titre et la réserve nécessaire.

Vérifier ensuite les ordres entiers. Le minimum annoncé n'assure pas que toute quantité supérieure donne de grands ordres : les arrondis peuvent faire varier les petits écarts irrégulièrement. Avec `q0=100`, trois hausses vendent 20, 1 et 16 ; avec `q0=125`, elles vendent 25, 2 et 19. Le contrôle d'ouverture actuel ne revalide pas chaque ordre de la quantité finale ; compléter sa lecture avec `play` et les diagnostics de suivi.

Comprendre le contrôle de perte

Pendant le trade, Jedi projette le résultat après `n` échelons de baisse, avec les achats et ventes demandés par le barème. Si ce résultat est inférieur à `−R`, il calcule une protection `CL` pour viser cette perte monétaire.

```
jedidemo tools play -c o0,6-14,17rf -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01,money=r:2.5%,o:1,n:2,loss=fallback" -2
```

À l'ouverture, un StopLoss de 125 à 6 € coexiste avec l'achat ordinaire à 8 €. Après l'achat de 25 à 8 €, le stock vaut 150 et l'investissement 1 450 €. La protection est recalculée à `(1450−500)/150`, soit environ 6,333333 €, arrondi à **6,33 €**.

L'achat ordinaire à 6 € est supprimé, car sous la protection. La simulation vend 150 à 6,33 € : **500,50 € de perte brute** et **8,40 € de frais**, soit **508,90 € de perte après frais**.

`loss=fallback`, défaut de `play`, suit d'abord l'ordre baissier ordinaire disponible, puis le StopLoss. `loss=active` donnerait priorité au StopLoss dès la première baisse : la chronologie simulée diffère.

Un gap sous la protection peut aggraver la perte. Si le cours connu a déjà dépassé la protection ou la perte acceptée, Jedi signale le problème ; il ne garantit pas de rattraper une vente non exécutée.

Comprendre la protection des gains

La protection `GP` commence quand le **gain projeté après `n` baisses** dépasse strictement `0`. Un gain actuel de 500 € ne suffit pas nécessairement pour `o:1` et `R=500 €`.

Pour observer `o`, simuler 25 allers-retours de 100 titres entre 10 et 12 €. Ce scénario volontairement répétitif montre le seuil ; il n'annonce pas 25 occasions réelles de gain :

```
jedidemo tools play -c o0,6-14,17r -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01,money=r:2.5%,o:1,n:2" "25x1,-1"
jedidemo tools play -c o0,6-14,17r -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01,money=r:2.5%,o:2,n:2" "25x1,-1"
```

Les deux parcours finissent avec 100 titres à 10 € et 1 000 € de gain brut. Avec `o:1`, des StopLoss de protection des gains apparaissent pendant le parcours. Avec `o:2`, aucun n'apparaît : le gain projeté n'a pas dépassé 1 000 €.

Une fois `GP` établie, Jedi conserve le gain monétaire protégé ou le relève si la projection s'améliore. Son prix est recalculé quand le stock change. Cette protection est calculée hors frais et reste soumise à l'exécution réelle. Un `o` plus grand retarde ici son activation ; il ne promet pas un gain plus grand et ne fixe pas un prix de vente immédiate.

Après ces vérifications, `tools backtest` permet d'observer le barème sur des cotations historiques. Ses hypothèses sont expliquées au chapitre 22.

19. Vérifier avec tools calc et tools play

Une valeur isolée aide à comprendre un coefficient. Un parcours montre comment les opérations s'enchaînent, avec leurs arrondis, leurs changements de paramétrage et leurs protections. Jedi fournit les deux outils : `calc` pour un calcul de type I, `play` pour une simulation de type I ou II.

Une simulation définie directement dans la commande ne crée aucun trade et n'enregistre aucune opération dans la base principale. Les commandes `scale create` et `note create`, utilisées ailleurs dans cette partie, écrivent en revanche dans cette base : garder l'espace d'essai du chapitre 02.

Choisir le calcul utile

`tools calc` reçoit un barème de type I, une position et une fonction. La position s'écrit `x:y:e`, comme au chapitre 12.

Fonction	Valeur calculée	Information supplémentaire
`kcb`	Coefficient de capital de base	Position
`kcp`	Coefficient de cours	Position
`ksp`	Coefficient de stock	Position
`cpe`	Cours théorique	`c0`
`spe`	Stock théorique	`q0`
`quantity`	Quantité entière à acheter ou vendre	`q0` et stock actuel
`price`	Prix arrondi au pas de cotation	`c0`, `tick` et sens

Au premier palier supérieur, le stock théorique de notre type I vaut environ 78,571429 titres :

```
jedidemo tools calc spe \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:0:0 q0=100
jedidemo tools calc quantity \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:0:0 q0=100 stock=80
```

Jedi arrondit le stock cible à 79. Depuis 80 titres, la quantité de l'ordre est donc **−1**, soit une vente d'un titre. Le résultat théorique décimal ne constitue pas une quantité d'ordre exécutable.

`calc` ne reproduit pas les frais, les protections ni la continuité d'un historique de paramétrages. Il refuse les barèmes de type II, dont les prix dépendent des exécutions précédentes. Pour ces situations, utiliser `play`.

Définir un parcours avec play

La première donnée décrit le barème, le stock de référence, le cours de référence et le pas de cotation. Les suivantes décrivent les actions :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1
```

Le séparateur `--` permet de transmettre les mouvements négatifs comme arguments. Les guillemets réunissent la définition en un seul argument ; le `\` final prolonge la ligne, sans espace ajouté au début de `q0`.

L'ouverture porte sur 100 titres à 10 €. La hausse vend 20 à 12 €, puis la baisse rachète 20 à 10 €. Le stock revient à 100 et l'investissement net hors frais devient 960 €.

Action	Interprétation
`1, -1`	Un échelon de hausse ou de baisse
`2, -2`	Deux échelons successifs dans le sens indiqué
`"2x1,-1"`	Répéter deux fois le couple hausse, baisse
`8.00`	Aller vers le prix de 8 €
`"20@12"`	Achat manuel de 20 titres à 12 €
`"-20@12"`	Vente manuelle de 20 titres à 12 €
`mark`	Poser une marque après l'opération précédente

Action	Interprétation

`8` signifie huit échelons ; `8.00` signifie un prix. Un mouvement vers un prix peut traverser plusieurs ordres. Une opération manuelle termine normalement le parcours ; l'opération qui suit une réinitialisation `merge` bénéficie d'un traitement particulier, expliqué au chapitre 16.

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" "2x1,-1"
```

Ces deux allers-retours finissent avec 100 titres et 920 € d'investissement : 80 € de gain brut au cours final de 10 €.

Simuler un regroupement avec reset ou merge

Ces simulations reproduisent le cas de 1 790 anciens titres regroupés en 179 nouveaux. Elles utilisent une définition littérale : aucun barème permanent ni trade n'est à créer. Le prix ancien de 0,80 € décrit l'opération de départ ; le prix de reprise de 12 € est déjà exprimé après regroupement.

Pour **reset** :

```
jedidemo tools play -c co0,6-14,17 -- \
  "scale=vr:10%,ks:0.8,1.2,ki:0%,kb:0,q0=1790,c0=0.8,tick=0.01" \
  "split=1:10,reinit=reset,12" 1 -1
```

La section `c` affiche le paramétrage et la section `o` les opérations :

Étape	Stock	Prix	Référence du nouveau paramétrage
Après split + reset, avant transaction	179	Reprise à 12 €	`q0=179`, `c0=12`, position `0:0:0`
Première hausse : vente de 36 titres nouveaux	143	13,20 €	Même référence
Baisse suivante : achat de 36 titres nouveaux	179	11,87 €	Même référence

Le prix de 11,87 € reprend l'arrondi effectivement produit par Jedi avec le pas de 0,01 €. Le split n'est pas réappliqué après la première transaction.

Pour **merge**, reprendre l'état de départ dans une autre simulation :

```
jedidemo tools play -c co0,6-14,17 -- \
  "scale=vr:10%,ks:0.8,1.2,ki:0%,kb:0,q0=1790,c0=0.8,tick=0.01" \
  "split=1:10,reinit=merge,12" "21@10.00" 1
```

Le paramétrage attend l'achat manuel. Après `21@10.00`, stock et `q0` valent 200 et `c0` vaut 11,79 €. La hausse suivante vend 40 titres à 11 €, laissant 160 titres. Le type II repart du prix réel de cette transaction manuelle pour le prochain mouvement.

La quantité `21` et le prix `10.00` décrivent les **nouvelles actions**, après le regroupement. Une vente manuelle s'écrit avec une quantité négative, par exemple `"-20@14.00"`. Jedi contrôle la vente par rapport au stock converti : `"-180@12.00"` serait refusé avec seulement 179 titres disponibles.

Les deux options appartiennent au même changement de paramétrage. `sync` ne peut accompagner cette combinaison. Après `merge`, fournir une transaction manuelle avant un déplacement automatique ou un prix cible ; sinon Jedi refuse le scénario. Dans le suivi réel, enregistrer uniquement la transaction effectivement exécutée.

Choisir les tableaux affichés

Lettre de `--command`	Lecture
`o`	Opérations et évolution du stock
`c`	Paramétrages et références
`r`	Ordres disponibles avant les opérations
`f`	Bilan financier brut

On peut combiner les lettres, par exemple `--command=o0,6-14,17crf`. Le sélecteur `-c o0,6-14,17` conserve onze colonnes du tableau des opérations : identifiant, type, mode, quantité, prix, position, achats, ventes, résultat brut, investissement du trade et stock détenu. Le résultat brut valorise le stock au prix de chaque opération ; il ne se limite pas aux gains encaissés.

Pour afficher aussi les frais, ajouter la colonne `16` : `-c o0,6-14,16-17`. Pour joindre le bilan financier aux opérations sélectionnées, ajouter `f` : `-c o0,6-14,17f`. Avec `operation list`, saisir seulement les indices, sans la lettre `o` : `-c 0,6-14,17`.

Le tableau `r` montre les ordres **avant** chaque opération ; une flèche repère l'ordre suivi. Il ne faut pas prendre sa dernière ligne pour un état d'ordres recalculé après la toute dernière opération.

Comparer les flux entre deux marques

Une marque découpe la lecture financière sans modifier le barème ni les ordres. Une seule marque est admise. Pour le bilan financier, la placer après une opération suivant l'ouverture : l'opération de base ne peut pas servir de marque.

```
jedidemo tools play -c o0,6-14,17f -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1 mark -1 1
```

L'aller-retour haussier produit 40 € de gain brut. Le second, baissier, produit encore 40 €. Le total vaut 80 € brut, **6,30 € de frais** et **73,70 € après frais** avec notre configuration à 0,35 %, sans minimum. Le tableau financier sépare les périodes avant et après la marque ; les frais se lisent dans les opérations si la colonne `16` est ajoutée au sélecteur.

Les changements de barème et les réinitialisations s'introduisent dans le parcours comme au chapitre 16. La définition peut aussi contenir `money` : vérifier alors `simulation.capital` et le mode `loss`, comme au chapitre 18.

Partir d'une opération enregistrée

Au lieu d'une définition directe, `play` peut recevoir l'identifiant d'une opération réelle. Il réutilise son contexte pour examiner la suite sans modifier les opérations enregistrées :

```
jedidemo tools play -c o0,6-14,17 -- 3 1 -1
```

Ici, **3 est un exemple d'identifiant**, à remplacer par celui d'une opération existant dans la base utilisée. Ce n'est pas un nom de trade ni une quantité.

Conserver la commande, la configuration et les résultats quand on compare deux réglages. `play` suit les mouvements que l'utilisateur lui fournit ; il ne mesure pas leur fréquence historique. La partie III expliquera comment `backtest` suit un fichier de cotations et quelles hypothèses il ajoute.

20. Suivre un exemple complet de trade

Rassemblons les calculs dans un parcours : dimensionner, observer les flux, changer un paramétrage, comparer deux suites et fermer le stock. Ce titre fictif est distinct d'Exa_01 dans la partie I. Tous les parcours ci-dessous sont exécutés en mémoire avec `tools play` ; ils ne créent aucun trade.

L'exemple utilise un type I, un cours initial de 10 €, un pas de 0,01 € et des frais de 0,35 %, sans minimum. Les commandes de `candidate` se reproduisent dans une base d'essai sans trade ouvert.

Dimensionner le projet

Par défaut, `candidate` contrôle trois échelons haussiers pour l'opérabilité. L'aide affiche la valeur configurée, soit 3 avec la configuration d'origine :

```
jedidemo tools candidate --help
```

La ligne `--operability-steps=COUNT` indique « défaut : 3 ». Si la configuration de cet espace d'essai change, la valeur affichée suit ce réglage.

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:1,n:2 --capital=20000
```

Avec 20 000 € de capital et 2,5 % de risque, `R` vaut 500 €. Deux échelons de baisse mènent à 6 €, soit 40 % sous l'entrée. La quantité retenue est **125**, pour une ouverture de **1 250 €**.

Le besoin maximal projeté hors frais vaut **1 855,60 €**, dont 605,60 € de réserve future. Ce montant porte sur trois échelons, selon l'horizon indiqué par `candidate`. Le chapitre 18 explique pourquoi cet horizon diffère de `n`.

Les parcours suivants **n'activent pas `money`** : ils isolent la mécanique du barème. Un trade avec protections peut suivre d'autres ordres. On ne déduit donc pas de ces parcours que le trade protégé exécuterait les mêmes achats et ventes.

Suivre les opérations

```
jedidemo tools play -c o0,6-14,17f -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -1 1 mark 1 1
```

Opération	Prix	Stock après opération	Investissement net hors frais
Achat initial de 125	10,00 €	125	1 250,00 €
Achat de 25	8,00 €	150	1 450,00 €
Vente de 25	10,00 €	125	1 200,00 €
Vente de 25	12,00 €	100	900,00 €
Vente de 2	14,00 €	98	872,00 €

La marque suit le retour à 10 €. Ce premier aller-retour a produit 50 € brut. Au cours final de 14 €, les 98 titres valent 1 372 € : le gain brut global est **500 €**. Il comprend la valeur du stock restant ; tout ce gain n'a pas été encaissé.

L'investissement maximal effectivement rencontré est ici 1 450 €. La réserve de `candidate` examinait un autre trajet, baissier : elle ne prédit pas les achats de chaque parcours.

Reprendre le même barème dans un nouveau paramétrage

Réappliquons le barème après la vente à 14 €, puis avançons d'un échelon :

```
jedidemo tools play -c o0,6-14,17cf -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -1 1 mark 1 1 \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 1
```

Le nouveau paramétrage conserve `q0=125` et `c0=10`, remet les compteurs locaux à zéro et reporte les coefficients cumulés : `kcpc=1,4` et `kspc≈0,785714`. Changer le paramétrage ne déclenche aucune transaction.

La hausse suivante vend **19 titres à 16,80 €**. Le stock devient 79, l'investissement net 552,80 € et la valeur du stock 1 327,20 €. Le gain brut vaut **774,40 €**. Les nouvelles coordonnées se lisent dans le nouveau paramétrage ; les anciennes opérations conservent les leurs.

Consigner une décision exceptionnelle

Pour un trade suivi réellement dans la base, une note peut expliquer une décision ou un contrôle qui ne se lit pas dans les opérations. Pour cet exemple, l'écriture ci-dessous est facultative et doit rester dans la base d'essai :

```
$ jedidemo note create 2026-10-05 "PEDA20 - réglage" "État vérifié :
> 98 titres détenus après la vente de 2 titres à 14 EUR.
> Même barème réappliqué pour vérifier la continuité.
> Ordres du courtier à contrôler avant de poursuivre."
```

`$` et `>` représentent les invites du shell et ne se saisissent pas. Le contenu comporte de vrais retours à la ligne. La note appartient au journal général ; `PEDA20` dans son résumé sert de repère et n'est pas un trade créé par `play`.

```
jedidemo note list PEDA20
```

Si cette note reçoit l'identifiant 1 dans une base vide :

```
jedidemo note show 1
jedidemo note update 1 --edit
```

Sinon, utiliser l'identifiant affiché par `note list`. L'éditeur permet d'ajouter ensuite la confirmation du courtier, notamment l'annulation d'anciens ordres. Sa préparation est décrite au chapitre 08.

Comparer une baisse au lieu de la hausse

Repartons du même état après la vente à 14 € et le changement de paramétrage. Remplaçons seulement le dernier mouvement par une baisse :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -1 1 mark 1 1 \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 -1
```

Cette baisse achète **20 titres à 11,20 €**. Le stock devient 118 et l'investissement net 1 096 €.

Suite après le changement	Stock	Cours final	Gain brut
Une hausse	79	16,80 €	774,40 €
Une baisse	118	11,20 €	225,60 €

Ces deux branches montrent des conséquences possibles du même réglage. Elles ne donnent pas la probabilité de chacune.

Fermer le stock et rapprocher le résultat

Dans la branche haussière, vendons manuellement les 79 titres restants à 16,80 € :

```
jedidemo tools play -c o0,6-14,17f -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -1 1 mark 1 1 \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  1 "-79@16.80"
```

Cette vente ferme volontairement le stock ; elle ne correspond pas à la quantité cible de l'ordre ordinaire suivant.

Bilan de la branche fermée	Montant
Achats totaux	1 450,00 €
Ventes totales	2 224,40 €
Stock final	0 titre
Gain brut	774,40 €
Frais cumulés	12,88 €

Bilan de la branche fermée	Montant
Gain après frais	761,52 €

Le bilan financier répartit le gain brut en 50 € avant la marque et 724,40 € après. Dans un trade réel, enregistrer l'exécution du courtier selon le chapitre 09, avec ses frais effectivement facturés.

À chaque étape, rapprocher le stock, les flux monétaires, le paramétrage actif et les prochains ordres. Les notes expliquent les événements exceptionnels ; elles ne remplacent pas l'enregistrement des exécutions. La partie III confrontera cette mécanique à des cotations historiques, avec les limites propres à la simulation.

Étudier les résultats historiques

Après l'usage quotidien et l'explication de la mécanique, cette partie examine ce que montrent les simulations sur des cotations passées. Elle suppose connus le barème, les opérations et les principales questions de capital.

Une **recherche** compare des barèmes sur une période pour en choisir un. Un **backtest** simule un barème fixé sur des cotations historiques, avec des conventions d'exécution. Une **vérification sur une autre période** examine le choix sur des données qui n'ont pas servi à le régler. Le **suivi daté** conserve les décisions avant d'en observer les suites.

Ces étapes sont illustrées par des commandes Jedi, des dates explicites et des rapports commentés. Le chapitre 21 prépare la question, les titres, les fichiers et leurs contrôles. Le chapitre 22 explique `tools backtest`, ses hypothèses et son rapport compact. Le chapitre 23 suit une étude sur `Euronext_Growth`. Le chapitre 24 organise la comparaison, les limites d'interprétation et le journal des résultats.

La lecture distingue gain brut, frais, gain net, investissement maximal et valorisation des titres encore détenus. Les cotations journalières ne révèlent pas l'ordre exact des mouvements à l'intérieur d'une séance ; les règles de simulation doivent donc être connues avant de commenter le résultat.

À la fin de cette partie, le lecteur saura préparer un examen historique, conserver sa définition et ses commandes, comparer des scénarios et identifier ce qui demande une vérification supplémentaire. Un résultat passé favorable reste une observation sur des données et hypothèses précises.

21. Préparer une étude historique

Poser une question vérifiable

Une étude commence par une question précise : comment un barème donné se comporte-t-il sur un titre et une période ? Quelle activité entraîne-t-il ? Quel investissement maximal demande-t-il ? Comment résiste-t-il à une baisse ou à un gap ?

Écrire la question avant de lire les gains. Fixer les titres, les dates, les frais et le mode de dimensionnement. Une recherche avec `score` sert à choisir des barèmes sur ses données ; un backtest avec une définition fixée sert à examiner leur parcours. Changer un paramètre après lecture du résultat constitue un nouvel essai à conserver.

Distinguer les périodes

Le chapitre 23 utilise ce calendrier :

Étape	Données accessibles	Usage
Préparation	Historique depuis début 2021	Contrôles et profondeur historique
Présélection au 31 décembre 2024	Cotations jusqu'à cette date	`profile` sur les 252 dernières séances, ATR sur 20
Recherche au 31 décembre 2024	Cotations jusqu'à cette date	`score` sur les 750 dernières séances
Vérification	Cotations de 2025	`backtest` avec le barème déjà fixé

La fenêtre de 252 séances décrit le comportement récent du titre. Celle de 750 expose les barèmes à un parcours plus long. Les données de vérification ne doivent pas servir à régler le choix qu'elles sont censées examiner.

Cette séparation n'efface pas ce que l'auteur sait déjà. Une étude réalisée après 2025 reste rétrospective, même si ses commandes isolent correctement les dates. Pour un suivi prospectif, consigner le choix à sa date réelle avant de disposer des observations suivantes.

Choisir les titres et conserver leur liste

Préciser le périmètre : marché, dataset, groupe ou liste explicite. Pour une étude arrêtée fin 2024, rechercher la liste des titres présents à cette date. Une liste actuelle peut omettre des sociétés sorties du marché et ajouter des introductions ultérieures.

Conserver cette liste avec les fichiers. Signaler les exclusions, les historiques trop courts et les titres disparus. Ne pas supprimer discrètement un titre parce que son résultat est défavorable. Un dataset est un ensemble d'appartenances dans une base ; son nom ne certifie ni l'origine des données ni une composition historique.

Dans une base existante, examiner le périmètre :

```
jedidemo market info demo --dataset=Demo
jedidemo market list demo --dataset=Demo --last-quote
```

Ces commandes supposent la base Demo du chapitre 10. Elles consultent la base sans télécharger de cotations.

Contrôler les fichiers préparés

Conserver le fournisseur, la date de récupération, le format d'origine, les conversions et les conditions d'utilisation ou de diffusion. Les fichiers individuels et les fichiers d'import de dataset ont des présentations différentes, décrites au chapitre 10.

Vérifier l'identité du titre, la devise, l'ordre des dates, les doublons, les volumes et la cohérence `Low ≤ Open, Close ≤ High`. Vérifier les séances absentes avec le calendrier du marché, plutôt qu'avec un simple comptage des jours ouvrés.

Identifier les divisions, regroupements et autres corrections de cours. Des cotations ajustées et non ajustées donnent des parcours différents. Ne pas mélanger une clôture ajustée avec des ouvertures et extrêmes non ajustés. Les cotations OHLCV ne constituent pas, à elles seules, un historique de dividendes ou de flux d'opérations sur titres pris en charge par la simulation.

Pour examiner un fichier individuel :

```
jedidemo tools profile cotations_EXA.txt
jedidemo tools stats cotations_EXA.txt --date=le:2025-12-31
jedidemo tools plot cotations_EXA.txt --date=le:2025-12-31
```

Le profil met en évidence les caractéristiques et les contrôles qu'il calcule ; `stats` et `plot` complètent l'examen. Une courbe régulière ou un import réussi ne suffisent pas à certifier l'exactitude du fournisseur.

Isoler réellement une année

Jedi accepte **un seul filtre `--date=OP:AAAA-MM-JJ` par commande** :

Code	Sens
`ge`	Date supérieure ou égale
`gt`	Date strictement supérieure
`le`	Date inférieure ou égale
`lt`	Date strictement inférieure

Le filtre de date est appliqué avant `--select`. Répéter `--date` ne construit pas une plage : la dernière valeur remplace la précédente. Pour isoler 2025 avec `--date=ge:2025-01-01`, utiliser un fichier ou une base **figés au 31 décembre 2025**, sans cotations ultérieures. Le fichier du chapitre 23 est livré ainsi.

Pour une recherche arrêtée à fin 2024, la borne supérieure suffit :

```
jedidemo tools score cotations_EXA.txt --date=le:2024-12-31 \
  --select=l750 --sort=fitness --budget=40
```

Ce fichier ne contient pas 750 séances avant cette date ; le rapport annonce la période réellement disponible. Le fichier Growth du chapitre 23 fournit la profondeur nécessaire. Toujours lire les dates et le nombre de séances affichés.

`--select=l252` conserve les 252 dernières lignes après filtrage ; `f100` les 100 premières ; `1:100` une plage d'indices à partir de 1. Une sélection discontinue, par exemple `f10,-12:-1`, joint des morceaux de parcours : elle peut créer un saut artificiel entre deux dates. La réserver à l'inspection, en connaissant cet effet.

La fraîcheur courante de `profile` n'est pas évaluée lors d'une sélection historique avec `--date` ou `--select`. Cela évite de comparer des cotations arrêtées en 2024 à une référence de dataset de 2025 ou 2026. Contrôler soi-même que chaque titre atteint bien la borne historique voulue.

Figer une copie d'étude

Une base maintenue avec `market update` évolue. Préparer une base de démonstration séparée pour l'étude et conserver les fichiers importés. Le lanceur `jedidemo` et l'option de dossier `JEDI_DEMO_DIR` permettent de séparer les essais ; les étapes figurent au chapitre 02.

Conserver aussi la version et la configuration effective :

```
study_dir="$HOME/jedi-study-2025"
mkdir -p "$study_dir"
jedidemo version > "$study_dir/version.txt"
jedidemo config > "$study_dir/config.txt"
jedidemo config --paths > "$study_dir/paths.txt"
```

Le dossier reçoit les sorties du terminal, indépendamment du dossier dans lequel le lanceur exécute Jedi. Avant de partager une configuration, retirer les clés de fournisseurs qu'elle peut contenir.

La fiche d'étude associe enfin la question, la liste de titres, les fichiers, les périodes, le barème, les frais et les commandes. Le chapitre 24 en propose un modèle.

22. Simuler un barème avec tools backtest

Un barème fixé sur un historique

`backtest` parcourt les cotations d'un titre et simule les opérations proposées par un barème fixé. Il utilise une base de simulation en mémoire : il n'ajoute aucun trade à votre portefeuille et ne met pas à jour les cotations de marché.

La première ouverture utilise le prix **Open** de la première séance sélectionnée où le dimensionnement permet l'achat. Après une liquidation complète, une nouvelle ouverture est tentée à la séance suivante. Le rapport agrège donc éventuellement plusieurs trades successifs sur le même titre.

Depuis le fichier fictif livré :

```
jedidemo tools backtest cotations_EXA.txt \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2"
```

Depuis la base Demo préparée au chapitre 10 :

```
jedidemo tools backtest --database=demo --security=EXA \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2"
```

`--scale` accepte une définition ou le nom d'un barème déjà enregistré dans cet espace. Le préfixe `I:` ou `II:` montré par `score` est retiré avant transmission. Ici, le constructeur donne un barème de type I dont le rapport affiche la définition opérationnelle.

Choisir les dates

Sans sélection explicite, `backtest` utilise **tout l'historique disponible**. Il ne reprend pas automatiquement la fenêtre de 750 séances de `score`.

```
jedidemo tools backtest cotations_EXA.txt --select=l750 \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2"
```

`--date` et `--select` suivent les règles du chapitre 21. Le début sélectionné change le prix d'ouverture et le point de départ de la mécanique. Un test commencé le 1er janvier avec un nouveau trade ne prolonge pas automatiquement un trade commencé l'année précédente.

Deux modes de dimensionnement

Sans `--capital`, chaque ouverture demande un **montant fixe**, calculé avec la configuration :

```
montant = partie entière de
          simulation.capital × simulation.risk_rate / simulation.distance
```

Avec les réglages d'origine, `20000 × 0.015 / 0.2 = 1500 €`. La quantité entière réellement achetée peut laisser une petite fraction de ce montant inutilisée. L'option `--distance` modifie ce calcul ; elle ne représente pas le nombre d'échelons `n` de `money`.

Ajouter `--money` sans `--capital` ajoute les protections du trade, mais conserve ce dimensionnement par montant fixe.

Avec **`--capital`**, Jedi demande aussi **`--money`** et calcule la quantité à chaque ouverture selon le risque, le plafond de capital et les exigences d'opérabilité :

```
jedidemo tools backtest cotations_EXA.txt \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2" \
  --capital=20000 --money="r:1.5%,o:1,n:2"
```

Le chapitre 18 explique `r`, `o` et `n`. Par défaut, ce mode exige au moins **deux titres par ordre contrôlé** et examine **trois échelons haussiers**. `--minimum-quantity` et `--operability-steps` modifient ces contrôles ; ces options exigent le mode `--capital`.

Une ouverture non opérable est reportée à la séance suivante et comptée dans le rapport. `--distance` ne se combine pas avec `--capital`. Le capital fourni reste la référence de dimensionnement : le backtest ne simule pas le financement simultané d'un portefeuille de plusieurs titres et ne réinvestit pas automatiquement les gains dans un capital augmenté. `candidate` reste nécessaire pour examiner le financement d'un projet réel avec les réserves de votre portefeuille.

Connaître les hypothèses d'exécution

Le fonctionnement courant utilise **`--execution=conservative`** et **`--opening-day-orders=next`**. Les ordres issus de l'achat initial deviennent actifs à la séance suivante. Le moteur retient **au plus une opération proposée par séance**, en plus de l'ouverture lorsqu'elle a lieu ; il ne déroule pas tous les échelons qu'une journée pourrait traverser.

Avec des données journalières, on connaît Open, High, Low et Close, mais pas l'ordre des mouvements entre l'ouverture et la clôture. Jedi applique donc des règles explicites :

1. Un ordre déjà exécutable au prix d'ouverture est prioritaire sur un ordre seulement touché plus tard dans la séance.
2. Si plusieurs ordres restent possibles sans chronologie connue, la politique choisit selon leur nature. Elle donne généralement priorité à la protection, puis à l'achat à la baisse, puis à la vente à la hausse. Pour une protection de perte encore non franchie à l'ouverture, l'achat à la baisse peut précéder cette protection ; une protection des gains est prioritaire sur cet achat.
3. Le prix d'ouverture est appliqué lorsqu'un gap rend l'ordre immédiatement exécutable ; sinon, le niveau de l'ordre est utilisé.

Ordre actif	Niveau	Open	Prix simulé
Achat Limit	8 €	7 €	7 €
Vente Limit	12 €	13 €	13 €
Vente StopLoss	6 €	4 €	4 €
Achat Stop	12 €	13 €	13 €

Un gap favorable peut améliorer un prix Limit ; un gap défavorable peut dépasser le niveau Stop et le risque prévu. Le nom `conservative` désigne cette convention, sans garantir le pire parcours intrajournalier possible.

La simulation ne reconstitue ni le carnet d'ordres, ni les exécutions partielles, ni le spread. Le volume quotidien ne sert pas à limiter automatiquement la quantité exécutée. Les frais viennent de la configuration ; les prix simulés ne remplacent pas l'application réelle des pas de cotation et des conditions du courtier.

Les variantes `--execution=legacy` et `--opening-day-orders=same` sont conservées pour des examens particuliers. `legacy` change les priorités et utilise les niveaux théoriques plutôt que les prix de gap. `same` autorise les ordres dès la journée d'ouverture. Les indiquer explicitement dans toute comparaison qui les utilise.

Lire d'abord le rapport compact

La première commande de ce chapitre, sur les 851 séances d'EXA, produit :

```
Historique : 2023-06-30 → 2026-10-02 (851 séances)
Montant demandé par ouverture : 1500.00 €
Ouvertures : 1 ; tentatives non opérables : 0
Premier achat : 2023-06-30, 150 titres pour 1500.00 €
Opérations, ouvertures comprises : 25
Gain brut cumulé : 647.11 €
Frais cumulés : 19.14 €
Gain net cumulé : 627.97 €
Investissement maximal, hors frais : 1500.00 €
Gain net / investissement maximal : 41.86 %
Titres détenus en fin de test : 122 ; valeur 1578.97 €
```

Le gain brut additionne les ventes, soustrait les achats et ajoute la valeur des titres restants à la dernière clôture. Le gain net retire les frais. L'investissement maximal est le plus grand des pics d'achats moins ventes de chaque trade, hors frais. Les trades du backtest se succèdent sans se chevaucher : ce maximum global doit donc apparaître dans au moins un des trades.

Les 41,86 % couvrent toute la période ; ce n'est pas un taux annuel. La valeur finale de 1 578,97 € est une composante du gain, pas un gain à ajouter aux 627,97 €. Les 122 titres ne sont pas vendus artificiellement à la fin : le rapport ne facture pas une liquidation finale qui n'a pas eu lieu.

Comprendre le maximum quand un trade est rouvert

Un autre exemple fictif illustre le calcul : le premier trade achète 100 titres à 10 €, puis les vend tous à 8,50 €. Sa perte brute est de 150 €. Le suivant achète de nouveau 100 titres à 10 €.

Trade	Pic d'investissement hors frais	Situation
Trade_01	1 000 €	Clos, perte brute de 150 €.
Trade_02	1 000 €	Ouvert après la clôture du précédent.
Maximum du backtest	**1 000 €**	Le plus grand des deux pics.

Le solde cumulé achats moins ventes vaut alors 1 150 € : les 150 € perdus restent dans le bilan financier. Ce solde sert à calculer le gain cumulé ; il ne constitue pas le maximum individuel d'un trade. Dans les rapports détaillés et techniques, son libellé est **solde achats-ventes**. Il peut dépasser le maximum, notamment après une perte passée.

Le tableau financier demandé par `-c f` suit désormais **l'ordre de création des trades**, du premier au dernier. La synthèse compacte `r`, la synthèse technique et le total de `f` utilisent le même maximum. Les rendements et les ratios fondés sur ce maximum sont recalculés avec cette valeur.

Examiner les opérations et les gaps

```
jedidemo tools backtest cotations_EXA.txt \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2" -c rfo
```

`r` conserve le rapport, `f` ajoute les résultats financiers et `o` les opérations. `t` liste les trades, `d` les ordres encore actifs et `p` le graphique. Ces sorties détaillées peuvent être larges : les examiner au terminal ou les conserver dans un fichier. Les sélecteurs de colonnes de `play` ne sont pas des options de `backtest`.

Le petit fichier `cotations_GAP.txt` permet de vérifier une perte dépassant le risque prévu :

```
jedidemo tools backtest cotations_GAP.txt \
  --scale="kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1" \
  --capital=20000 --money="r:2.5%,o:1,n:2" -c ro
```

Jedi achète 125 titres à 10 €, puis exécute la protection à l'ouverture suivante à 4 €, alors que son niveau était à 6 €. Le risque nominal de 500 € devient une perte brute de 750 €, et une perte nette de 756,13 € avec les frais. Ce parcours fictif de deux séances explique une règle ; il ne constitue pas une étude de rendement.

Comparer des scénarios fixés

Changer seulement le barème, sur les mêmes données et avec les mêmes options, permet une comparaison lisible. Par exemple :

```
jedidemo tools backtest cotations_EXA.txt \
  --scale="vr:5%,np:2,nn:2,ki:0%,kb:1,kd:0.0"
```

Conserver aussi un scénario perdant et les hypothèses qui l'expliquent. L'affichage `--display=technical` ajoute les diagnostics détaillés de compatibilité ; le rapport compact fournit la première lecture économique. Le chapitre 24 organise l'interprétation commune aux scénarios.

23. Exemple d'étude sur Euronext Growth

Les captures chiffrées de ce chapitre sont celles de l'étude de référence de l'édition 1.0. Avec le calcul du maximum d'investissement corrigé en v1.1, relancer les commandes pour obtenir les valeurs d'investissement, les rendements et les classements actualisés.

Un parcours à reproduire, puis à adapter

Ce chapitre montre comment organiser une étude de titres d'Euronext Growth : présélection à fin 2024, recherche sur les données antérieures, puis vérification en 2025. Les fichiers livrés contiennent **trois titres fictifs** EXA, EXB et EXC ; ils ne sont pas un export d'Euronext Growth. Le nom du dataset sert à montrer l'organisation de l'étude.

Les sorties sont calculées sur ces données de démonstration. Elles n'indiquent ni le rendement du marché Growth, ni celui de titres réels. Cet exemple rédigé après 2025 est **rétrospectif** ; les dates des commandes encadrent les calculs sans transformer l'exercice en décision réellement prise en 2024.

Préparer une base historique figée

Après l'installation des lanceurs au chapitre 02, copier les fichiers TXT de `examples/` dans `~/jedi-examples/`. Utiliser une base principale d'essai sans trade engagé et les réglages suivants dans sa configuration :

```
fees:
  rate: 0.0035
  min: 0.0
simulation:
  capital: 20000
  risk_rate: 0.015
  distance: 0.2
```

`Growth_demo_2021_2025.txt` contient 3 909 lignes de cotations : 1 303 jours de semaine par titre, du 4 janvier 2021 au 31 décembre 2025. `labels_growth_demo.txt` fournit les identités. La composition reste fixe ; le calendrier simplifié inclut des jours fériés.

Créer une base distincte de Demo, une seule fois :

```
jedidemo market init growthdemo
jedidemo market import growthdemo Euronext_Growth \
  Growth_demo_2021_2025.txt labels_growth_demo.txt
jedidemo market info growthdemo --dataset=Euronext_Growth
jedidemo market list growthdemo --dataset=Euronext_Growth --last-quote
```

Vérifier trois titres, 3 909 cotations et une dernière cotation au 31 décembre 2025 pour chacun. Ne pas prolonger cette base par un update de 2026 : elle constitue l'entrée figée de cette étude.

Présélectionner avec les données de fin 2024

```
jedidemo tools profile --database=growthdemo --dataset=Euronext_Growth \
  --date=le:2024-12-31 --rank-criteria=4 --include-rejected
```

Le calcul dispose de quatre années de profondeur et utilise les 252 dernières séances jusqu'à cette date pour ses critères récents. Il donne :

Titre	Total	Tendance/an	Liquidité médiane	ATR20	Sw5/an	Statut
EXA	6	10,2 %	1,2 M€	3,5 %	12,5	retenu
EXC	6	1,8 %	737 k€	3,5 %	20,8	retenu
EXB	—	−19,9 %	741 k€	3,4 %	9,4	écarté

Les quatre premiers rangs donnent le total ; les trois suivants restent informatifs. EXB est écarté pour sa tendance. EXA et EXC ont un total identique : EXA apparaît d'abord par le départage des rangs dans leur ordre d'importance. EXB n'entre pas dans ces rangs.

La fraîcheur courante n'est pas évaluée car l'historique est filtré. Vérifier que les trois historiques vont bien jusqu'à fin 2024 ; la date de fraîcheur de fin 2025 affichée par `market info` ne sert pas ici à mesurer un retard.

Pour la suite de l'exercice, choisir EXA. Dans une étude réelle, consigner également pourquoi ce titre a été retenu plutôt qu'EXC.

Rechercher un barème avant 2025

```
jedidemo tools score --database=growthdemo --security=EXA \
  --date=le:2024-12-31 --select=l750 --sort=fitness --budget=40
```

La période utilisée est **2022-02-16 → 2024-12-31**, soit 750 séances. La recherche adaptative évalue 40 barèmes parmi 11 793 candidats disponibles. Le premier résultat compact est :

```
Fitness  Op/mois  Inv.max  Constructeur typé
  0.437    0.924    1.000  I: kc:1.5,1.38,1.23,1.14,0.52,0.4,
                             ki:0%, kb:1, kd:0.2
```

La définition est coupée sur deux lignes ici pour la lecture ; dans la commande, la transmettre comme une seule valeur. La fitness brute de 0,437 concerne la période de recherche, sans déduire les frais. Elle ne prédit pas la fitness de 2025. Le budget limité signifie que la première ligne est la meilleure selon le tri parmi les barèmes effectivement évalués.

Fixer pour l'exercice cette définition, les frais de 0,35 %, le montant de 1 500 € par ouverture et les conventions d'exécution par défaut. Un choix réel demande aussi une lecture de la mécanique et un contrôle d'opérabilité au prix de l'ouverture envisagée.

Vérifier le choix sur 2025

```
jedidemo tools backtest --database=growthdemo --security=EXA \
  --date=ge:2025-01-01 \
  --scale="kc:1.5,1.38,1.23,1.14,0.52,0.4,ki:0%,kb:1,kd:0.2"
```

La base s'arrête au 31 décembre 2025 : le filtre inférieur isole donc cette année. Le test démarre avec un nouveau trade ; il ne transporte pas les positions du parcours de recherche.

```
Historique : 2025-01-01 → 2025-12-31 (261 séances)
Premier achat : 2025-01-01, 108 titres pour 1498.56 €
Opérations, ouvertures comprises : 7
Gain brut cumulé : -13.11 €
Frais cumulés : 7.10 €
Gain net cumulé : -20.21 €
Investissement maximal, hors frais : 1498.56 €
Gain net / investissement maximal : -1.35 %
Titres détenus en fin de test : 108 ; valeur 1450.47 €
```

Le parcours de recherche favorable est suivi d'un résultat net légèrement négatif. Les ventes et rachats génèrent des allers-retours, mais la valeur finale du stock contribue aussi au résultat. La commande avec `-c rfo` permet d'examiner les sept opérations.

Le début au 1er janvier reflète le calendrier fictif, pas une séance réelle du marché. Le résultat décrit ce fichier, ce barème et ces conventions. Il doit être conservé tel quel avant d'essayer d'autres réglages.

Conserver les commandes et les résultats

```
study_dir="$HOME/jedi-study-growth-2025"
mkdir -p "$study_dir"
jedidemo version > "$study_dir/version.txt"
jedidemo config > "$study_dir/config.txt"
jedidemo tools score --database=growthdemo --security=EXA \
  --date=le:2024-12-31 --select=l750 --sort=fitness --budget=40 \
  > "$study_dir/score-before-2025.txt"
jedidemo tools backtest --database=growthdemo --security=EXA \
  --date=ge:2025-01-01 \
  --scale="kc:1.5,1.38,1.23,1.14,0.52,0.4,ki:0%,kb:1,kd:0.2" \
  > "$study_dir/backtest-2025.txt"
```

Ces redirections conservent les rapports standard ; l'avancement de `score` peut rester affiché au terminal. Joindre les deux fichiers source, la liste des titres, la question et la définition choisie. Le chapitre 24 explique le journal daté ; une note rédigée aujourd'hui doit porter sa date réelle.

Passer aux données réelles

Préparer les titres effectivement présents sur Euronext Growth à la date retenue, avec leurs historiques et les opérations sur titres pertinentes. Décrire les exclusions et la disponibilité des données des sociétés sorties du marché. Vérifier la devise et l'unité des volumes.

Importer ces fichiers dans **une autre base d'étude**, avec un nom distinct et une borne finale explicite. Adapter les noms de source et de titres dans les commandes. Conserver séparément la présélection, la recherche et la vérification.

Si plusieurs titres sont simulés séparément, leurs investissements maximaux et leurs gains ne décrivent pas automatiquement un portefeuille finançable : les achats peuvent être simultanés. Examiner ensuite les projets d'ouverture et les réserves avec `candidate`, au prix et à la date de la décision réelle.

24. Interpréter, comparer et suivre les résultats

Expliquer les euros avant de classer

Un résultat se lit avec ses dates, son barème, son premier achat et ses conventions. Les mesures compactes répondent à des questions différentes :

Mesure	Question
Gain brut	Quel est le solde ventes − achats + valeur finale des titres ?
Frais	Quel coût de courtage a été imputé aux opérations ?
Gain net	Que reste-t-il après ces frais ?
Investissement maximal	Quel est le plus grand pic des achats moins ventes d'un des trades successifs, hors frais ?
Gain net / investissement maximal	Que représente le résultat net par rapport au plus grand montant engagé dans un trade ?
Activité	Combien d'opérations et d'interventions le parcours entraîne-t-il ?
Stock final	Quelle part du résultat dépend encore de titres détenus ?

Les ventes ne sont pas toutes du bénéfice, et un titre encore détenu n'est pas une somme encaissée. Le rapport valorise le stock restant à la dernière clôture ; il n'ajoute pas une vente finale fictive.

La fitness de `score` divise le gain **brut** par le premier achat. Le rendement compact de `backtest` divise le gain **net** par l'investissement maximal. Comparer les deux pourcentages sans rapprocher leurs numérateurs, leurs dénominateurs et leurs périodes donne une lecture trompeuse. L'annexe 04 rassemble ces définitions.

Comparer sur une base commune

Garder identiques les titres, les cotations, les dates, les frais, le dimensionnement et les règles d'exécution. Fixer les barèmes avant d'observer la période de comparaison. Relever pour chacun le net, les frais, le pic d'investissement, les opérations et le stock final.

Une simulation plus active peut gagner davantage brut et moins net. Une autre peut sembler peu consommatrice de capital parce que des ventes ont réduit l'investissement ; elle peut encore porter un stock important. Un gain élevé sur une hausse continue ne démontre pas à lui seul l'intérêt des allers-retours.

Une comparaison simple à deux barèmes suffit souvent. Les options `--sort=activity` et `--sort=investment` de `score` aident à explorer les compromis ; en recherche adaptative, elles changent aussi le raffinement. Pour comparer les mêmes barèmes, reprendre leurs définitions dans `backtest`.

Lire les limites du parcours

Les données journalières ne donnent pas la chronologie de chaque mouvement. Un seul ordre proposé est retenu par séance. Les gaps, le coût réel d'exécution, les exécutions partielles, les interruptions de cotation et les opérations sur titres peuvent produire un parcours réel différent.

Le chapitre 22 montre comment un gap dépasse un risque nominal. Un StopLoss déclenche une action selon les possibilités du marché ; il ne garantit pas un prix ni un plafond de perte. Le résultat de plusieurs backtests individuels ne remplace pas une vérification du financement commun des trades.

L'annualisation disponible dans certains affichages experts transforme un ratio selon une durée. Elle n'ajoute pas d'observations et n'atteste pas qu'un rendement soit reproductible. Pour une courte période, les euros, les opérations et le stock final restent les premières informations à lire.

Vérifier sur d'autres dates

Conserver le barème choisi et examiner une autre période, notamment une phase différente. Indiquer si chaque test repart d'un nouvel achat ou si l'on suit un trade réel déjà engagé. Des dates de départ différentes donnent des prix de référence et des quantités entières différents.

Pour une nouvelle période, préparer sa borne finale selon le chapitre 21 et conserver le choix avant de lire son résultat. Si l'on ajuste ensuite le barème, garder le premier rapport et créer une nouvelle ligne d'étude. Le nombre d'essais compte dans l'interprétation : chercher longtemps une configuration favorable rend le meilleur résultat moins surprenant.

Tenir un journal daté

Le journal conserve ce qui a été décidé, pourquoi, avec quelles données, puis ce qui a été observé. `note` convient aux choix d'étude et aux événements exceptionnels d'un trade : gap, suspension, incident de courtier, exécution partielle, changement de paramétrage ou opération sur titres.

L'éditeur facilite une note complète :

```
jedidemo note create --edit
```

Ou saisir un texte multiligne :

```
jedidemo note create 2026-10-07 "Étude Growth 2025 : protocole" "Étude rétrospective sur données fictives.
Choix calculé jusqu'au 2024-12-31 ; vérification sur 2025.
Barème fixé : kc:1.5,1.38,1.23,1.14,0.52,0.4,ki:0%,kb:1,kd:0.2.
Net 2025 : -20.21 EUR ; investissement maximal : 1498.56 EUR."
```

La date est celle de la rédaction de cette note ; elle ne prétend pas qu'une décision ait été prise en 2024. Pour votre propre journal, utiliser la date réelle. Ne pas mélanger les champs positionnels avec `--date`, `--summary` ou `--contents`.

Une note décrit l'événement ; elle ne modifie ni les opérations ni les ordres. Enregistrer séparément chaque exécution réelle avec `operation`, puis vérifier l'état du trade. Conserver les fichiers d'étude hors de la note et y indiquer leur emplacement.

Fiche de reproduction

Champ	À conserver
Question et date	Objectif, date réelle du choix, caractère prospectif ou rétrospectif
Logiciel	Sortie de `version`, langue et configuration utile
Données	Fournisseur, récupération, fichiers, conversions, ajustements, droits d'usage
Population	Liste datée des titres, exclusions et raisons
Périodes	Borne de choix, fenêtre de recherche, début et fin de vérification
Paramètres	Définition exacte du barème, montant ou capital et `money`, frais
Exécution	`conservative` ou `legacy`, `next` ou `same`
Commandes	Commandes complètes et rapports conservés
Résultats	Brut, frais, net, investissement maximal, opérations et stock final
Suites	Décision pratique, changements ultérieurs et événements exceptionnels

Cette fiche permet de relire une étude, d'expliquer une différence entre deux rapports et de distinguer le choix initial des modifications suivantes. Pour un projet réel, revenir à `candidate` avec le prix actuel, puis au suivi des opérations exécutées décrit dans la partie I.

Pour les opérations réelles, contrôler les frais contre les avis d'opéré. Une erreur peut être corrigée avec `operation fees ID MONTANT`, selon le chapitre 08, puis vérifier `trade results`. Cette correction change le bilan net du portefeuille enregistré ; les études `backtest` et `score` recréent leurs opérations et leurs frais à partir de leurs propres paramètres.

Annexe 1. Référence des commandes et options

Les valeurs intégrées et l'usage de chaque entrée sont détaillés dans l'annexe 05.

Trouver l'aide de la version installée

Lancer `jedi` sans argument affiche les familles de commandes. Une famille seule, par exemple `jedi tools`, affiche ses sous-commandes. L'aide détaillée s'obtient au niveau de la sous-commande :

```
jedidemo tools backtest --help
jedidemo market import --help
jedidemo note create --help
```

`version` et `config` se consultent directement : `jedi version`, `jedi config` et `jedi config --paths`. Ils n'ont pas de sous-commande d'aide. Le fichier livré jedi-help-fr.txt rassemble les aides de la version de référence ; l'aide de votre exécutable reste celle de la version effectivement utilisée.

Les noms peuvent être abrégés jusqu'à un préfixe non ambigu. Les exemples gardent les noms complets. Les champs nommés et les champs positionnels constituent des formes alternatives : ne pas les mélanger pour les mêmes champs. Entourer de guillemets les noms avec espaces et les expressions contenant des caractères interprétés par le shell.

L'option globale `--border=PRESET` précède la famille de commande. Les styles sont `ascii`, `reduce-dascii`, `modern`, `reducedmodern`, `fancy`, `markdown`, `empty`, `blank`. `--` termine la lecture des options : il est utile avant des arguments négatifs, notamment dans `play`.

Portefeuille et journal

Famille	Sous-commandes	Usage
`security`	`create`, `update`, `delete`, `show`, `list`	Identité, ticker, pas d'achat et de vente, taxe d'achat du titre
`quote`	`get`, `set`, `update`, `refresh`	Interrogation externe, saisie manuelle ou enregistrement de cours courants
`scale`	`create`, `update`, `delete`, `show`, `list`	Barèmes de type I ou II enregistrés
`trade`	`create`, `update`, `delete`, `show`, `list`	Identité du trade et gestion du capital
`trade`	`orders`, `status`, `results`, `plot`	Ordres, situation, résultats et graphique
`settings`	`create`, `delete`, `show`, `list`	Barème applicable à une date et une heure, split, synchronisation, réinitialisation
`operation`	`create`, `fees`, `delete`, `list`	Exécutions réellement réalisées ; correction des frais par identifiant d'opération
`note`	`create`, `update`, `delete`, `show`, `list`	Journal et événements exceptionnels ; `--edit` pour l'éditeur
`database`	`dump`, `load`, `schema`	Sauvegarde, restauration et schéma de la base principale

Les champs obligatoires sont décrits par chaque aide. La suppression d'une entité utilisée peut être refusée ; une suppression d'opération n'est pas son remplacement automatique. Les exemples de correction et d'adaptation figurent aux chapitres 08 et 09.

`quote get ISIN` affiche un cours externe sans enregistrer de titre ou de cotation. `quote set` enregistre un cours manuel. `quote update` actualise un titre enregistré ; `quote refresh` traite une sélection et, par défaut, les titres des trades en cours. `--mode=live|eod` s'applique à `update` et `refresh`, pas à `get`. `--force` appartient à `refresh` ; il permet de forcer la mise à jour selon son filtrage.

Corriger les frais d'une opération

```
jedidemo operation fees 42 3.50
jedidemo operation fees 42 0
jedidemo operation fees --help
```

La syntaxe est `jedi operation fees <operation_id> <amount>`. L'identifiant numérique est celui de l'opération, à lire avec `operation list`, et non celui du trade. Le montant remplace le **total** des frais, taxe éventuelle comprise ; un montant nul les annule. Utiliser un point décimal et une valeur positive ou nulle, arrondie au centime. Une opération ancienne ou d'un trade clôturé peut être corrigée. Les résultats nets, listes et exports reprennent la nouvelle valeur. Les frais historiques sont aussi repris lors d'un nouveau `play` depuis une opération du même trade. Le chapitre 08 donne la procédure complète.

Adapter un paramétrage : split, reinit et sync

Un nouveau paramétrage demande `--scale`, `--date` et `--time`. Il doit suivre les derniers paramétrages et opérations de toute la base ; le paramétrage précédent du trade doit déjà avoir été utilisé par une opération.

Option ou combinaison	Effet
`--split=NEW:OLD`	Convertir les unités ; `1:10` signifie une nouvelle action pour dix anciennes.
`--reinit=reset,PRICE`	Stock courant comme `q0`, prix choisi comme `c0`, position `0:0:0`.
`--reinit=merge,PRICE`	Référence en attente ; une transaction manuelle réelle définit ensuite `q0` et `c0`.
`--split` + `--reinit=reset,PRICE`	Accepté dans next-version / v1.1 ; reset sur le stock entier converti.
`--split` + `--reinit=merge,PRICE`	Accepté dans next-version / v1.1 ; merge sur le stock entier converti.
`--sync=POSITION`	Synchronisation réservée aux barèmes de type I.
`--sync` + `--split` ou `--reinit`	Refusé.

Avec split + reinit, `PRICE` et les valeurs de l'opération suivante sont exprimés **après le split**. Tant que merge attend son opération, Jedi ne propose aucun ordre automatique. Les droits fractionnaires ne sont pas comptabilisés. Le chapitre 09 fournit les cas pratiques et le chapitre 19 les simulations.

Bases de marché

Sous-commande	Arguments principaux	Usage
`market init`	`SOURCE`	Créer une base de marché
`market import`	`SOURCE DATASET QUOTES LABELS`	Importer un dataset complet
`market replace`	`SOURCE DATASET QUOTES LABELS`	Remplacer le contenu d'un dataset
`market update`	`SOURCE DATASET QUOTES LABELS`	Fusionner sans suppression ; ignorer les doublons et signaler les conflits
`market sync-group`	`SOURCE GROUP FILE`	Créer ou synchroniser les membres d'un groupe
`market group`	`SOURCE GROUP ISIN`	Ajouter ou retirer un membre avec `--add` ou `--remove`
`market info`	`SOURCE`	Métadonnées et contrôles ; `--dataset=CODE` facultatif

Sous-commande	Arguments principaux	Usage
`market list`	`SOURCE [REGEX]`	Liste ; `--dataset=CODE`, `--last-quote`
`market quote`	`SOURCE ISIN`	Historique ; `--date`, `--select`

`--universe=NAME` sélectionne l'univers, `france` dans la configuration d'origine. Les arguments QUOTES, LABELS et FILE sont des chemins de fichiers. Les formats et les effets des imports, remplacements et mises à jour sont expliqués au chapitre 10.

`import`, `replace` et `update` acceptent `--date-order=auto|ymd|dmy|mdy`, `--year-pivot=NN` et `--from=DATE`. Un update refuse une extension à plus de sept jours calendaires de la fin actuelle, sauf avec `--allow-gap`. Une cotation conflictuelle n'est pas écrasée par update ; reprendre un corpus corrigé avec `replace`. `sync-group` accepte `--as-of=DATE` et `--reference-source=NAME` pour décrire sa composition.

`market list` accepte des expressions regex par champ, dont `group` et `dataset`, avec && et ||. Cela ne fournit pas une option multi-groupes à `profile`. Jedi expose un seul `--group=CODE` dans `profile` ; aucune commande dédiée ne supprime un groupe. `market quote` affiche un tableau OHLCV et ne propose pas d'export CSV.

Options communes de consultation

Les filtres regex des listes qui les acceptent sont insensibles à la casse. `champ=motif` cible un champ ; && signifie ET, || signifie OU, avec priorité de ET. Les parenthèses ne sont pas prises en charge. La liste des champs varie selon la commande : consulter son aide.

`--running` limite certaines listes aux trades en cours. `--select` choisit des enregistrements après les filtres ; ses indices commencent à 1, les indices négatifs partent de la fin. `fN` et `lN` désignent les premières et dernières lignes ; `I:J` une plage ; la virgule combine les sélections.

Les listes qui proposent `--csv` peuvent ajouter les entêtes avec `--headers`. `--extended` n'a pas un effet universel : pour `operation list` en CSV, il ajoute notamment les noms associés. `--column-indexes` sélectionne des colonnes d'affichage et ne se combine pas avec CSV.

Pour une liste d'opérations lisible :

```
jedidemo operation list -c 0,6-14,17
```

Le `o` de `play -c o0,6-14,17` désigne une section ; il ne précède pas les indices dans `operation list`.

Outils historiques : source et sélection

`profile`, `score`, `backtest`, `plot` et `stats` acceptent un fichier individuel ou `--database=SOURCE --security=IDENT`, avec `--universe` facultatif. IDENT est un ISIN ou un ticker. `profile` ajoute les sélections collectives `--dataset`, `--group` et les `--security` répétés.

Un seul `--date=OP:YYYY-MM-DD` est accepté par commande, avec `ge`, `gt`, `le`, `lt`. Le filtre de date précède `--select`. Répéter `--date` remplace la valeur précédente. Pour isoler une année avec une borne inférieure, figer la source à sa borne supérieure, selon le chapitre 21.

tools profile

Option	Rôle
`--dataset=CODE`	Sélectionner un dataset
`--group=CODE`	Filtrer par un groupe
`--include-rejected`	Afficher les écartés sans les classer
`--rank-criteria=N`	Additionner les N premiers rangs, de 1 à 7 ; défaut 7
`--sort=KEY`	`global`, `ticker`, `trend`, `liquidity`, `volatility`, `oscillation`, `drawdown`, `gap`, `step`
`--limit=COUNT`	Limiter les lignes affichées

L'ordre des sept critères est tendance, liquidité, volatilité, oscillation, drawdown, gap, pas. Les quatre premiers guident la présélection ; le chapitre 04 explique les choix pratiques et les statuts. Les critères récents utilisent 252 séances, l'ATR 20 ; la profondeur et la fraîcheur sont des contrôles distincts.

tools score

Option	Valeurs et défaut
`--sort`	`fitness` par défaut ; `activity`, `investment` ; alias 1, 2, 3
`--ranking`	`raw` par défaut ; `jedi`, `both`
`--search`	`adaptive` par défaut ; `grid`
`--budget`	500 backtests de recherche par défaut en adaptive
`--display`	`compact` par défaut ; `study`, `technical`

Sans constructeur explicite, la recherche explore les trois familles internes. Les constructeurs positionnels `P1`, `P2`, `P3` sélectionnent une famille pour les usages experts. Par défaut, la sélection retient les 750 dernières séances après filtrage de date.

`raw` trie directement fitness, activité et investissement maximal. `compact` en affiche les 20 premiers avec le constructeur typé. `study` ajoute la lecture économique nette et les frais. `technical` expose les diagnostics et l'influence des paramètres en recherche adaptative.

Le classement expert `jedi` ajoute une préférence fondée sur les contrôles du moteur, notamment l'opérabilité, l'activité et les diagnostics de robustesse ; `both` affiche les deux lectures. Les diagnostics de robustesse examinent les résultats sur des variantes ou découpages ; ils ne garantissent pas le comportement futur. Conserver le classement, l'affichage et la stratégie de recherche avec les résultats. Le chapitre 05 présente le parcours courant en `raw`.

tools backtest

Option	Rôle et défaut
`--scale=DEF`	Définition littérale ou nom enregistré, requis
`--money=DEF`	Gestion du capital ; requise avec `--capital`
`--capital=AMOUNT`	Dimensionnement par capital, risque et opérabilité
`--distance=COEFFICIENT`	Calcul du montant fixe ; incompatible avec `--capital`
`--minimum-quantity=COUNT`	Minimum par ordre contrôlé, défaut 2 ; mode capital
`--operability-steps=COUNT`	Échelons contrôlés, défaut 3 ; mode capital
`--execution=MODE`	`conservative` par défaut ; `legacy`
`--opening-day-orders=WHEN`	`next` par défaut ; `same`
`--display=MODE`	`compact` par défaut ; `technical`
`-c`, `--command=COMMANDS`	Sections `r`, `f`, `o`, `t`, `d`, `p`

Sans `--capital`, le montant est calculé à partir de la configuration de simulation ; il n'existe pas d'option `--amount` pour `backtest`. Le rapport compact utilise `r` par défaut, le technique `rf`. Les sections `T`, `K`, `S` conservent les listes étendues de compatibilité. Voir le chapitre 22 pour les prix de gap, réouvertures et valorisations finales.

tools candidate

L'ISIN est positionnel et `--scale` est requis. `--price` remplace le cours enregistré et permet d'examiner un titre non enregistré. Choisir soit `--amount`, soit `--money` ; ces deux options sont incompatibles. `--capital` fournit K pour le financement et, avec `money`, le dimensionnement ; sinon K vient du capital dédié configuré.

`--minimum-quantity` vaut 2 par défaut ; `--operability-steps` vaut `diagnostic_steps`, soit **3 dans la configuration d'origine**. L'aide affiche sa valeur effective. `--extended` expose le détail du dimensionnement. L'opérabilité et le financement sont expliqués séparément au chapitre 06.

Définitions des barèmes

Les signatures reconnues par `scale create --builder=...` et par les outils sont :

Champs	Forme
`kc,ks,ki,kb`	Type I avec listes de prix et de stocks explicites
`kc,ki,kb,kd`	Type I avec construction optimisée de `ks`
`vr,ks,ki,kb`	Type II avec liste de stocks explicite
`vr,np,nn,ki,kb,kd`	Type II avec construction optimisée de `ks`
`vr,np,sr,nn,mi,ki,kb`	Type II, constructeur de compatibilité

Sans `--name`, `scale create --comment=... --builder=...` attribue `B_kc_NN` pour le type I ou `B_vr_NN` pour le type II, Legacy compris. Le numéro suivant est calculé séparément dans chaque famille. Un nom explicite doit être conforme au type et avoir un numéro positif d'au moins deux chiffres. Les définitions opérationnelles équivalentes sont refusées à la création et à la modification, même avec `--force`. Le chapitre 13 détaille les règles et la conservation des anciennes données.

`kc` et `ks` sont des listes décimales ; `np`, `nn`, `kb` sont des entiers. `vr`, `sr`, `mi` et `ki` acceptent un coefficient décimal ou un pourcentage ; `kd` est décimal. Employer le point décimal : `kc:1.4,1.2,0.8,0.6`. Les virgules séparent les valeurs et les champs.

Dans le constructeur de compatibilité, `sr` définit la fraction de stock visée au dernier échelon intérieur haussier ; `mi` règle le budget des achats intérieurs baissiers rapporté au montant initial. Les compteurs `np` et `nn` comprennent les bornes ; il reste `np-1` et `nn-1` échelons intérieurs. Cette forme a ses propres règles : examiner la définition opérationnelle et les ordres avec `scale show` et `play` avant de la comparer aux autres.

tools calc, play, plot et stats

`calc` accepte les fonctions `kcb`, `kcp`, `ksp`, `cpe`, `spe`, `quantity`, `price`, avec les paramètres `clé=valeur` nécessaires. Il calcule la mécanique du **type I**. `play` convient aux types I et II, à un état créé ou à un identifiant d'opération existante. Ses arguments décrivent les déplacements, prix cibles, transactions ou changements de paramètres.

Dans `play`, `-c o0,6-14,17` choisit les colonnes d'opérations ; ajouter `16` pour les frais. Les sections sont `t`, `k`, `s`, `c`, `o`, `r`, `f`, `d` ; `--verbose` développe les messages. Le chapitre 19 fournit des commandes complètes. Des arguments aléatoires, comme le déplacement `0`, demandent une attention particulière à la reproductibilité.

`plot` trace le cours et sa régression ; `--legend=tl|tr|bl|br` choisit le coin de la légende, sinon automatique. `stats` affiche les indicateurs statistiques de l'historique sélectionné. Ces deux outils complètent l'inspection des données ; leurs sorties ne choisissent pas un investissement à votre place.

Annexe 2. Configuration des fournisseurs de cours API et Web

Cette annexe complète le chapitre 08. Elle s'adresse à la personne qui doit **créer ou modifier** un fournisseur de cours courant. Cette tâche demande de savoir lire une documentation d'API ou une page HTML, écrire une expression régulière, examiner une réponse HTTP et vérifier les conditions d'accès de la source. Un fournisseur configuré n'est pas nécessairement disponible ou autorisé. Les exemples de cette annexe emploient des adresses et réponses **fictives** ; ils servent à comprendre la forme de la configuration, pas à obtenir un cours réel.

L'annexe 05 recense toutes les valeurs intégrées, y compris les champs actuellement non utilisés.

Fournisseurs intégrés et séparation des bases

La configuration d'origine essaie les pages Web ABC Bourse, Tradingsat et Boursorama, puis l'API Alpha Vantage. Ce sont des définitions de collecte, dont la disponibilité dépend du fournisseur et de la structure actuelle de sa réponse. Le nom `live` indique une collecte Web, sans promettre un cours en temps réel ou sans délai. `eod` indique le parcours API ; vérifier la séance effectivement renvoyée.

Ces appels traitent le cours courant de la base principale. Ils ne remplissent pas un historique OHLCV et ne remplacent pas `market import` ou `market update`, qui lisent des fichiers préparés. Le délai de renouvellement `quotlife` vaut 240 minutes dans la configuration d'origine ; `quote refresh --force` permet une actualisation forcée selon sa sélection.

Les exemples emploient `jedidemo` pour travailler dans l'espace d'essai. Les mêmes sous-commandes avec `jedi` s'appliquent au portefeuille de production.

Repérer le fichier et comprendre l'ordre des essais

`jedidemo config --paths` affiche le chemin du fichier de configuration effectivement utilisé (`config_file`) et celui du journal (`log_file`). Dans l'installation courante, le fichier est nommé `jedi.conf` ; les environnements de développement et de test emploient d'autres noms. Sa syntaxe est **YAML**, format qui utilise l'indentation pour ranger les clés sous leur parent. Jedi part d'une configuration intégrée, puis fusionne celle du fichier personnel : les objets de clés sont fusionnés, tandis qu'une valeur simple ou une liste fournie par l'utilisateur remplace la valeur intégrée correspondante. Écrire un petit remplacement est donc possible, mais une nouvelle liste `website_order` remplace toute l'ancienne liste.

```
jedidemo config --paths
```

Les cours courants se trouvent sous `market_data` dans le YAML. Le nom `market_data` ici désigne la configuration des fournisseurs ; il ne transforme pas `quote get` en lecture de la base historique `market`. La configuration intégrée essaie `live` puis `eod`. `live` lit des pages Web ; `eod` consulte une API pour un cours de fin de séance. Dans chaque mode, Jedi parcourt les noms de fournisseurs dans l'ordre configuré, puis les identifiants possibles pour ce fournisseur. Le premier résultat accepté arrête les essais.

Clé YAML	Rôle dans Jedi 1.0
`market_data.mode_order`	Chaîne ou liste des modes `live` et `eod` ; si absente ou vide, repli sur `live`.
`market_data.website_order`	Liste des noms à essayer dans `websites` ; si absente ou vide, parcours des entrées de `websites`.
`market_data.api_order`	Liste des noms à essayer dans `api_providers` ; si absente ou vide, parcours des entrées de `api_providers`.
`market_data.websites`	Définition des fournisseurs de pages Web.
`market_data.api_providers`	Définition des fournisseurs API.

Un nom inscrit dans un ordre sans définition correspondante est ignoré par le parcours. Pour ne tenter qu'un fournisseur précis, renseigner l'ordre correspondant ; une définition conservée dans `websites` ou `api_providers` ne suffit pas à l'activer si elle est absente d'une liste d'ordre non vide.

La présence de fournisseurs Web intégrés, dont ABC Bourse et Boursorama, ne constitue pas une autorisation de collecte : vérifier les conditions d'accès du fournisseur avant utilisation.

Configuration d'un fournisseur Web

L'exemple ci-dessous décrit une **page fictive** qui afficherait dans son HTML `data-price="12,34"` et `data-change="-0,50"`. L'adresse `.invalid` ne désigne aucun service utilisable. Remplacer l'ensemble par les détails d'une source dont l'accès est permis et dont la page a été vérifiée.

```
market_data:
  mode_order: ["live"]
  website_order: ["demo_web"]
  websites:
    demo_web:
      url: "https://cours.example.invalid/security/{key}"
      key_strategy:
        candidates:
          - template: "{isin}"
      regex_price: 'data-price="(?<price>\d+(?:[.,]\d+)?)"'
      regex_variation: 'data-change="(?<variation>(?:&#x2B;|&#x2D;|[+-])?\d+(?:[.,]\d+)?)"'
```

Cette configuration peut être lue par Jedi si elle figure sous `market_data` dans le fichier de configuration. Elle ne garantit ni que le HTML réel correspond, ni que le nombre est un cours correct. Le groupe nommé `price` est obligatoire pour `regex_price`. Si `regex_variation` est présente, son groupe nommé `variation` doit pouvoir être extrait ; sinon la tentative auprès de ce fournisseur échoue et Jedi poursuit le repli. Si elle est **absente, nulle ou une liste vide**, Jedi rend `0.0` pour la variation, valeur qui signifie dans ce cas « non extraite » et non « variation effectivement nulle ». `regex_price` ne peut être absente, nulle ni vide.

Clé sous `websites.<nom>`	Sens et forme
`url`	Adresse HTTP(S) comportant normalement `{key}` ; Jedi encode la clé construite dans ce segment de chemin.
`key_strategy.candidates`	Liste ordonnée de `template` ; les champs utilisables sont `{isin}`, `{ticker}`, `{name}` et les variables dérivées déclarées par `vars`.
`key_exceptions`	Dictionnaire facultatif `ISIN: clé` ; pour un ISIN présent, sa clé unique remplace les candidats ordinaires.
`regex_price`	Motif requis contenant `(?<price>...)`.
`regex_variation`	Motif facultatif contenant `(?<variation>...)` lorsqu'il est configuré.
`http_error_patterns`	Motifs facultatifs pour reconnaître une page d'erreur parmi ses premières lignes ; `null` désactive ce contrôle, une liste remplace les motifs intégrés.
`min_body_chars`, `min_body_lines`	Longueur minimale du corps en caractères ou lignes ; par défaut, 100 caractères et aucune limite de lignes. Les anciens noms `min_response_chars` et `min_response_lines` sont aussi lus.

Pour chaque candidat, Jedi remplace les variables connues dans le modèle. S'il reste un champ non résolu, ce candidat est omis. `quote get <isin>` fournit seulement l'ISIN ; les modèles exigeant ticker ou nom ne fonctionnent que si ces champs existent dans le titre enregistré, par exemple avec `quote update`. Les clés identiques sont dédupliquées. Si aucun candidat n'est disponible, la page n'est pas demandée.

Pour fabriquer une variable dérivée, la forme réellement lue par le code est :

```
key_strategy:
  candidates:
    - template: "{slug}-{isin}"
      vars:
```

```
      slug:
        from: "name"
        ops:
          - downcase
          - {gsub: {pattern: "\\.", replace: ""}}
          - {gsub: {pattern: " ", replace: "-"}}
```

`from` choisit `isin`, `ticker` ou `name`. Les opérations possibles dans `vars.<variable>.ops` sont `trim`, `upcase`, `downcase` et `gsub`, qui remplace les correspondances d'une expression régulière. Dans ce bloc, `"\\."` est la façon YAML d'écrire le motif regex `\.`. **Le champ `ops` placé directement sur un candidat n'est pas appliqué par `ScrapingKeyBuilder` dans Jedi 1.0** : placer les opérations dans `vars`, comme ci-dessus. La configuration intégrée présente un tel champ sur un candidat Boursorama, mais il ne faut pas compter sur cet effet pour écrire un nouveau fournisseur.

Les formes de regex acceptées

`regex_price` et `regex_variation` acceptent soit une chaîne, soit une liste. Dans la liste, une chaîne est un motif à essayer ; une sous-liste de chaînes est **concaténée en un seul motif**, puis essayée. Les alternatives sont tentées dans l'ordre jusqu'à ce qu'une extraction numérique réussisse :

```
regex_price:
  - 'data-price="(?<price>\d+(?:[.,]\d+)?)"'
  - ['class="last">', '(?<price>\d+(?:[.,]\d+)?)', '<']
```

Les trois fragments du second élément sont joints sans espace ni saut de ligne. Des apostrophes YAML autour de la regex préservent les antislashs. La capture doit représenter un nombre que Crystal peut convertir en décimal : Jedi retire ` `, les espaces extérieurs et remplace la virgule par un point, `+` par `+` et `-` par `-`. Il ne retire pas automatiquement tous les espaces entre chiffres, les symboles monétaires ni les autres entités HTML. Vérifier la capture, son titre et sa date, plutôt que le seul fait qu'une regex trouve un nombre.

Configuration d'un fournisseur API

L'API intégrée illustre le format `GLOBAL_QUOTE` d'Alpha Vantage. La configuration ci-dessous reprend **la forme des clés** avec une adresse et un nom de fournisseur fictifs. Elle décrit un service compatible avec cette structure de réponse précise ; elle ne constitue pas un adaptateur universel pour toute API JSON.

```
market_data:
  mode_order: ["eod"]
  api_order: ["demo_api"]
  api_providers:
    demo_api:
      api_key: "REMPLACER_PAR_UNE_CLE_PERSONNELLE"
      symbol_candidates:
        mode: "ticker_plus_suffixes"
        suffixes: [".PA", ""]
      quote:
        base_url: "https://api.example.invalid"
        path: "/query"
        query:
          function: "GLOBAL_QUOTE"
          symbol: "{symbol}"
          apikey: "{api_key}"
        price_path: ["Global Quote", "05. price"]
        prev_close_path: ["Global Quote", "08. previous close"]
        trading_day_path: ["Global Quote", "07. latest trading day"]
        change_percent_path: ["Global Quote", "10. change percent"]
        change_percent_strip_percent: true
```

Ne placer une vraie clé que dans une configuration personnelle protégée ; éviter de partager ce fichier, l'URL complète et les journaux bruts. Dans Jedi 1.0, le contexte de diagnostic peut contenir la clé lorsqu'elle figure dans l'URL. La clé `"yyyyy"` livrée pour Alpha Vantage est seulement un espace réservé.

Clé sous `api_providers.<nom>`	Rôle effectivement utilisé
`api_key`	Chaîne substituée dans la requête via `{api_key}`.
`symbol_candidates.mode`, `suffixes`	Avec `ticker_plus_suffixes`, ajoute chaque suffixe au ticker, dans l'ordre ; un ticker contenant déjà `.` ou `:` est gardé tel quel. Sans ticker, Jedi essaie l'ISIN sans suffixe.
`quote.base_url`, `quote.path`, `quote.query`	Construisent l'URL et ses paramètres ; valeurs de paramètres sous forme de chaînes, avec substitutions `{symbol}`, `{api_key}` ou `{q}` (`{q}` représente l'ISIN disponible).
`quote.price_path`, `quote.prev_close_path`	Chemins de clés JSON nécessaires pour lire le prix et la clôture précédente ; un élément numérique peut désigner un indice dans un tableau.
`quote.trading_day_path`	Chemin facultatif de date `AAAA-MM-JJ` en mode `eod` ; sinon Jedi utilise la date locale du jour.
`quote.change_percent_path`, `quote.change_percent_strip_percent`	Pourcentage renvoyé par l'API ; si le chemin est absent, calcul à partir du prix et de la clôture précédente. `true` retire le signe `%` du texte.

Dans l'adaptateur actuel, Jedi vérifie aussi les réponses contenant `Note` ou `Information` comme signes de limitation, et attend un objet `Global Quote` non vide. Même si des chemins JSON sont paramétrables, une autre API avec un objet racine différent peut donc nécessiter **une modification du code**. Les champs `symbol_path` et `change_path` existent dans la configuration intégrée, mais ne sont pas consultés dans la méthode actuelle d'extraction : ne leur attribuer aucun effet dans un nouveau paramétrage. `prev_close_path` reste requis par le code même si `change_percent_path` fournit déjà la variation.

Diagnostiquer sans inventer de cours

Commencer par vérifier le chemin et la syntaxe du fichier. Pour un titre dont l'ISIN est connu, une interrogation ne modifiant aucune base permet de constater si un fournisseur répond :

```
jedidemo config --paths
jedidemo quote get FR0000120271
```

Cette commande d'exemple est documentée dans l'aide ; **elle ne préjuge ni d'une réponse effective ni du droit d'interroger la source configurée**. `quote get` n'a pas d'option `--mode` et ne lit pas le cours enregistré. Pour cibler un mode sur un titre déjà enregistré, `quote update <id-ou-isin> --mode=eod` est possible, mais **enregistre** le résultat si l'appel réussit ; choisir un titre et une procédure de test adaptés avant de l'utiliser. `quote refresh --mode=eod` peut mettre à jour plusieurs titres et ne sert pas de premier essai de diagnostic. `quote set` permet de saisir un cours vérifié sans requête externe.

Symptôme ou étape	Vérification concrète
Configuration non prise en compte	Chemin `config_file`, indentation YAML, liste d'ordre qui contient le nom exact du fournisseur ; les listes remplacent les listes intégrées.
Aucun identifiant Web	ISIN présent, champs ticker/nom nécessaires au modèle, `vars` et transformations, exception par ISIN.
HTTP `429`, autre erreur, ou corps très court	Quota, URL, redirection, accès permis, page de consentement ; le statut `200` ne garantit pas la présence d'un cours.

Symptôme ou étape	Vérification concrète
Motif absent ou capture non numérique	HTML exact reçu, groupe nommé `price`/`variation`, format décimal et entités HTML ; contrôler que le prix correspond au bon titre.
API vide, limitée ou JSON inattendu	Symbole essayé, suffixes, clé et quota, structure `Global Quote` et chemins JSON.
Variation `0.0`	Vérifier si `regex_variation` était omise : ce zéro peut signifier « non extraite ».

Jedi essaie successivement motifs, clés, fournisseurs et modes selon leur configuration ; le contexte du dernier échec ne raconte pas nécessairement **tous** les essais antérieurs. Le journal peut contenir un extrait de la réponse, l'URL, une erreur de parsing et éventuellement une clé API : le consulter localement et masquer ces éléments avant tout partage. Une erreur de configuration, une réponse réseau et une extraction du mauvais chiffre sont des problèmes distincts. Tester le prix et sa date contre une source autorisée indépendante avant de l'enregistrer ou de l'utiliser dans un trade. Conserver la provenance et les conditions d'utilisation des données, selon le chapitre 21.

Annexe 3. Script de sélection externe ProScreener JEDI

Rôle du screener

Le ProScreener JEDI fournit une liste de titres à examiner dans ProRealTime. Ses critères sont cohérents avec les quatre principales questions de `profile` : tendance, liquidité, volatilité et oscillation.

La tendance, la liquidité et la volatilité déclenchent les filtres d'entrée dans la liste. L'oscillation apporte une information de choix ; elle n'est pas éliminatoire dans ce script. La liste laisse ensuite l'utilisateur choisir le titre, le barème et le projet à évaluer.

Préparer l'exécution

Dans ProRealTime, ouvrir ProScreener, créer un screener par programmation et coller le listing ci-dessous. Choisir explicitement le marché ou la liste de titres à examiner et travailler sur les **barres journalières**. Le programme indique `TIMEFRAME(Daily)`.

Lire les résultats après la fin de la séance choisie. Une barre en formation peut encore changer de clôture, de volume et d'amplitude. Pour rapprocher les résultats de Jedi, vérifier la date de la dernière cotation du dataset avec `market info` et `market list --last-quote`.

Le manuel officiel ProScreener, version 6.1.0 du 1er décembre 2025, consulté le 3 octobre 2026, indique 256 chandeliers utilisés pour les calculs, ou 1 024 sur les plateformes Premium. La fenêtre de 252 séances du script tient dans ce périmètre. Le manuel indique aussi trois ou six critères de sortie selon la licence.

Le listing utilise trois colonnes actives. Une variante à quatre colonnes est conservée en commentaire : utiliser cette variante seulement si la plateforme l'accepte, en remplaçant la dernière instruction active. Garder une seule instruction `SCREENER` active.

Ce que mesure le script

Mesure	Calcul et rôle
Tendance	Pente sur 252 séances, annualisée ; filtre strictement supérieur à −12 %.
Liquidité	Médiane de clôture × volume sur 252 séances ; filtre supérieur ou égal à 100 000.
ATR20	Moyenne simple de 20 amplitudes réelles, divisée par la clôture ; filtre supérieur ou égal à 1,5 %.
Oscillation	Mouvements et retournements d'au moins 5 % sur 252 séances, annualisés ; information affichée.

Avec des cours en euros et des volumes en titres, la liquidité est exprimée en euros par séance. Le seuil ne convertit pas les devises. L'ATR du script est calculé directement avec une moyenne simple ; conserver ce calcul pour la cohérence avec Jedi.

Les colonnes actives sont `Trend %/y`, `ATR20 %` et `Sw5/y`. La liquidité est filtrée même lorsqu'elle n'est pas affichée. Le programme accepte un historique d'au moins 252 barres ; il ne reproduit pas le contrôle de profondeur d'au moins trois années de `profile`.

Listing complet

```
// JEDI - ProScreener journalier
// Cible : ProRealTime / ProScreener
// Fenêtre principale : 252 séances ; ATR : 20 séances
//
// Filtres équivalents aux principaux critères éliminatoires de Jedi profile :
//   - tendance annualisée > -12 %
//   - montant quotidien médian >= 100 000
```

```
//   - ATR(20) simple >= 1,5 %
//
// L'oscillation à 5 % est calculée et affichée mais n'est pas éliminatoire.

TIMEFRAME(Daily)

// ---------------------------------------------------------------------
// 1. Historique minimal disponible dans ProScreener
// ---------------------------------------------------------------------
validHistory = BarIndex >= 251

// ---------------------------------------------------------------------
// 2. Tendance sur 252 séances
// Approximation de JEDI : régression de LOG(Close) par numéro de barre,
// annualisée sur 252 séances.
// ---------------------------------------------------------------------
trendSlope = LinearRegressionSlope[252](LOG(Close))
trendPct = (EXP(trendSlope * 252) - 1) * 100
trendOK = trendPct > -12

// ---------------------------------------------------------------------
// 3. Liquidité : médiane de Close * Volume sur 252 séances
// ---------------------------------------------------------------------
FOR i = 0 TO 251 DO
  $tradedValue[i] = Close[i] * Volume[i]
NEXT

ArraySort($tradedValue, ASCEND)
medianTradedValue = ($tradedValue[125] + $tradedValue[126]) / 2
liquidityOK = medianTradedValue >= 100000

// ---------------------------------------------------------------------
// 4. Volatilité : ATR(20) SIMPLE, comme Jedi
// Ne pas utiliser AverageTrueRange[20], qui applique le lissage de Wilder.
// ---------------------------------------------------------------------
atrSum = 0
FOR i = 0 TO 19 DO
  tr1 = High[i] - Low[i]
  tr2 = ABS(High[i] - Close[i + 1])
  tr3 = ABS(Low[i] - Close[i + 1])
  trueRange = MAX(tr1, MAX(tr2, tr3))
  atrSum = atrSum + trueRange
NEXT

atr20 = atrSum / 20
atrPct = 0
IF Close > 0 THEN
  atrPct = atr20 / Close * 100
ENDIF
volatilityOK = atrPct >= 1.5

// ---------------------------------------------------------------------
// 5. Oscillation : nombre de mouvements/reversals de 5 % sur 252 séances
// Algorithme calqué sur Jedi ; valeur annualisée d'après la durée calendaire.
// Ce critère est informatif, pas éliminatoire.
// ---------------------------------------------------------------------
pivot = Close[251]
direction = 0
swings5 = 0

FOR i = 250 DOWNTO 0 DO
  c = Close[i]

  IF direction = 0 THEN
    changePct = (c / pivot - 1) * 100

    IF changePct >= 5 THEN
      direction = 1
      swings5 = swings5 + 1
      pivot = c
    ELSIF changePct <= -5 THEN
      direction = -1
      swings5 = swings5 + 1
```

```
      pivot = c
    ENDIF

  ELSIF direction > 0 THEN

    IF c > pivot THEN
      pivot = c
    ELSE
      reversalPct = (pivot - c) / pivot * 100
      IF reversalPct >= 5 THEN
        direction = -1
        swings5 = swings5 + 1
        pivot = c
      ENDIF
    ENDIF

  ELSE

    IF c < pivot THEN
      pivot = c
    ELSE
      reversalPct = (c / pivot - 1) * 100
      IF reversalPct >= 5 THEN
        direction = 1
        swings5 = swings5 + 1
        pivot = c
      ENDIF
    ENDIF

  ENDIF
NEXT

spanYears = (Days - Days[251]) / 365.25
swingsPerYear = 0
IF spanYears > 0 THEN
  swingsPerYear = swings5 / spanYears
ENDIF

// ---------------------------------------------------------------------
// 6. Sélection JEDI
// ---------------------------------------------------------------------
selected = validHistory AND trendOK AND liquidityOK AND volatilityOK

// Version GRATUITE : ProScreener accepte 3 critères de sortie.
// La liquidité est bien filtrée mais n'est pas affichée afin de laisser
// visibles tendance, volatilité et oscillation.
SCREENER[selected](trendPct AS "Trend %/y", atrPct AS "ATR20 %", swingsPerYear AS "Sw5/y")

// Version PREMIUM : remplacer la ligne SCREENER précédente par celle-ci
// pour afficher aussi la liquidité :
// SCREENER[selected](trendPct AS "Trend %/y", medianTradedValue AS "Med value/d", atrPct AS "ATR20 %",
  swingsPerYear AS "Sw5/y")
```

Lire et utiliser la liste

Commencer par quelques titres dont la tendance, l'ATR et l'oscillation sont compréhensibles. Un ATR supérieur à 6 % mérite la même attention que la vigilance correspondante de `profile`, bien que ce script conserve le titre. Une faible oscillation reste également à examiner.

Les gaps, le recul maximal, le pas suggéré et les contrôles de qualité longs sont à consulter dans Jedi ou sur l'historique. Le screener ne fournit pas un classement global à sept critères.

La mesure de tendance du script utilise les numéros de barres et une annualisation sur 252 séances ; Jedi utilise les dates de l'historique. La source des cours, leurs ajustements, les volumes et les populations comparées peuvent donc produire des écarts. Examiner un écart important sur ces éléments avant de changer les règles.

Un résultat de screener ne transmet aucun cours à la base principale de Jedi. Pour le titre choisi, examiner son fichier de cotations ou son historique en base avec `profile` et `score`, puis lancer `candidate` avec un prix actuel vérifié. Le chapitre 03 décrit ce raccordement ; le chapitre 10 explique les fichiers et groupes.

L’objectif reste une liste de choix cohérente sur les principaux critères, accompagnée d’une lecture des points de vigilance.

Annexe 4. Glossaire et notations Jedi

Objets et parcours

Terme	Sens dans Jedi
Titre	Instrument identifié par un ISIN, un nom et un ticker.
Barème	Règles qui associent un état mécanique à des prix et quantités cibles.
Constructeur	Paramètres permettant de fabriquer un barème ; le préfixe `I:` ou `II:` de `score` annonce son type.
Définition opérationnelle	Définition effectivement utilisée après construction et éventuelles transformations ; elle apparaît dans les rapports.
Trade	Parcours d'achats et de ventes sur un titre, avec ses paramétrages successifs.
Paramétrage	Barème applicable à partir d'une date et d'une heure, avec les éléments nécessaires à sa continuité.
Opération	Achat ou vente exécuté et enregistré. Un ordre proposé ne constitue pas une opération.
Stock	Nombre de titres détenus à un moment du parcours.
Allers-retours	Ventes et achats successifs suivant les déplacements du barème.
Opérabilité	Possibilité de produire les ordres contrôlés avec des quantités entières suffisantes.
Financement	Possibilité de couvrir l'ouverture et les besoins projetés, en tenant compte des autres trades.
Note	Entrée datée du journal ; elle décrit un choix ou un événement et ne modifie pas le trade.

Positions et coefficients

Notation	Sens
`x:y:e`	Position : sauts haussiers, sauts baissiers et échelon dans le paramétrage actif.
Échelon	Étape locale du barème.
Palier	Niveau de référence pour une série d'échelons ; atteindre une borne provoque un saut.
`q0`, `c0`	Quantité et prix de référence du trade ; initialement ceux du premier achat.
`kc`, `ks`	Listes de coefficients de prix et de stock d'un barème de type I.
`vr`	Variation relative servant à construire le pas d'un barème.
`np`, `nn`	Nombre d'échelons vers les bornes positive et négative.
`sr`	Fraction de stock visée au dernier échelon intérieur haussier du constructeur de compatibilité.
`mi`	Paramètre de budget des achats baissiers de ce constructeur, rapporté au montant initial.
`ki`	Fraction d'investissement appliquée lors des changements de palier.

Notation	Sens
`kb`	Coefficient de déplacement des prix de palier.
`kd`	Paramètre du constructeur qui transforme les quantités ; il ne désigne pas un rendement.
`kcb`, `kcp`, `ksp`	Coefficients calculés de capital, de prix et de stock.
`cpe`, `spe`	Cours et stock théoriques à une position cible.
Type I	Échelle reposant sur des coefficients de référence ; `calc` en vérifie les calculs.
Type II	Barème recentré à partir des opérations réelles ; l'examiner avec `play`.
Optimisé	Variante de construction ou de continuité ; ce qualificatif ne prouve pas un meilleur gain.

Les chapitres 13 à 16 donnent le contexte des paramètres. `q0` et `c0` peuvent être redéfinis par une réinitialisation ; lire le paramétrage actif avant de les interpréter comme le tout premier achat historique.

Split et nouvelle origine

Notation	Sens
`split=NEW:OLD`	Nombre d'actions nouvelles pour les anciennes ; `1:10` est un regroupement de dix pour une.
Stock effectif	Stock à utiliser après conversion, même si la dernière opération historique conserve l'ancienne quantité.
`reinit=reset,prix`	Nouvelle origine mécanique immédiate ; position `0:0:0`, stock effectif comme `q0`, prix choisi comme `c0`.
`reinit=merge,prix`	Nouvelle origine établie par la transaction manuelle suivante ; références en attente jusque-là.
`split + reinit`	Combinaison de next-version / v1.1 ; convertir le stock avant reset ou merge, saisir les prix après split.
Rompu	Droit fractionnaire issu d'un regroupement ; sa gestion et son règlement ne sont pas comptabilisés par Jedi.

Le chapitre 09 détaille les effets et les restrictions. Une réinitialisation change les références mécaniques, pas les résultats financiers passés.

Ordres et protection

Terme	Sens
Limit	Achat à un prix maximal ou vente à un prix minimal.
Stop	Ordre déclenché au franchissement d'un niveau ; le prix d'exécution peut s'en écarter.
StopLoss	Ordre de protection : coupe-perte ou protection de gains, selon l'état du trade.
Pas de cotation, `tick`	Incrément de prix utilisé pour arrondir un ordre.

Terme	Sens
Gap	Écart entre la clôture précédente et l'ouverture suivante ; il peut franchir un niveau sans exécution à ce niveau.
`conservative`	Politique OHLC courante, avec priorité aux ordres exécutables à l'ouverture et traitement des gaps.
`next`	Ordres issus du premier achat actifs à la séance suivante ; défaut du backtest.

Le Stop de fonctionnement du barème et la protection StopLoss répondent à des rôles distincts. Les règles détaillées figurent aux chapitres 17 et 22.

Capital

Notation	Sens
`K`	Capital affecté au trading ; référence monétaire du dimensionnement et du financement.
`r`	Fraction de K correspondant au risque nominal d'un trade.
`R`	Risque monétaire : `K × r`.
`o`	Coefficient d'objectif ; seuil monétaire `0 = o × R` avant protection des gains.
`n`	Nombre d'échelons baissiers utilisé pour le trajet de risque.
`T`	Montant demandé à l'ouverture, hors frais.
`money`	Définition `r:...,o:...,n:...` ; les trois champs sont nécessaires.
Réserve future	Besoin d'achats supplémentaires projeté sur l'horizon affiché.
Marge après candidat	Capital restant après les besoins des trades engagés et du candidat.
`diagnostic_steps`	Réglage des contrôles opérationnels, 3 à l'origine ; sa valeur effective apparaît dans l'aide concernée.

Le risque nominal n'est pas un plafond garanti de perte. La réserve de `candidate` est une projection hors frais sur un horizon ; elle n'est pas la preuve que tous les parcours ultérieurs seraient financés. Voir les chapitres 06 et 18.

Données et fraîcheur

Terme	Sens
OHLCV	Ouverture, plus haut, plus bas, clôture et volume d'une séance.
Fichier préparé	Cotations identifiées, converties dans le format Jedi et contrôlées avant utilisation.
Source	Nom d'une base de marché, par exemple `demo`.
Univers	Périmètre général entrant dans le nom de la base, `france` par défaut.
Dataset	Ensemble importé, avec ses appartenances, fichiers et métadonnées.
Groupe	Liste nommée de titres, indépendante de l'appartenance aux datasets.

Terme	Sens
Dernier import complet	Date et périmètre du dernier import ou remplacement complet ; historique de l'import.
Dernière cotation disponible	Date la plus récente présente ; elle peut concerner très peu de titres.
Fraîcheur globale	Date atteinte par au moins 95 % des titres du dataset, calculée sur la base réelle après import ou update.
Retard individuel	Écart en séances de marché avec cette référence : 0 normal, 1–2 vigilance, plus de 2 écarté.

La fraîcheur n'est pas calculée par rapport à la date du jour. Un fichier, une sélection historique ou une référence de dataset ambiguë peut ne pas permettre son évaluation. Les chapitres 04 et 10 expliquent les cas.

Critères de profile

Critère	Lecture
Tendance	Direction estimée des clôtures sur 252 séances, exprimée en taux annuel.
Liquidité	Médiane de Close × Volume ; montant échangé par séance, en euros si les prix le sont.
ATR20	Moyenne des amplitudes réelles sur 20 séances, incluant les écarts avec la clôture précédente ; exprimée en pourcentage du cours.
Oscillation	Amplitude du parcours et mouvements de retournement ; `Sw5/a` compte les mouvements d'au moins 5 % et les annualise.
Drawdown	Plus grande baisse depuis un sommet antérieur sur la fenêtre étudiée.
Gap%	Fréquence des gaps d'au moins 3 % sur la fenêtre récente.
Pas%	Pas de barème suggéré à partir des caractéristiques du titre ; ce n'est pas un ordre imposé.
Total	Somme sans pondération des N premiers rangs ; plus faible signifie mieux classé dans la sélection courante.
Statut	Retenu, vigilance ou écarté selon les contrôles ; un écarté ne participe pas au classement.

Résultats et étude

Terme	Définition
Gain brut	Ventes − achats + stock final valorisé à la dernière clôture, hors frais.
Gain net	Gain brut − frais imputés.
Investissement courant	Achats − ventes cumulés, hors frais.
Investissement maximal	Pic achats moins ventes d'un trade ; pour `backtest` et `score`, plus grand de ces pics sur les trades successifs, hors frais.

Terme	Définition
Solde achats–ventes du backtest	Bilan cumulé conservant les gains et pertes des trades clos ; il sert au calcul du gain cumulé.
Fitness	Dans `score`, gain brut / montant du premier achat ; couvre toute la période.
Inv.max	Dans `score`, investissement maximal / premier achat, hors frais.
Op/mois	Toutes les opérations, ouvertures comprises, ramenées à 21 séances à partir de la première opération.
Rnet%	Résultat net rapporté à l'investissement maximal dans l'affichage économique.
Rann%	Annualisation de ce rapport selon la durée ; elle ne constitue pas une prévision.
Recherche adaptive	Exploration puis approfondissement de zones choisies ; le budget limite les barèmes évalués.
Recherche grid	Évaluation exhaustive des candidats disponibles selon le constructeur.
Classement raw	Tri direct des mesures affichées.
Classement jedi	Lecture experte ajoutant des préférences et diagnostics du moteur.
Vérification séparée	Test d'un barème fixé sur des données qui n'ont pas servi à le choisir.
Suivi daté	Conservation d'une décision à sa date réelle, puis de ses observations ultérieures.

Voir le chapitre 24 pour interpréter ces résultats.

Annexe 5. Configuration par défaut et usage des paramètres

Cette annexe décrit toutes les entrées de la configuration intégrée à Jedi, dans la constante `App::YAML_CONFIG` de `src/jedi/app/application.cr`. Elle correspond aux sources de cette édition, en préparation de v1.1. Une valeur intégrée sert de point de départ : elle ne décrit pas nécessairement votre configuration effective.

Les tableaux donnent la valeur d'origine et l'usage réellement fait par le code. Les quelques champs présents mais non consultés sont signalés. Le bloc YAML complet en fin d'annexe reproduit toutes les valeurs, y compris les motifs techniques de collecte des cours.

Lire la configuration effective

```
jedi config --paths
jedi config
```

La première commande affiche les chemins, notamment `config_file`. La seconde affiche ces chemins et la configuration effective, après fusion avec le fichier personnel. Dans l'espace d'essai, utiliser les mêmes commandes avec `jedidemo` ; son fichier peut être différent. Le chapitre 02 explique ces espaces.

Le fichier personnel utilise le format YAML : l'indentation range les entrées sous leur parent. Jedi fusionne les ensembles de clés imbriqués ; une valeur simple ou une liste fournie remplace entièrement la valeur ou la liste correspondante. Il n'est donc pas nécessaire de recopier toute la configuration.

Pour affecter **50 000 € à la production et à la simulation**, ces seules entrées suffisent dans le fichier de l'espace concerné :

```
trading:
  capital: 50000
simulation:
  capital: 50000
```

La configuration intégrée fixe les deux capitaux à **20 000**, pas à 50 000. Une valeur de 50 000 affichée par `config` peut donc venir de ce remplacement personnel. Les valeurs s'appliquent à la prochaine invocation de Jedi ; les données et les opérations déjà enregistrées ne sont pas recalculées.

Affichage, surveillance et contrôles

Entrée	Valeur intégrée	Usage effectif
`border`	`"ascii"`	Style des bordures des tableaux. `--border=PRESET`, placé avant la famille de commande, le remplace pour cette exécution.
`watchrate`	`10.0`	Seuil de proximité dans la liste des ordres, en pourcentage : `10.0` signifie 10 %. Une distance absolue strictement inférieure à ce seuil reçoit le repère de surveillance si aucun repère de disponibilité ne s'applique. Ce réglage modifie l'affichage, pas le risque ni les prix d'ordres.

Entrée	Valeur intégrée	Usage effectif
`diagnostic_steps`	`3`	Horizon de contrôle des quantités sur les prochaines hausses. Sert aussi de défaut à `--operability-steps` de `candidate` et du backtest dimensionné par capital. Pour les réserves de `candidate`, l'horizon vaut le plus grand de ce nombre et de `money.n`.
`quotlife`	`240.0`	Âge maximal, en minutes, du cours enregistré avant que le résolveur du cours courant tente une actualisation : 240 minutes = 4 heures. Si cette tentative échoue, un ancien cours enregistré peut être réutilisé avec un avertissement.
`tickcheck`	`true`	Contrôle du prix enregistré par `operation create` contre le pas de cotation du titre. Un prix préfixé par `+` contourne ce contrôle pour cette opération. `play` et les moteurs de backtest enregistrent leurs opérations simulées sans ce contrôle, mais les ordres restent arrondis au pas de cotation.

Les styles de `border` sont `ascii`, `reducedascii`, `modern`, `reducedmodern`, `fancy`, `markdown`, `empty` et `blank`. Par exemple :

```
jedidemo --border=modern trade list
```

`watchrate` compare le prix proposé à celui du cours enregistré ; il ne déclenche aucune opération. `quotlife` est consulté par le résolveur du cours courant, notamment lors des ordres de trades. Ce n'est pas une tâche d'actualisation périodique : `quote get` interroge les fournisseurs sans utiliser ce délai ni modifier la base. Les ordres de `play` utilisent le prix de la simulation, sans ce renouvellement Web.

Frais des opérations

Les entrées suivantes sont sous `fees`.

Entrée	Valeur intégrée	Usage effectif
`rate`	`0.0035`	Taux proportionnel de frais : `0.0035` = 0,35 % du montant de l'opération. Utilisé sans `--fees` explicite, ainsi que pour les nouvelles opérations de `play`, `backtest` et `score`.

Entrée	Valeur intégrée	Usage effectif
`min`	`0.0`	Minimum monétaire par opération, dans la même monnaie que les prix. Le calcul retient le plus grand du montant proportionnel et de ce minimum, puis arrondit au centime.

Avec les valeurs intégrées, une opération de 1 000 € supporte 3,50 € de frais proportionnels. Pour un achat, la taxe éventuelle du titre s'ajoute au taux avant la comparaison avec le minimum. Un montant explicite `operation create --fees=...` remplace le calcul automatique. Les projections de financement de `candidate` sont hors frais. Les frais simulés entrent dans les résultats de `play`, `backtest` et `score` ; ils ne changent pas les coefficients du barème.

Une modification de `fees.rate` ou `fees.min` concerne les nouvelles opérations ; elle ne recalcule pas les frais déjà enregistrés. Pour corriger une opération existante, utiliser `operation fees ID MONTANT`, expliqué au chapitre 08. Le montant remplace directement le total enregistré ; les paramètres de frais et la taxe ne lui sont pas ajoutés.

Capital réel et limites de money

Sous `trading` :

Entrée	Valeur intégrée	Usage effectif
`capital`	`20000`	Capital de référence des protections `money` des trades réels. Sert aussi au dimensionnement et au financement de `candidate` si `--capital` est absent. N'impose pas la quantité du premier achat d'un trade.

Sous `money` :

Entrée	Valeur intégrée	Usage effectif
`risk_rate_min`	`0.005`	Borne inférieure admise pour le champ `r` de `money` : 0,5 %.
`risk_rate_max`	`0.05`	Borne supérieure admise pour `r` : 5 %.
`target_coeff_min`	`0.5`	Borne inférieure admise pour le coefficient d'objectif `o`.
`target_coeff_max`	`5.0`	Borne supérieure admise pour `o`.
`steps_down_min`	`1`	Borne inférieure admise pour le nombre entier d'échelons baissiers `n`.
`steps_down_max`	`5`	Borne supérieure admise pour `n`.

Ces bornes sont **inclusives**. Ce sont des limites de validation, pas une définition `money` automatiquement attribuée aux trades. Les trois champs `r`, `o` et `n` doivent être fournis, par exemple `money=r:2%,o:2,n:2`. `r` désigne la fraction risquée, `o` multiplie le risque pour définir l'objectif, et `n` fixe le nombre d'échelons de baisse projetés. Les entrées `risk_rate_min` et `risk_rate_max` peuvent aussi être écrites comme chaînes de pourcentage dans le YAML, par exemple `"0.5%"`.

Paramètres de simulation

Les entrées suivantes sont sous `simulation`. Les taux y sont des fractions numériques : `0.015` signifie 1,5 %, tandis que `watchrate: 10.0` signifie directement 10 %.

Entrée	Valeur intégrée	Usage effectif
`capital`	`20000`	Capital de référence des protections de `play`. Entre aussi dans le montant initial de référence de `score` et du backtest sans `--capital`.
`distance`	`0.2`	Fraction utilisée au dénominateur du montant initial de référence : `0.2` = 20 %. Utilisée par `score` et le backtest sans `--capital` ; `--distance` peut la remplacer dans ce dernier. N'intervient pas dans `play`.
`risk_rate`	`0.015`	Fraction utilisée au numérateur du montant initial de référence : `0.015` = 1,5 %. Utilisée par `score` et le backtest sans `--capital`. Ne fournit jamais implicitement le champ `r` de `money`, y compris dans `play`.

Le montant initial de référence est calculé au démarrage :

```
montant initial = capital × risk_rate / distance
valeurs intégrées : 20 000 × 0,015 / 0,2 = 1 500 €
capital personnel de 50 000 : 50 000 × 0,015 / 0,2 = 3 750 €
```

Le résultat est tronqué à un montant entier. Garder un capital et une `distance` strictement positifs. Pour les protections, le moteur se replie sur `trading.capital` si le capital de simulation n'est pas positif. **`play` n'utilise pas le montant initial de référence** : son ouverture est celle de `q0` titres au prix `c0`, ou celle de l'opération choisie comme point de départ.

Quel réglage pour quel outil ?

Usage	Capital	Taux de risque	Distance ou horizon
Ordres d'un trade réel avec `money`	`trading.capital`	`r` du trade	`n` du trade et son barème.
`tools play` avec `money`	`simulation.capital`	`r` fourni dans la simulation	`n` et barème de la simulation.
`tools candidate --money=...`	`--capital`, sinon `trading.capital`	`r` de `--money`	Distance obtenue à `n` échelons du barème.
`tools backtest` sans `--capital`	`simulation.capital` pour le montant initial	`simulation.risk_rate` pour ce montant ; `money.r` pour les protections si activées	`simulation.distance`, remplacée par `--distance` si fourni, pour le montant ; `money.n` pour les protections.
`tools backtest --capital=...`	Capital explicite	`r` de `--money`, qui est requis	Distance obtenue à `n` échelons ; `--distance` est incompatible avec ce mode.

Usage	Capital	Taux de risque	Distance ou horizon
`tools score`	`simulation.capital` dans le montant initial	`simulation.risk_rate` dans ce montant	`simulation.distance` dans ce montant ; les simulations de score fonctionnent sans les protections `money`.

Avec `candidate --amount=...`, le montant est explicitement choisi : il n'est pas dimensionné à partir de `simulation.distance` ou de `simulation.risk_rate`.

Exemple de play avec un capital personnel de 50 000 €

Après le remplacement de configuration montré plus haut, cette simulation fictive fournit elle-même `r`, `o` et `n` :

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,q0=1000,c0=10,tick=0.01,money=r:2%,o:2,n:2,loss=active"
    -1
```

Le risque vaut **1 000 €** (`50 000 × 2 %`) et le seuil d'objectif **2 000 €** (`1 000 × 2`). La projection porte sur deux échelons du barème. Le premier achat porte bien sur 1 000 titres à 10 €, soit **10 000 € hors frais**, malgré le montant initial de référence de 3 750 €. Dans ce scénario, l'étape baissière exécute une vente de protection à 9 €, soit une perte brute de 1 000 € avant frais. Changer seulement `simulation.distance` ou `simulation.risk_rate` ne change pas ce parcours ; changer `money.r` peut changer la protection.

Le chapitre 18 détaille le risque et le dimensionnement ; le chapitre 19 présente les parcours de `play`.

Univers et ordre de collecte des cours

Les entrées suivantes sont sous `market_data`.

Entrée	Valeur intégrée	Usage effectif
`default_universe`	`"france"`	Univers utilisé par défaut pour choisir la base historique de marché, notamment par `market` et les outils lisant un historique en base. `--universe` le remplace lorsqu'il est proposé. Ne sélectionne ni un dataset ni le groupe PEA.
`mode_order`	`["live", "eod"]`	Ordre des modes de collecte lorsque la commande ne fixe pas un mode : pages Web `live`, puis API `eod`. Le premier résultat accepté termine la recherche.
`api_order`	`["alphavantage"]`	Noms des fournisseurs API à essayer, dans cet ordre, en mode `eod`. Le nom doit correspondre à une définition dans `api_providers`.
`website_order`	`["abcbourse", "tradingsat", "boursorama"]`	Noms des fournisseurs Web à essayer, dans cet ordre, en mode `live`. Le nom doit correspondre à une définition dans `websites`.

L'univers concerne la base historique ; les autres entrées de cette section concernent la collecte d'un **cours courant**. Elles ne remplissent pas un historique OHLCV. `live` désigne une lecture Web, sans garantir un cours en temps réel ; `eod` désigne le parcours API. Lors du calcul des ordres réels, le résolveur demande `live` par défaut : modifier `mode_order` ne force pas ce parcours à utiliser `eod`.

Une liste d'ordre vide fait parcourir toutes les définitions correspondantes ; elle ne désactive pas les fournisseurs. Pour une liste non vide, une définition absente de cette liste n'est pas essayée. L'annexe 02 explique la configuration et le diagnostic des fournisseurs.

Fournisseur API intégré

Les entrées suivantes sont sous `market_data.api_providers.alphavantage`.

Entrée	Valeur intégrée	Usage effectif
`api_key`	`"yyyyy"`	Valeur envoyée au fournisseur via `{api_key}`. yyyyy est une valeur d'attente, à remplacer par sa propre clé pour utiliser cette API.
`symbol_candidates.mode`	`"ticker_plus_suffixes"`	Fabrique les symboles API à partir du ticker et des suffixes. Un ticker contenant déjà `.` ou `:` est conservé tel quel. Sans ticker, Jedi utilise l'identifiant disponible, ici l'ISIN.
`symbol_candidates.suffixes`	`[".PAR", ".PA", ""]`	Suffixes essayés dans l'ordre. Le dernier, vide, essaie le ticker seul. Aucun suffixe n'est ajouté à un ISIN utilisé faute de ticker.

Sous `market_data.api_providers.alphavantage.quote` :

Entrée	Valeur intégrée	Usage effectif
`base_url`	`"https://www.alphavantage.co"`	Adresse de base du fournisseur, à laquelle le chemin est ajouté.
`path`	`"/query"`	Chemin de la requête sous cette adresse de base.
`query.function`	`"GLOBAL_QUOTE"`	Valeur du paramètre HTTP `function` demandant la réponse `GLOBAL_QUOTE`.
`query.symbol`	`"{symbol}"`	Paramètre HTTP remplacé par chaque symbole fabriqué précédemment.
`query.apikey`	`"{api_key}"`	Paramètre HTTP remplacé par `api_key`.
`symbol_path`	`["Global Quote", "01. symbol"]`	**Non utilisé** par l'extraction actuelle. Sa présence n'entraîne pas de vérification du symbole renvoyé.
`price_path`	`["Global Quote", "05. price"]`	Chemin, dans la réponse JSON, vers le prix à lire.
`trading_day_path`	`["Global Quote", "07. latest trading day"]`	Chemin vers la date de séance. En mode `eod`, cette date est enregistrée à minuit local.

Entrée	Valeur intégrée	Usage effectif
`prev_close_path`	`["Global Quote", "08. previous close"]`	Chemin vers la clôture précédente. Le code la lit même lorsque le pourcentage de variation est fourni directement.
`change_path`	`["Global Quote", "09. change"]`	**Non utilisé** par l'extraction actuelle : la variation en montant n'est pas lue à ce chemin.
`change_percent_path`	`["Global Quote", "10. change percent"]`	Chemin vers le pourcentage de variation. S'il n'est pas fourni, Jedi calcule la variation à partir du prix et de la clôture précédente.
`change_percent_strip_percent`	`true`	Retire le caractère `%` du texte de variation avant sa conversion en nombre.

Un chemin tel que `["Global Quote", "05. price"]` signifie : ouvrir l'objet `Global Quote`, puis y lire `05. price`. Le lecteur actuel vérifie aussi qu'un objet `Global Quote` non vide existe. Changer seulement ces chemins ne suffit donc pas nécessairement pour utiliser une autre API.

Fournisseurs Web intégrés

Chaque site possède sa propre adresse, ses modèles d'identifiants et ses motifs de capture. Les tableaux indiquent les entrées et leur usage ; les adresses et les regex sont placées sous chaque tableau, sur toute la largeur de la page. Dans les chaînes YAML entre guillemets, l'antislash final permet de poursuivre la même chaîne à la ligne suivante sans ajouter de caractère. Les valeurs sont identiques à celles du code. Leur présence ne garantit pas que le site répondra ou que son HTML restera compatible.

ABC Bourse

Sous `market_data.websites.abcbourse` :

Entrée	Valeur intégrée	Usage effectif
`url`	Adresse ci-dessous	Page du titre.
`regex_price`	Motifs ci-dessous	Extraction du prix.
`regex_variation`	Motifs ci-dessous	Extraction de la variation.

Dans l'adresse, `{key}` est remplacé par chaque identifiant candidat. Les regex recherchent les nombres dans le HTML : le groupe nommé `price` fournit le prix ; `variation` fournit la variation en pourcentage.

```
url: "https://www.abcbourse.com/cotation/{key}"
regex_price: "id=\"lastcx\">(?<price>\\d+(?:[.,]\\d+)*)\\s*&#x20AC;"
regex_variation: "id=\"varcx\"\\s+class=\"quote_(up|down)b\">(?<variati\
  on>(&#x2B;|&#x2D;|\\+|-)?\\d+(?:[.,]\\d+)*)%"
```

Sous `key_strategy.candidates` :

Entrée	Valeur intégrée	Usage effectif
`[0].template`	`"{ticker}p"`	Ticker suivi de `p`.
`[1].template`	`"{isin}"`	ISIN seul.

Les modèles sont essayés dans cet ordre. Le premier est écarté si le ticker est indisponible.

Tradingsat

Sous `market_data.websites.tradingsat` :

Entrée	Valeur intégrée	Usage effectif
`url`	Adresse ci-dessous	Page du titre.
`regex_price`	Motifs ci-dessous	Extraction du prix.
`regex_variation`	Motifs ci-dessous	Extraction de la variation.

Dans l'adresse, `{key}` est remplacé par chaque identifiant candidat. Les regex recherchent les nombres dans le HTML : le groupe nommé `price` fournit le prix ; `variation` fournit la variation en pourcentage.

```
url: "https://www.tradingsat.com/{key}"
regex_price: "class=\"price\">(?<price>\\d+(?:[,.]\\d+)*)\\s*€"
regex_variation: "class=\"variation var-(down|neutral|up)\">(?<variatio\
  n>[+-]?\\d+(?:[,.]\\d+)*)\\s*%"
```

Sous `key_strategy.candidates` :

Entrée	Valeur intégrée	Usage effectif
`[0].template`	`"{name_slug}-{isin}"`	Nom transformé et ISIN.

Ce modèle est écarté si le nom nécessaire est indisponible. `name_slug` est construit par les transformations ci-dessous.

Sous `key_strategy.candidates[0].vars.name_slug` :

Entrée	Valeur intégrée	Usage effectif
`from`	`"name"`	Nom du titre.
`ops[0]`	`"downcase"`	Passage en minuscules.
`ops[1].gsub.pattern`	`"\\."`	Recherche des points.
`ops[1].gsub.replace`	`""`	Supprime les points.
`ops[2].gsub.pattern`	`" "`	Recherche des espaces.
`ops[2].gsub.replace`	`"-"`	Remplace les espaces par des tirets.

Boursorama

Sous `market_data.websites.boursorama` :

Entrée	Valeur intégrée	Usage effectif
`url`	Adresse ci-dessous	Page du titre.
`regex_price`	Motifs ci-dessous	Extraction du prix.
`regex_variation`	Motifs ci-dessous	Extraction de la variation.

Dans l'adresse, `{key}` est remplacé par chaque identifiant candidat. Les regex recherchent les nombres dans le HTML : le groupe nommé `price` fournit le prix ; `variation` fournit la variation en pourcentage.

```
url: "https://www.boursorama.com/cours/{key}/"
regex_price: "c-faceplate__info.+?price.+?ist-last>(?<price>\\d+(?:[,.]\
  \\d+)*)<"
regex_variation: "c-faceplate__info.+?fluctuation.+?ist-variation>(?<va\
  riation>[+-]?\\d+(?:[.,]\\d+)*)%"
```

Sous `key_strategy.candidates` :

Entrée	Valeur intégrée	Usage effectif
`[0].template`	`"1rP{ticker}"`	Préfixe `1rP` et ticker.
`[1].template`	`"1rEP{ticker}"`	Préfixe `1rEP` et ticker.
`[2].template`	`"{ticker}"`	Ticker seul.
`[2].ops.ticker`	`["upcase", "trim"]`	**Non appliqué.**

Les trois modèles sont essayés dans cet ordre et écartés si le ticker est indisponible. **Les transformations `upcase` et `trim` sous `[2].ops.ticker` ne sont pas utilisées à cet emplacement** par le constructeur actuel. Les opérations prises en charge se trouvent sous `vars.<variable>.ops`.

Configuration intégrée complète

Ce bloc reproduit les valeurs intégrées sous une présentation YAML directement lisible. Les commentaires du code ne sont pas des entrées de configuration. Il sert de référence ; un fichier personnel peut se limiter aux seuls remplacements souhaités.

```
border: ascii
market_data:
  default_universe: france
  mode_order:
  - live
  - eod
  api_order:
  - alphavantage
  website_order:
  - abcbourse
  - tradingsat
  - boursorama
  api_providers:
    alphavantage:
      api_key: yyyyy
      symbol_candidates:
        mode: ticker_plus_suffixes
        suffixes:
        - .PAR
        - .PA
        - ""
      quote:
        base_url: https://www.alphavantage.co
        path: /query
        query:
          function: GLOBAL_QUOTE
          symbol: '{symbol}'
          apikey: '{api_key}'
        symbol_path:
        - Global Quote
        - 01. symbol
        price_path:
        - Global Quote
        - 05. price
        trading_day_path:
        - Global Quote
        - 07. latest trading day
        prev_close_path:
        - Global Quote
        - 08. previous close
        change_path:
        - Global Quote
        - 09. change
        change_percent_path:
        - Global Quote
        - 10. change percent
        change_percent_strip_percent: true
  websites:
    abcbourse:
      url: https://www.abcbourse.com/cotation/{key}
      key_strategy:
        candidates:
```

```
        - template: '{ticker}p'
        - template: '{isin}'
      regex_price: "id=\"lastcx\">(?<price>\\d+(?:[.,]\\d+)*)\\s*&#x20A\
        C;"
      regex_variation: "id=\"varcx\"\\s+class=\"quote_(up|down)b\">(?<v\
        ariation>(&#x2B;|&#x2D;|\\+|-)?\\d+(?:[.,]\\d+)*)%"
    tradingsat:
      url: https://www.tradingsat.com/{key}
      key_strategy:
        candidates:
        - template: '{name_slug}-{isin}'
          vars:
            name_slug:
              from: name
              ops:
              - downcase
              - gsub:
                  pattern: \.
                  replace: ""
              - gsub:
                  pattern: ' '
                  replace: '-'
      regex_price: "class=\"price\">(?<price>\\d+(?:[,.]\\d+)*)\\s*€"
      regex_variation: "class=\"variation var-(down|neutral|up)\">(?<va\
        riation>[+-]?\\d+(?:[,.]\\d+)*)\\s*%"
    boursorama:
      url: https://www.boursorama.com/cours/{key}/
      key_strategy:
        candidates:
        - template: 1rP{ticker}
        - template: 1rEP{ticker}
        - template: '{ticker}'
          ops:
            ticker:
            - upcase
            - trim
      regex_price: "c-faceplate__info.+?price.+?ist-last>(?<price>\\d+(\
        ?:[,.]\\d+)*)<"
      regex_variation: "c-faceplate__info.+?fluctuation.+?ist-variation\
        >(?<variation>[+-]?\\d+(?:[.,]\\d+)*)%"
fees:
  rate: 0.0035
  min: 0.0
watchrate: 10.0
diagnostic_steps: 3
quotlife: 240.0
tickcheck: true
trading:
  capital: 20000
money:
  risk_rate_min: 0.005
  risk_rate_max: 0.05
  target_coeff_min: 0.5
  target_coeff_max: 5.0
  steps_down_min: 1
  steps_down_max: 5
simulation:
  capital: 20000
  distance: 0.2
  risk_rate: 0.015
```